MARQ: A Multi-Agent Rainbow Deep Q-Networks Framework using AAREN for Mastering Imperfect Information Games

James Gabriel P. Mabagos
Bachelor of Science in Computer Science

Mark Lawrence M. Quibot
Bachelor of Science in Computer Science

Senior Thesis submitted to the faculty of the
Department of Computer Science
College of Computer Studies, Ateneo de Naga University
in partial fulfillment of the requirements for their respective
Bachelor of Science degrees

Project Advisor: Marianne A. Tolentino
Estrella H. Montealegre
Michelle Elija B. Santos
Ian Peter L. Lastimosa

September 5 2025
Naga City, Philippines

Keywords: Imperfect Information, DQN, Multi-Agent

Copyright 2025, James Gabriel P. Mabagos and Mark Lawrence M. Quibot

The Senior Project entitled

MARQ: A Multi-Agent Rainbow Deep Q-Networks Framework using AAREN for Mastering Imperfect Information Games

developed by

James Gabriel P. Mabagos

Bachelor of Science in Computer Science

Mark Lawrence M. Quibot

Bachelor of Science in Computer Science

and submitted in partial fulfillment of the requirements of their respective Bachelor of Science degrees has been rigorously examined and recommended for approval and acceptance.

Estrella H. Montealegre
Panel Member
Date signed: _____

Michelle Elija B. Santos
Panel Member
Date signed: _____

Ian Peter L. Lastimosa
Panel Member
Date signed: _____

Marianne A. Tolentino
Project Advisor
Date signed: _____

The Senior Project entitled

MARQ: A Multi-Agent Rainbow Deep Q-Networks Framework using AAREN for Mastering Imperfect Information Games

developed by

James Gabriel P. Mabagos

Bachelor of Science in Computer Science

Mark Lawrence M. Quibot

Bachelor of Science in Computer Science

and submitted in partial fulfillment of the requirements of their respective Bachelor of Science degrees is hereby approved and accepted by the Department of Computer Science, College of Computer Studies, Ateneo de Naga University.

Lowie Vincent S. Bisana MIT

Chair, Department of Computer Science

Date signed: _____

Joshua C. Martinez MIT

Dean, College of Computer Studies

Date signed: _____

Declaration of Original Work

We declare that the Senior Project entitled

MARQ: A Multi-Agent Rainbow Deep Q-Networks Framework using AAREN for Mastering Imperfect Information Games

which we submitted to the faculty of the

Department of Computer Science, Ateneo de Naga University

is our own work. To the best of our knowledge, it does not contain materials published or written by another person, except where due citation and acknowledgement is made in our senior project documentation. The contributions of other people whom we worked with to complete this senior project are explicitly cited and acknowledged in our senior project documentation.

We also declare that the intellectual content of this senior project is the product of our own work. We conceptualized, designed, encoded, and debugged the source code of the core programs in our senior project. The source code of third party APIs and library functions used in our program are explicitly cited and acknowledged in our senior project documentation. Also duly acknowledged are the assistance of others in minor details of editing and reproduction of the documentation.

In our honor, we declare that we did not pass off as our own the work done by another person. We are the only persons who encoded the source code of our software. We understand that we may get a failing mark if the source code of our program is in fact the work of another person.

James Gabriel P. Mabagos
4 - Bachelor of Science in Computer Science

Mark Lawrence M. Quibot
4 - Bachelor of Science in Computer Science

This declaration is witnessed by:

Marianne A. Tolentino
Project Advisor

MARQ: A Multi-Agent Rainbow Deep Q-Networks Framework using AAREN for Mastering Imperfect Information Games

by

James Gabriel P. Mabagos and Mark Lawrence M. Quibot

Project Advisor: Marianne A. Tolentino

Department of Computer Science

ABSTRACT

This research presents MARQ, a Multi-Agent Rainbow Deep Q-Networks, designed to master imperfect information games through the board game Stratego. MARQ combines a Rainbow Deep Q-Network with league-based self-play training to address the large state space of Stratego and its hidden information. The architecture integrates an Attention as Recurrent Neural Network (AAREN) module that produces learned embeddings from observed opponent behavior. AAREN is trained to predict the identity of revealed pieces, and the Q-network reads its embeddings as inputs for implicit piece identity inference. The system employs Distributional Reinforcement Learning with Noisy Networks to balance strategic unpredictability and stability. The methodology involves developing a game environment and training the AI agent through multi-channel state representations, incorporating AAREN-generated embeddings for hidden information modeling and weighted piece values computed through systematic power analysis to optimize decision making under uncertainty. A structured participant study introduces enthusiasts without prior Stratego experience to create a synchronized learning environment, combining quantitative metrics including win rate percentage, average game length, and decision-making speed, with qualitative usability assessments across performance, reliability, and applicability dimensions. Through this study MARQ aims to deliver strategic decisions comparable to human opponents.

ACKNOWLEDGEMENTS

TABLE OF CONTENTS

1 Introduction **1**

1.1 Project Context 1

1.2 Purpose and Description 2

1.2.1 Research Questions 2

1.3 Objectives 3

1.4 Scope and Limitation 3

1.5 Significance of the Study 3

1.6 Definition of Terms 4

2 Review of Related Literature **7**

2.1 Artificial Intelligence in Imperfect Information Games 7

2.1.1 Student of Games 8

2.1.2 Deep Q-Networks and Dueling Architectures 8

2.1.3 Multi-Agent Reinforcement Learning 8

2.1.4 Counterfactual Regret Minimization 9

2.1.5 Opponent Modeling in Imperfect-Information Games 9

2.2 RNN 10

2.2.1 History Aggregation and State Representation Learning 10

2.2.2 LSTM 11

2.2.3 Aaren 11

2.2.4 Residual Networks and Self-Attention in Board Games 11

2.3 Stratego 12

2.3.1 Deep Reinforcement Learning in Stratego . . . 12
2.4 Other Applications . . . 13
2.5 Application to Other Game Environments . . . 14

3 Theoretical Frameworks **15**
3.1 Stratego Game Domain . . . 15
3.2 MARQ Framework Overview . . . 16
3.3 MARQ Policy Definition . . . 17
3.3.1 Core Policy Framework . . . 17
3.3.2 Implicit Representation Optimization . . . 19
3.4 Deep Q-Network Architecture . . . 20
3.4.1 Rainbow DQN Components . . . 20
3.4.2 Training Stability via Regularization . . . 22
3.5 AAREN and Learned Embedding System . . . 22
3.5.1 Recurrent Attention Computation . . . 22
3.5.2 Embedding Updates . . . 23
3.6 Multi-Agent Learning and Strategy Mixing . . . 23
3.6.1 League-Based Training Architecture . . . 23
3.6.2 Strategy Mixing for Exploitation Resistance . . . 24
3.6.3 Reward Structure . . . 24
3.7 Game Environment and State Management . . . 25
3.7.1 Information Set Management . . . 25
3.7.2 Temporal State Evolution . . . 25
3.7.3 Valid Action Filtering . . . 25
3.8 MARQ Move Selection and Decision-Making . . . 25
3.9 Generalizability . . . 26
3.9.1 Mathematical Convergence Properties . . . 26
3.10 Algorithmic Evolution and Architecture Ablations . . . 28
3.10.1 Value Optimization and Exploration. . . . 28
3.10.2 History and Memory Representation. . . . 29
3.10.3 Architectural Refinements. . . . 29
3.10.4 Strategic Initialization and Efficiency. . . . 29

4 Methodology **30**

4.1 Project Planning . . . 30

4.2 System Development . . . 30

4.2.1 Piece Value Computation . . . 31

4.2.2 AAREN and Game History . . . 32

4.3 Model Training . . . 33

4.3.1 Test-Time Search . . . 38

4.3.2 Deployment . . . 39

4.4 Game Environment . . . 41

4.4.1 Evaluation . . . 41

5 Results and Discussion **42**

5.1 Normal DQN Performance Ceiling . . . 42

5.1.1 Win Rate Plateau . . . 42

5.1.2 Loss–Performance Decoupling . . . 43

5.1.3 Q-Value Overestimation . . . 43

5.1.4 LSTM Prediction Accuracy Ceiling . . . 44

5.1.5 Opponent-Specific Performance Asymmetry . . . 44

5.1.6 Draw Dominance . . . 44

5.2 Behavioral Patterns from Training Logs . . . 45

5.2.1 Passive Shuffling Convergence . . . 45

5.2.2 Opponent-Specific Adaptation Failure . . . 45

5.2.3 Loss-Reward Divergence . . . 46

5.2.4 LSTM Accuracy . . . 46

5.3 Draw Dominance . . . 46

5.4 Architectural Comparison: Unified versus Dual-Model Pipeline . . . 47

5.5 Architectural Bottleneck Analysis . . . 48

5.5.1 Q-Value Overestimation . . . 48

5.5.2 Scalar Value Collapse . . . 48

5.5.3 LSTM Information Bottleneck . . . 49

5.5.4 Exploration Collapse . . . 49

5.5.5 Sparse Reward Propagation . . . 50

5.6 Empirical Validation: Rainbow DQN with AAREN . . . 50
5.6.1 Win Rate . . . 51
5.6.2 Draw Elimination . . . 51
5.6.3 Loss–Performance Correlation . . . 52
5.6.4 Sample Efficiency . . . 52
5.6.5 AAREN and LSTM Accuracy . . . 53
5.6.6 Learned Exploration Active . . . 53
5.6.7 Extended Training at 75,000 Episodes . . . 53
5.7 Evidence for Architectural Necessity . . . 55
5.8 Supporting Evidence from Related Work . . . 56
5.9 Implications for MARQ . . . 56

6 Conclusion and Recommendations **58**
6.1 Conclusion . . . 58
6.2 Recommended Architectural Enhancements . . . 59
6.2.1 Value Estimation . . . 59
6.2.2 LSTM to AAREN Transition . . . 60
6.2.3 Exploration Recovery . . . 60
6.2.4 Long-Range Credit Assignment . . . 61
6.3 Computer Science Contributions . . . 61
6.3.1 Five-Phase Curriculum Design . . . 61
6.3.2 Embedding Snapshots for Replay Compatibility . . . 61
6.3.3 Expectamax Search Integration . . . 62
6.3.4 Potential-Based Reward Shaping with Phase Annealing . . . 62
6.4 Limitations and Future Work . . . 62

A Gantt Chart **64**

B System UI and Training **67**
B.1 Interpretability Dashboard . . . 74
B.2 Architecture Comparison Training Charts . . . 76
B.3 AI Transparency, Inclusion, and Originality of Intent . . . 83

C Implementation Log and Lessons Learned **84**

C.1 Narrowing the C51 support to ±2 . . . 84

C.2 Measuring the average Q-value on the actions taken . . . 85

C.3 Diagnosing a flat value function . . . 85

C.4 Making the training loop faster . . . 86

C.5 Removing the curriculum's environment changes . . . 87

C.6 Cold start from demonstrations . . . 87

C.7 The formal_3 run and the start of formal_4 . . . 89

C.8 A material-only potential (formal_5) . . . 93

C.9 Asynchronous learner . . . 95

C.10 What the falling loss of formal_4 did and did not show . . . 96

C.11 Correcting the claims of the thesis chapters . . . 98

C.12 Replay priorities from the cross-entropy . . . 99

C.13 An audit of the rest of the Rainbow learner . . . 100

C.14 What AAREN can add to the observation . . . 101

C.15 Moved-piece planes and three learner fixes (formal_6) . . . 103

C.16 Why the network does not learn the rank of a revealed piece . . . 106

C.17 A mean loss, revealed attacks in the batch, and the opponent pool (formal_7) . . . 108

C.18 What the agent has to learn, and in what order . . . 111

C.19 A five-fold material signal and stages without a cap (formal_8) . . . 113

C.20 Engineering notes . . . 115

LIST OF FIGURES

2.1 Stratego Game board. 13
3.1 Stratego pieces. 16
3.2 Stratego Game board. 17
4.1 MARQ framework architecture. 31
4.2 MARQ training architecture. 33
4.3 Dueling network heads. 34
4.4 ResNet backbone. 34
4.5 79-channel input representation. 35
4.6 Game sequence diagram. 40
5.1 Cumulative win count and win rate over 37,213 episodes. The win rate plateaus early and remains flat, indicating policy stagnation. 43
5.2 Policy loss (TD error). 43
5.3 Average Q-values climb. 44
5.4 Piece-identity prediction accuracy. 45
A.1 Timeline: August 1 - September 5 65
A.2 Timeline: September 11 - November 30 66
A.3 Timeline: October 27 - January 27 66
A.4 Timeline: December 1 - February 14 66
A.5 Timeline: February 15 - May 4 66
B.1 Game Menu 68
B.2 Game Rules and Instructions 68
B.3 Setup Phase 69
B.4 Game Start 69

B.5 Change image of pieces to numbers 70
B.6 Battle and History 70
B.7 Victory Screen 71
B.8 Training Progress: Episode 10,000 72
B.9 Training Progress: Episode 20,000 73
B.10 Training Progress: Episode 34,000 74
B.11 Interpretability dashboard snapshots. 75
B.12 DQN+LSTM Training Progress at 15,302 Episodes 76
B.13 DQN+LSTM Additional Metrics at 15,302 Episodes 77
B.14 Rainbow DQN+AAREN Training Progress at 9,778 Episodes 78
B.15 Rainbow DQN+AAREN Additional Metrics at 9,778 Episodes 79
B.16 AAREN End-to-End Training Metrics at 9,778 Episodes 80
B.17 Rainbow DQN+AAREN Extended Training Progress at 75,047 Episodes 81
B.18 Rainbow DQN+AAREN Extended Additional Metrics at 75,047 Episodes 82
B.19 AAREN End-to-End Training Metrics at 75,047 Episodes 83

LIST OF TABLES

3.1 Key symbols and descriptions. 18
4.1 Computed piece values with combat and survival metrics. 32
4.2 Strategic ability bonuses for special-capability pieces. 32
4.3 State representation input channels. 36
5.1 Normal DQN performance after 37,213 episodes of self-play training. 42
5.2 Dual-model versus unified architectural pipelines. 47
5.3 Self-play performance comparison between Vanilla DQN with LSTM and Rainbow DQN with AAREN. 51
5.4 Rainbow DQN with AAREN extended training (75k episodes) against heuristic opponents. 54
5.5 MARQ title components mapped to normal DQN failure modes and the Rainbow DQN evidence so far. 55
5.6 Observed limitations mapped to theoretical causes, MARQ components, and validation status. 57

Chapter 1

Introduction

1.1 Project Context

Games have long served as benchmarks for measuring Artificial Intelligence (AI) progress [49]. Many real-world problems, from military tactics to multiplayer games, require intelligent decision-making in environments where all information is not available. A common example is the "fog of war" mechanic found in many multiplayer games [64], where the game environment is not fully revealed to any single player, placing these games in the category of imperfect information games [52]. In such games, players must develop strategies without full knowledge of the game's environmental state [64]. Determining the optimal strategy under these conditions remains a standing challenge for Artificial Intelligence [36].

This study focuses on mastering imperfect information games using Multi-Agent Reinforcement Learning (MARL) with Deep Q-Networks (DQN). In MARL, multiple agents learn and adapt in the same environment, making each decision based on its own observations and rewards [39]. With DQN, the agents use deep neural networks to approximate the Q-function that maps state-action pairs to expected reward. This study introduces the MARQ (Multi-Agent Rainbow Deep Q-Networks) framework, an extension of the DQN architecture with Rainbow enhancements: distributional value estimation (C51) and noisy exploration networks [26]. An AAREN (Attention as Recurrent Neural Network) module [22] processes extended observation sequences and produces learned embeddings for implicit inference of hidden opponent pieces through end-to-end training. League-based self-

play training [46] introduces agents to different historical opponents to prevent similar strategies. Stratego is a good example of a test environment for our framework, agents compete against each other and must develop strategies through repetition of experience while reasoning under uncertainty about the enemy's setup and the environments hidden states.

1.2 Purpose and Description

This research evaluates whether Multi-Agent Reinforcement Learning (MARL) with Deep Q-Networks (DQN) can produce agents that can play imperfect information games at a competitive level. The board game Stratego serves as the test domain because it combines hidden piece identities, a large state space ($\approx 10^{535}$), and long game horizons. Existing analytical tools for Stratego have little to no support for the deeper tactical reasoning that we want to achieve.

The MARQ framework was developed to test whether targeted architectural changes, specifically distributional value estimation, attention-based history encoding, and learned exploration, can overcome the performance ceiling observed in a standard DQN baseline. The experimental results, detailed in later chapters, show that these changes doubled the win rate and reduced the draw rate from 50% to under 7%.

1.2.1 Research Questions

This study is guided by three main research questions:

1. How does the MARQ framework perform in terms of convergence speed and win-rate against heuristic baselines in a partially observable environment?
2. Does the MARQ framework demonstrate statistically significant superiority over standard DQN and rule-based agents across varying opponent difficulty tiers?
3. What is the impact of integrating the AAREN module on the agent's ability to infer hidden opponent piece configurations compared to approaches without explicit belief state modeling?

1.3 Objectives

The main objective of this study is to develop and evaluate the MARQ framework for Stratego, where agents must operate under hidden state variables. The specific objectives are:

- Identify and evaluate reinforcement learning algorithms for competitive play in Stratego.
- Design and implement a game environment for Stratego
- Build and implement a training model using Multi-Agent Reinforcement Learning with Deep Q-Networks and AAREN.
- Train and optimize the AI agent to respond to varying game states, hidden information, and opposing strategies.

1.4 Scope and Limitation

The study is limited to the development of an AI agent for imperfect information games, with Stratego as the test environment. The scope includes the design and implementation of Stratego as a game environment, the development of a training framework using Multi-Agent Reinforcement Learning (MARL) with Deep Q-Networks (DQN) and AAREN, and the training and optimization of the AI agent to adapt to varying game states. Agent performance is evaluated through agent-vs-agent testing and ablation analysis within the Stratego environment. Training is limited to the personal devices of the researchers and does not rely on cloud-based computing resources.

1.5 Significance of the Study

Current imperfect information game systems either require large computational resources or use model-free approaches with limited strategic depth. The MARQ framework contributes a specific combination of techniques: implicit opponent modeling through AAREN-generated embeddings, distributional value estimation via C51, and learned exploration via Noisy Networks. The experimental results show that this combination resolved the scalar value collapse and exploration death observed in a standard DQN baseline, producing an agent that wins 49.88% of self-play games compared to 22% for the baseline. The five-phase curriculum and gradient bridge technique apply to other partially observable or adversarial domains that use recurrent encoders with experience replay.

1.6 Definition of Terms

AAREN (Attention as Recurrent Neural Network)

A neural architecture that combines the parallel training efficiency of Transformers with the sequential processing of RNNs. In the MARQ framework, AAREN processes observation sequences to produce learned embeddings that encode implicit piece identity inference.

Counterfactual Regret Minimization (CFR)

An iterative algorithm that approximates Nash equilibrium by minimizing regret, widely applied in imperfect information games such as Poker and Stratego.

Deep Q-Networks (DQN)

A reinforcement learning algorithm that combines Q-learning with deep neural networks, enabling agents to approximate optimal decision-making in complex environments.

Distributional Reinforcement Learning

An approach that models the full distribution of returns rather than just the expected value. The MARQ framework uses C51 (Categorical DQN) with 51 atoms to capture uncertainty in value estimates.

Fog of War

A game mechanic used in strategy games that conceals parts of the map or opponent actions, requiring players to utilize prediction and information-gathering strategies to reduce uncertainty.

Imperfect Information Games

Games where players have incomplete knowledge of the current game state, requiring them to make strategic decisions under uncertainty (e.g., Stratego, Poker, Multiplayer Online Battle Arenas).

Information Set Monte Carlo Tree Search (IS-MCTS)

A search algorithm for imperfect information games that samples consistent game states from belief

distributions and performs tree search on information sets rather than individual states.

League Training

A self-play training paradigm where agents compete against a diverse pool of historical opponents, preventing strategy cycling and ensuring robust generalization across different playing styles.

Multi-Agent Reinforcement Learning (MARL)

An extension of reinforcement learning in which multiple agents interact within the same environment, learning coordinated or competitive strategies through simultaneous training.

Nash Equilibrium

A game-theoretic concept where no player can gain an advantage by unilaterally changing their strategy, often used as a benchmark for decision-making in imperfect information games.

Perfect Information Games

Games in which all players have complete knowledge of the current game state and available moves, such as Chess and Go.

AAREN Embeddings

Embedding vectors produced by the AAREN module from observed opponent action sequences.

Rainbow DQN

An enhanced DQN architecture that combines multiple improvements including distributional RL, noisy networks for exploration, and dueling network architecture for improved stability and sample efficiency.

Recurrent Neural Network (RNN)

A type of neural network designed to process sequential data by maintaining memory of previous inputs, useful in imperfect information games for recalling past moves or revealed information.

Reinforcement Learning (RL)

A machine learning paradigm where agents learn optimal strategies through trial-and-error interactions with the environment, guided by rewards and penalties.

Stratego

A two-player strategy board game characterized by hidden information and asymmetric piece abilities, where the objective is to capture the opponent's flag or render them immobile.

Chapter 2

Review of Related Literature

2.1 Artificial Intelligence in Imperfect Information Games

Games can be classified as an imperfect information game by having incomplete information about the current game state for both players [49, 39]. An example would be in MOBAs (Multiplayer Online Battle Arena), the fog of war mechanic hides portions of the map, including objectives and enemy positions, from both teams. By placing a tool, such as a ward in League of Legends or an observer in Dota 2, the player gains a temporary vision in the map for strategic planning [15]. This feature engages players to create strategic reasoning under certainty, utilizing prediction, coordination, and resource management to gain informational advantages over opponents. Games have a history of being a metric on how AI progresses and performance in the current time [49, 29]. The case is the same for both perfect information and imperfect information games [49]. For perfect information games, like chess, they primarily use Alpha-Beta pruning, a search algorithm that uses trees to explore possible chess moves and prunes the branches that will not affect the final decision [41]. In imperfect information games, a search algorithm like Alpha-Beta Pruning would be difficult to implement because the algorithm is made for perfect information games [48]. A popular algorithm that is used in imperfect information games would be Monte Carlo Tree Search with Reinforcement Learning because MCTS has integration with neural network systems like AlphaZero [44, 10]. In terms of reinforcement learning, the training focuses on trial and error to provide the best possible decision [67].

2.1.1 Student of Games

A unified learning algorithm that can support both perfect information and imperfect information is Student of Games. Combining self-play learning, strategic search, and principles from game theory [49]. SoG achieved strong performance in Chess and Go in perfect information games and in imperfect information games like heads-up no-limit Texas hold 'em poker, and defeated the state-of-the-art agent in Scotland Yard. When comparing to MARL, SoG supports and has been tested in both game types and uses game theoretic reasoning, which computes or approximates the Nash equilibria. MARL with DQN uses trial and error learning and self-play learning, which relies on the exploration of the environment. MARL has also been tested in complex video games [39], unlike SoG, which has not been tested in video games.

2.1.2 Deep Q-Networks and Dueling Architectures

Deep Q-networks (DQN) and their variants have become a standard framework for decision-making in complex environments with high-dimensional observations. Hessel et al. (2021) combined several independent improvements to DQN, including double Q-learning, prioritized replay, dueling networks, and noisy nets, into a unified agent called Rainbow [26]. Rainbow achieved state-of-the-art performance on competitive benchmarks, and the ablation study showed that each component contributed to the overall gain; removing any single one degraded performance.

A key architectural component of Rainbow is the dueling network architecture, proposed by Wang et al. (2016) and detailed in modern surveys [26, 52]. The dueling design splits the Q-function into a state-value stream and an advantage stream, improving policy evaluation in environments with many similar-valued actions. MARQ uses the same split by feeding its shared ResNet-based representation into dueling value and advantage streams. In the experimental results, this decomposition contributed to the loss-performance coupling observed in the Rainbow agent ($r = -0.787$), where the normal DQN showed no such correlation ($r = 0.051$).

2.1.3 Multi-Agent Reinforcement Learning

Reinforcement Learning can solve complex problems through trial and error, by learning from the environment to be able to provide optimal decisions [39]. This can be the case for Single-Agent Reinforcement Learning(SARL). SARL has its limits in solving many real-world problems. Multi-

Agent Reinforcement Learning(MARL) was introduced as a solution for the challenges of SARL, by having multiple agents in an environment that interact with each other, the decisions they could make would be optimized [39]. Optimized, meaning that these agents go through trial and error together to make the best decision given the reward. As for the implementation of this in Stratego, two agents would compete with each other to know the optimal decisions given the environment, which has imperfect information and opposing strategies. As MARL has its advantages, it also has challenges. The first would be Non-Stationarity, and the second is scalability, as for non-stationarity, each agent learns simultaneously, meaning the environment constantly changes [13]. For scalability, as more agents are being used, the computational power rises, making it expensive [37, 13]. The Ataraxos system [50] addressed non-stationarity through KL-divergence regularization, constraining policy updates toward both exploratory and stable anchors with time-varying coefficients. Ataraxos reached superhuman performance in no-press Diplomacy using this approach. MARQ adopts a similar dynamic damping schedule for its own training stability.

2.1.4 Counterfactual Regret Minimization

Counterfactual Regret Minimization(CFR) is an iterative algorithm that approximates the Nash equilibria commonly used on imperfect information games like poker and Stratego [37, 60]. The Nash equilibria are what CFR approximates to make the best possible decision with minimal regret. Regret in this scenario is the what-ifs of the algorithm if the algorithm chooses the other decision. For CFR to make decisions, it repeatedly minimizes regret to approximate the Nash equilibria [37]. Other works have made variations of the base CFR, showing improved convergence rate, but require a significant amount of effort by researchers. A framework is proposed named AutoCFR, instead of relying on manually crafted algorithms, it automatically designs new CFR variants using an outer loop to search over algorithm designs and an inner loop to test them on equilibrium finding tasks [59].

2.1.5 Opponent Modeling in Imperfect-Information Games

In imperfect-information games, an agent's optimal strategy depends not only on the environment but also on the behavior of the opponent. Classical opponent modeling often builds models from observed action frequencies, and computes best responses in a game-theoretic framework. Li et al. (2024) formalize opponent-model search in games with incomplete information, providing algorithms

to compute optimal or robust strategies given various forms of opponent knowledge [36].

In the deep RL setting, modern approaches encode observations of opponents directly into the network trunk, jointly learning a policy and an opponent model without explicitly predicting actions [52]. This shows that learning implicit opponent representations from interaction traces can be more effective than handcrafted models, especially in high-dimensional state spaces. Our AAREN module follows this tradition of implicit opponent modeling but is specialized to board games with hidden piece identities. Instead of maintaining explicit probability distributions over opponent piece types, AAREN aggregates observed opponent action sequences into dense embedding vectors. These vectors are then concatenated as additional input channels to the Q-network, allowing the agent to reason about the opponent's likely pieces and strategy in a learned representation space.

2.2 RNN

Recurrent Neural Network is a type of Neural Network designed to process sequential data. Unlike common Neural Networks, RNNs have a memory because the information passed to the next step is based on the memory from the previous step. RNN can be applied to imperfect information games such as Stratego because of its memory. By using the memory to remember the pieces that have been revealed, the decision could be optimized [44].

2.2.1 History Aggregation and State Representation Learning

Learning from histories is a central challenge in partially observable and non-Markovian environments. Wang et al. (2026) show how to systematically construct non-Markovian decision processes by aggregating interaction histories into state representations [54]. Their work demonstrates that explicit history modeling through learned aggregators improves agent performance in tasks with complex temporal dependencies.

Representation learning for RL has also explored the use of embeddings to generalize across states and goals. Echchahed and Castro (2025) find in their survey that separating dynamics from rewards enables policy transfer across tasks through learned representations [19]. The AAREN embeddings in MARQ are a form of history-dependent state representation: they encode the opponent's action history into a compact vector that augments the board state. The embeddings are learned jointly with the Q-network rather than handcrafted as belief distributions, and the network infers implicit

piece identities from observed behavior. The experimental results (Chapter 5) show that this joint training produced a 6.5 percentage point accuracy improvement over the LSTM baseline.

2.2.2 LSTM

Long Short-Term Memory is a popular RNN variant that has the capabilities of RNN but also processes data with long-term dependencies [2]. LSTM mitigates the vanishing gradient problem that RNN faces. The vanishing gradient problem is basically as in the name, vanishing gradient, as color in a gradient, the message in this case starts strong and gradually fades away when it passes through each layer, making the learning weak or close to zero. LSTM can be used to solve imperfect information games because of its capability to process data with long-term dependencies while having the memory of RNN.

2.2.3 Aaren

Aaren, known as Attention as a recurrent neural network, combines the parallel training efficiency of Transformers with the streaming update efficiency of RNNs [22]. Aaren, like transformers, can be trained in parallel as well as RNN, can process sequential data with memory. In the study, Aaren was tested in event forecasting, time series forecasting, and classifications. Aaren achieved similar accuracy to Transformers but was more time and memory-efficient.

2.2.4 Residual Networks and Self-Attention in Board Games

Residual networks (ResNets) have become a common backbone in modern board-game RL systems. AlphaZero-style agents for Go, chess, and Stratego use a deep tower of residual convolutional blocks as the network trunk, and this depth is necessary for capturing spatial dependencies on the board [10]. Modern architectures often mix these residual blocks with self-attention layers to bridge local and global knowledge, and the combination has improved playing strength in board games by handling long-range patterns that pure convolutions miss [4].

MARQ adopts a ResNet backbone with spatial self-attention for the same reason: each board position attends to all other positions, capturing long-range piece relationships. The architecture adds a dueling Q-network head and the AAREN opponent embedding on top of this backbone. In the experimental evaluation, the 6-block ResNet with self-attention provided full receptive field

coverage of the 10x10 board while keeping inference latency low enough for real-time play.

2.3 Stratego

The origin of Stratego can be traced to a traditional board game called Jungle, where instead of human pieces instead it used animals. In the jungle, the pieces are not hidden, making it a perfect information game. Stratego is a two-player board game where players each have a 40-piece army. In the tabletop version of Stratego, one player is assigned to the blue army and the other player is assigned to the red army. Each army has a flag assigned to it, which cannot be moved when the game starts. There are a total of 80 troop pieces, while the total number of variations is 12. The pieces rank range from 1 to 10, and a piece that's not numbered is a bomb and the flag. The player also has time to move pieces before the game starts to make use of the fog of war and create creativity and optimal positions on the board. As the pieces have ranks, the higher the number, the higher the power. There is an instance where the lower-ranked can defeat a higher-ranked ranked, which is the piece that has a rank of one, named the spy, can defeat the strongest piece that has a rank of ten, named the marshal, if the spy attacks first. As for the bombs, almost all pieces are vulnerable except the miner, the only one who can defuse the bomb. And the pieces that cannot be moved are the bomb and the flag. Another piece that has a special ability is the scout; it can move either horizontally or vertically, not just one square. If both pieces have the same number, and the piece attacks, both of them would be eliminated. There are multiple win conditions in Stratego. The obvious one would be capturing the flag, another one would be that the opponents have no mobile pieces, another would be that there are no miners that can defuse the bomb to capture the flag, and lastly, the opposing player had the time run out.

2.3.1 Deep Reinforcement Learning in Stratego

Stratego is a popular imperfect-information board game where players must reason about hidden piece identities over long time horizons. Pérolat et al. (2022) demonstrated with DeepNash that a model-free multiagent RL approach can reach human expert level in Stratego, using deep convolutional networks trained with game-theoretic regularization [46]. Their architecture relies on ResNet-style networks to process board features under imperfect information. Subsequent work combined self-play reinforcement learning with test-time search and reached superhuman performance [50].

MARQ uses a similar deep ResNet backbone but adds explicit opponent modeling via AAREN embeddings. The agent conditions its decisions on the opponent's observed action history, and the experimental results show that this conditioning contributed to the win rate doubling from 22% (normal DQN) to 49.88% (Rainbow with AAREN) in self-play.

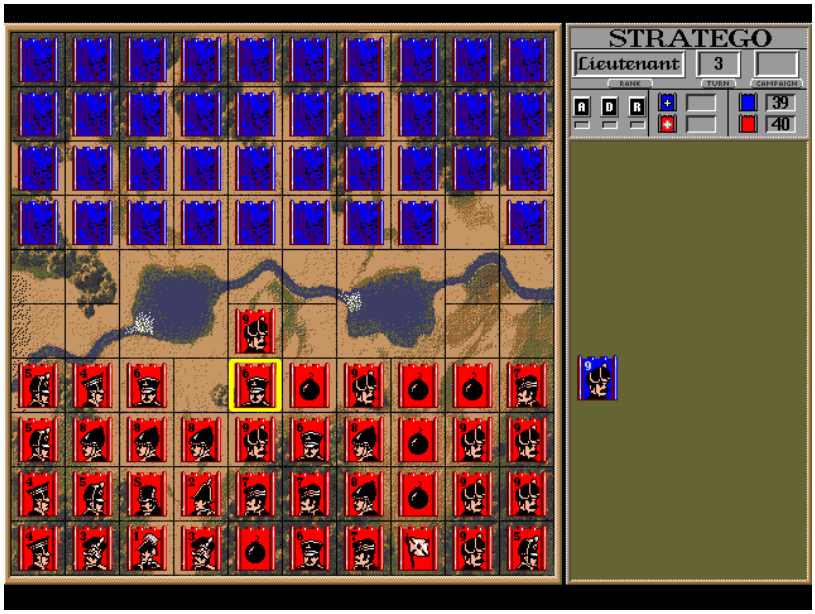


Figure 2.1: Stratego Game board.

2.4 Other Applications

An extension of MARL with DQN was demonstrated in robotics. In these environments, robots are trained to make decisions based on rewards. The study assigned different energy costs to each robotic arm, allowing the robot to learn how to adjust its movement based on cost, using less energy when the reward is small and exerting more effort if the reward is appropriate [40]. MARL with Deep Q-learning was also used in traffic signal control. Applied a cooperative multi-agent DQN framework to manage six interconnected intersections, where each intersection acted as an independent learning agent. Using the SUMO simulator, each agent selected signal phases to minimize congestion and improve overall traffic flow. Their approach introduced a reward function based on the standard deviation of stopping vehicles across lanes, showing balanced lane utilization rather than simply

reducing queue lengths. This design showed that MARL with DQN can enhance not only traffic efficiency but also environmental sustainability by reducing emissions and fuel consumption. In businesses in developing countries, it is essential that growth should be the top priority; however, this creates a problem in increasing taxation for improving infrastructure to further support the current population. Using MARL, we can create a model system where multi-agent RL acts as different parties that discover the optimal tax policies without needing to rely on traditional economic models [66].

2.5 Application to Other Game Environments

Although this study focuses on Stratego, the core MARQ components (distributional Q-learning, attention-based history aggregation, and league self-play) are not Stratego-specific. Other game domains with hidden information could use the same architecture. Sports management games such as Football Manager involve sequential decisions under uncertainty about player development and market dynamics. Real-time games such as Clash Royale involve hidden opponent hands and resource levels. These domains share the partial observability structure that MARQ was designed for, though applying the framework to them would require domain-specific state representations and reward functions. No experiments were conducted outside Stratego in this study.

Chapter 3

Theoretical Frameworks

3.1 Stratego Game Domain

Stratego is an imperfect information board game with European roots, popularized in the Netherlands in the 1940s and released in North America in 1961 [46]. The game is played on a 10x10 board that includes two 2x2 "lake" areas in the center, which are impassable. Each player commands an army of 40 pieces of varying ranks, and during the game, the identity and rank of the pieces are kept secret from the opposing player. The objective is to capture the opponent's flag or eliminate all of their movable pieces.

Each army consists of 12 different types of pieces, with ranks ranging from 1 (the Spy) to 10 (the Marshal), alongside Bombs and a Flag. Higher-ranked pieces defeat lower-ranked ones during combat, except in special situations: the Spy can defeat a Marshal if attacking first, the Miner is the only piece that can defuse Bombs, and Scouts can move multiple squares in a straight line. There are 3 ways to win the game capturing the flag, immobilizing all movable enemy pieces, or forcing the opponent to run out of time.

The high theoretical state space of Stratego shows how difficult the game is to solve:

$$|\mathcal{S}| = \binom{40}{1, 1, 2, 3, 4, 4, 4, 5, 8} \times 10^{10} \times 2^{40} \approx 10^{535} \tag{3.1}$$

Perolat et al. [46] established the permutation for game state. The agent must use reasoning with limited information rather than trying to look at every possible outcome, making the computation



Figure 3.1: Stratego pieces.

intractable.

3.2 MARQ Framework Overview

The MARQ framework is a Multi-Agent Rainbow Deep Q-Networks system that trains agents through self-play and deep reinforcement learning [10]. Agents learn by competing against many different opponents, including their own previous versions and exploratory variants, within a league-based self-play system.

The central problem is reasoning about hidden information without enumerating every possible state. The number of possible game states exceeds 10^{535}, and initial setups reach 10^{66} per player [46]. MARQ handles this scale through implicit belief representations, probabilistic reasoning, and pattern recognition. Deep reinforcement learning estimates values while attention mechanisms process game history and mixed strategies prevent exploitation, all within real-time computational constraints.

The architecture connects these components in a pipeline. The AAREN module takes raw game positions and move sequences and produces learned embeddings that encode opponent piece identity inferences. These embeddings are the input for all subsequent reasoning. A league-based training

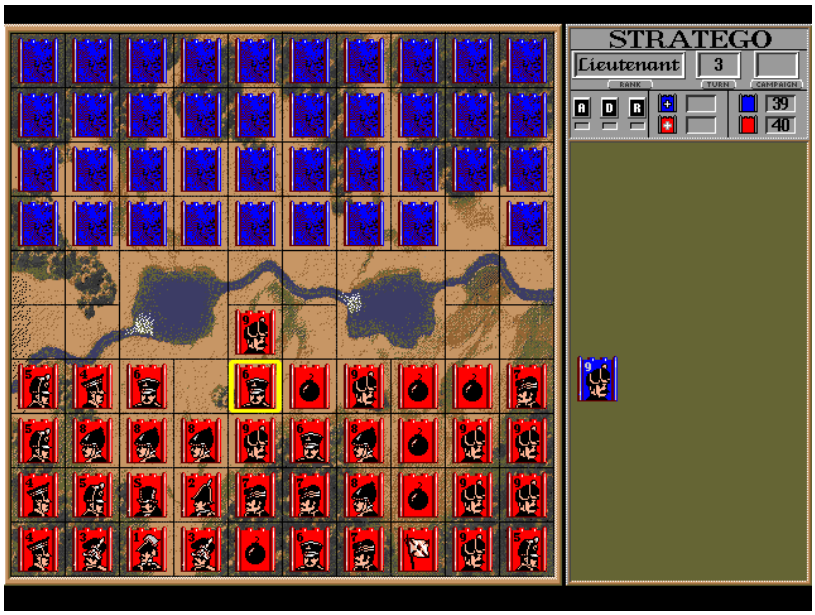


Figure 3.2: Stratego Game board.

method trains and evaluates MARQ agents simultaneously.

3.3 MARQ Policy Definition

Policy optimization in reinforcement learning improves an agent's strategy to maximize total reward [52]. Games with hidden information and multiple players require policies that perform well against fixed opponents and also handle uncertainty and shifting strategies from other agents. MARQ combines belief state reasoning with value-based decisions.

Policy learning in MARQ serves two goals. The first is winning games through tactical choices. The second is improving the quality of the embeddings that the AAREN module produces. The two objectives reinforce each other: better embeddings sharpen decision-making, and good decisions reveal opponent information that in turn refines the embeddings.

3.3.1 Core Policy Framework

A policy in the MARQ framework, denoted $\pi_i(a|I_i)$, defines a probability distribution over actions for agent i conditioned on its current information set I_i. In perfect information games, policies operate

Table 3.1: Key symbols and descriptions.

Symbol	**Description**	**Example (MARQ Framework)**
$\pi_i(a\|I_i)$	Policy for agent i choosing action a given information set I_i	Probability of moving a piece forward given current visible board state and belief about opponent pieces
$\mathcal{B}_t$	Implicit belief representation at time t	Probability distribution over possible identities and positions of hidden opponent pieces inferred by AAREN
$v_i^{\pi}(h, \mathcal{B}_t)$	Value function for agent i following policy π given history h and belief $\mathcal{B}_t$	Expected reward for current board position considering uncertainty about opponent setup
$Q_{\text{network}}(s', a)$	Deep Q-Network approximation of action-value function	Neural network estimate of value for attacking with a suspected Marshal vs opponent piece
$R^{\text{capture}}i, t+1$	Immediate reward from piece captures weighted by piece values	+9 points for capturing opponent Marshal, -1 for losing a Scout
$O = \{o_1, \ldots, o_t\}$	Observation sequence processed by AAREN attention mechanism	Sequence of board states, moves, and attack outcomes throughout the game
α_i	Attention weights for historical observations in AAREN	Higher weight on turn when opponent's Marshal was revealed, lower on routine Scout moves
π^*	Nash equilibrium policy profile	Optimal mixed strategy that cannot be exploited by opponent strategy changes
$Q_{eval}(\mathcal{B})$	Embedding quality evaluator output	Confidence score for learned representation accuracy
$H(\pi)$	Policy entropy measure	Ensures minimum unpredictability to prevent exploitation
ϵ	Exploration rate for agent behavior	Controls exploration vs exploitation balance in multi-agent learning
γ	Discount factor for future rewards	Importance of long-term vs immediate rewards in value estimation

on complete game states. MARQ policies operate under uncertainty about the true state of the environment. The information set I_i contains all observations available to the agent: visible board positions, revealed piece identities, and the history of observed movements and combat outcomes.

The core difficulty is that agents must act without knowing the opponent's private information. In Stratego, this means the rank and position of every unrevealed opponent piece remain unknown. The policy needs to take account for variations of the possible hidden pieces alongside the opponents strategy.

MARQ uses a implicit belief representations $\mathcal{B}_t$ rather than calculating every possible game state. The AAREN module learns and updates these representations as new information arrives. The policy takes the following form:

$$\pi_i(a|I_i, \mathcal{B}_t) = P(\text{action} = a \mid \text{information set } I_i, \text{implicit belief } \mathcal{B}_t) \tag{3.2}$$

The policy learns to weight actions based on their expected value across the distribution of opponent configurations rather than overfitting to a single enemy configuration.

3.3.2 Implicit Representation Optimization

MARQ treats representation accuracy as a training objective and uses revealed piece identities as supervised labels for embedding refinement.

When combat reveals the piece identities, the system calculates the prediction error using Kullback-Leibler (KL) divergence.

$$\mathcal{E}_{\text{pred}}(t) = \sum_{p \in \text{Pieces}_{\text{revealed}}(t)} D_{\text{KL}}\left(\text{Representation}_t(p) \| \delta_{s_p^*}\right) \tag{3.3}$$

Here, $\delta_{s_p^*}$ is the Dirac delta distribution concentrated on the true piece type. The error quantifies the distance of the predicted distribution to the actual identity.

This prediction error becomes is the reward signal that encourages the policy to better predict actual piece configurations:

$$R_{\text{accuracy}}(t) = -\mathcal{E}_{\text{pred}}(t) = -\sum_{p \in \text{Pieces}_{\text{revealed}}(t)} \left[\text{Representation}_t(p = s_p^*) > 0\right] \cdot \log \text{Representation}_t(p = s_p^*) \tag{3.4}$$

The optimal policy maximizes:

$$\pi_i^* = \arg\max_{\pi_i} \mathbb{E}_{\tau\sim\pi_i}\left[\sum_{t=0}^{T}\gamma^t\left(R_{\text{game}}(t) + \eta\cdot R_{\text{accuracy}}(t)\right)\right] \tag{3.5}$$

The hyperparameter $\eta \in \mathbb{R}^+$ controls the accuracy versus immediate game rewards. The value function in MARQ extends the standard Bellman equation to account for game state belief, expressing the expected cumulative reward as a function of the current known information set and implicit belief.

$$v_i^\pi(I, \mathcal{B}_t) = \mathbb{E}_\pi\left[R_{i,t+1}^{\text{capture}} + \lambda_1 R_{i,t+1}^{\text{info}} + \lambda_2 R_{i,t+1}^{\text{position}} + \gamma v_i^\pi(I_{t+1}, \mathcal{B}_{t+1}) \mid I_t = I, \mathcal{B}_t\right] \tag{3.6}$$

Three components make up the reward function. The capture reward R^{capture} for capturing a piece. The information reward R^{info} for information revelation. The position reward R^{position} for territorial control.

MARQ training aims to reach a Nash equilibrium. At equilibrium, no player can improve their expected payoff.

$$v_i(\pi^*) = \max_{\pi_i} v_i(\pi_i, \pi^*_{-i}) \tag{3.7}$$

Reaching Nash equilibria in a large imperfect information game is computationally intractable. MARQ seeks approximate equilibria through self-play training with diverse opponent populations.

3.4 Deep Q-Network Architecture

The Deep Q-Networks in MARQ operates on implicit belief representations instead of a complete game states [52]. The network approximates Q-values over probability distributions of opponent configurations, $\mathcal{B}_t$:

$$Q(s, a) = \mathbb{E}_{s'\sim\mathcal{S}_t\subseteq\mathcal{B}_t}[Q_{network}(s', a)]$$

A standard replay buffer stores experience. Transitions $(\mathcal{B}_t, a_t, r_t, \mathcal{B}_{t+1})$ are sampled uniformly to train the value function.

3.4.1 Rainbow DQN Components

The implementation uses the standards of the Rainbow DQN architecture [26].

Distributional RL (Categorical DQN). The Categorical DQN (C51) [8] models the value distribution directly. It captures the uncertainty of the game by approximating the distribution with 51 atoms spanning the range from V_{min} to V_{max}.

$$Z_\theta(s,a) = \sum_{i=1}^{N} p_i(s,a)\delta_{z_i} \tag{3.8}$$

Training minimizes the Kullback-Leibler divergence between the predicted distribution and the projected target distribution:

$$D_{KL}(\Phi\hat{\mathcal{T}}Z_{\theta'}||Z_\theta) \tag{3.9}$$

where Φ is the projection operator that maps the Bellman update onto the fixed support atoms.

Noisy Networks for Exploration. Noisy Linear layers [61] adds a learned noise into the end of the network layers. This creates an unpredictable strategy without a fixed exploration variable, and the noise parameters are tuned during training for each state.

$$y = (\mu_w + \sigma_w \odot \epsilon_w)x + (\mu_b + \sigma_b \odot \epsilon_b) \tag{3.10}$$

Dueling Network Architecture. Compared to a single network in deep Q-networks, the dueling architecture [38] branches to 2 streams for state value estimation $V(s)$ and advantage estimation $A(s,a)$:

$$Q(s,a;\theta,\alpha,\beta) = V(s;\theta,\beta) + \left(A(s,a;\theta,\alpha) - \frac{1}{|\mathcal{A}|}\sum_{a'} A(s,a';\theta,\alpha)\right) \tag{3.11}$$

This separation improves policy evaluation when many actions carry similar value.

An exploration bonus is added to the Q-values. The bonus is based on the information gathered by garnering territory, and identifying opponent piece identity.

$$Q_{augmented}(s,a) = Q_{base}(s,a) + \lambda \cdot U(a) \tag{3.12}$$

Here $U(a)$ represents the entropy of the learned distribution at the target location. This term encourages the agent to investigate unknown opponent pieces.

3.4.2 Training Stability via Regularization

The recent study of the imperfect information game, Ataraxos reached superhuman performance [50]. Its designers found that training stability matters as much as the algorithm itself. Deep reinforcement learning agents face two recurring problems which is wasting updates on low-information experiences, a waste of sampling and training, and cycling between strategies without settling on a robust one, non-stationary.

Advantage Filtering. Advantage filtering hand picks the most informative transitions to learn from [55]. In reinforcement learning, the temporal-difference (TD) error measures the gap between predicted values and outcomes. A standard replay buffer samples past transitions at random and stores it in the buffer, treating every move equally. Advantage filtering instead samples the buffer and retains only the transitions with the largest TD errors:

$$\mathcal{B}_{filtered} = \mathrm{Top}_{n/k}\left(\mathcal{B}_{sampled}, |\delta_i|\right) \tag{3.13}$$

where $\delta_i = r_i + \gamma^n V(s'_i) - V(s_i)$ is the n-step TD error for transition i.

3.5 AAREN and Learned Embedding System

The AAREN module [22] will serve as the memory and history system within MARQ. It produces learned embeddings from observations that encode the inferences from those observations. AAREN handles long sequences, retaining details over hundreds of moves while avoiding the gradient problems that degrade older models such as LSTMs. An attention mechanism lets AAREN look back across the full sequence of moves at once [22].

3.5.1 Recurrent Attention Computation

AAREN implements an attention-based recurrent computation that maintains a state triplet (a_t, c_t, m_t) across the observation sequence. Traditional recurrent neural networks suffer from vanishing gradients over long sequences. AAREN avoids this entirely by providing a direct access to the full history through attention weights while maintaining $O(1)$ inference complexity per timestep.

Given a sequence of observations $O = \{o_1, \ldots, o_t\}$ where each o_i encapsulates the visible board state and public action outcomes, the AAREN cell first projects each observation into key and value

representations:

$$k_t = W_k(o_t), \quad v_t = W_v(o_t) \tag{3.14}$$

The projection matrices W_k and W_v are learned during training. The key k_t determines how relevant the current observation is for belief updates. The value v_t contains the content to be integrated into the belief representation. The attention score s_t is a numerical representation of each step in the sequence relative other steps and to the learned query vector q:

$$s_t = q^T k_t \tag{3.15}$$

The accumulator a_t aggregates value representations across the 1 episode sequence:

$$a_t = a_{t-1} \exp(m_{t-1} - m_t) + v_t \exp(s_t - m_t) \tag{3.16}$$

The term $\exp(m_{t-1} - m_t)$ adjusts the accumulator when a new maximum appears. The term $\exp(s_t - m_t)$ computes the relative attention weight for the current observation.

3.5.2 Embedding Updates

When a battle reveals a piece's true type, the AAREN embeddings is updated with ground truth. The inferred distribution at the revealed position collapses to certainty:

$$\text{Representation}_{pos}[t] = \begin{cases} 1.0 & \text{if } t = g \\ 0.0 & \text{otherwise} \end{cases} \tag{3.17}$$

The system updates the revealed piece count and adjusts remaining beliefs every time a revelation happens. For unrevealed positions $p \neq pos$:

$$\mathcal{B}_p[t] \leftarrow \mathcal{B}_p[t] \cdot \frac{\text{max_count}[t] - \text{revealed_count}[t]}{\text{remaining_count}[t]} \tag{3.18}$$

3.6 Multi-Agent Learning and Strategy Mixing

3.6.1 League-Based Training Architecture

The league has a changing pool of agents so that the model encounters many different strategies [46]. The main agent is the current policy under training. The historical pool stores earlier versions of

the main agent, each representing a different stage of development. Opponents are sampled during training according to a fixed probability distribution, to have less bias on the opponents fought.

$$P(\text{opponent}) = \begin{cases} 0.5 & \text{Main Agent (Self-Play)} \\ 0.5 & \text{Uniform(Historical Pool)} \end{cases} \tag{3.19}$$

Preventing the agent from cycling through the same strategies repeatedly. Reaching mastery of the past versions of itself.

3.6.2 Strategy Mixing for Exploitation Resistance

Deterministic strategies are suboptimal in imperfect information games where opponents have varying strategies [46, 53]. An adversary who knows the deterministic policy of the opponent reduces the game to a perfect information setting enabling the perfect strategy for an opponent, eliminating the strategic uncertainty that defines games like Stratego. MARQ counters this at two levels.

At the action selection level, Noisy Linear layers inject learned noise into the policy. The network learns the appropriate degree of randomization for each game state rather than relying on fixed exploration rate such as ϵ-greedy.

At the strategy level, diverse pool of opponent exposure during training achieves a more generalized learning. The league system maintains a population of historical agent checkpoints, each representing a different strategic approach discovered during training.

3.6.3 Reward Structure

The reward function $R(s, a, s')$ guides learning and shaping agent goals in Stratego's long horizon environment [18]:

$$R_{total} = R_{move} + R_{capture} + R_{tactical} + R_{info} \tag{3.20}$$

The move reward R_{move} gives small bonuses for advancing pieces (approximately +0.05) to encourage the agent to prioritize the territory gain and control. The capture reward $R_{capture}$ scales by the rank of the captured piece according to $0.15 \times \frac{\text{Rank}}{11.0}$. The tactical reward $R_{tactical}$ assigns specific bonuses for key strategic captures: +0.5 for Miner vs Bomb defusals and +1.0 for Spy vs Marshal assassinations. The information reward R_{info} provides +0.3 for revealing high-value opponent pieces.

3.7 Game Environment and State Management

The Stratego environment handles board state management, action validation, and observation filtering for agent training and evaluation.

3.7.1 Information Set Management

The environment maintains distinct representations for each information context:

$$\mathcal{I}_{player} = \{s : s \text{ is consistent with player observations}\} \tag{3.21}$$

Each player receives only the subset of game state information available through its observation function.

3.7.2 Temporal State Evolution

Game states evolve according to an update function.

$$s_{t+1} = \text{EnvStep}(s_t, a_t^1, a_t^2, \theta_t) \tag{3.22}$$

Here θ_t represents stochastic elements such as hidden information revelation and time limits.

3.7.3 Valid Action Filtering

The environment enforces game rules through valid action constraints:

$$\mathcal{A}_{valid}(s, player) = \{a : \text{IsLegalMove}(a, s, player) \wedge \text{RespectsGameRules}(a, s)\} \tag{3.23}$$

Only legal moves consistent with Stratego's rules and piece capabilities are available to the agent.

3.8 MARQ Move Selection and Decision-Making

Algorithm 1 outlines the MARQ move selection process and loop. The first four lines establish the information context. AAREN runs over the public observation sequence to produce an updated

history embedding $\mathcal{H}_t$, which is then projected through a softmax layer to yield a probability distribution $\mathcal{B}_t$ over the possible identities of each unrevealed opponent piece. The visible board state S and the history embedding $\mathcal{H}_t$ are concatenated into a single 79-channel tensor, and the Rainbow DQN processes that tensor to produce a Q-value distribution over all actions.

The algorithm then diverges based on move type. Non-attack moves take $Q(s, a_i)$ directly as their final score, since those moves carry no contact uncertainty. Attack moves receive an additional correction. The agent queries $\mathcal{B}_t$ at the target square, retrieves the rank-probability vector P, and computes an expected utility V_{exp} by summing over all possible opponent ranks weighted by the outcome of each encounter. The final score blends the network's learned estimate with this hand-computed expectation through the mixing coefficient λ. A high λ trusts the DQN; a low λ defers to the probabilistic expectation. This separation is necessary because the DQN's value estimate for an attack encodes long-run strategic considerations, while V_{exp} captures the immediate piece-trade arithmetic more explicitly.

The final selection step depends on whether the agent is in training mode and whether the temperature T is above zero. During self-play, a Boltzmann distribution over V_{final} governs sampling, which injects controlled stochasticity and prevents the agent from cycling into a fixed attack pattern that opponents can predict and counter. At evaluation, the temperature is set to zero and the agent commits to the greedy arg max over legal actions.

3.9 Generalizability

3.9.1 Mathematical Convergence Properties

Incremental Learning Bounds

The algorithm bounds value function estimation error as follows:

$$\|V_t - V^*\|_\infty \leq \frac{C}{\sqrt{t}} + \epsilon_{approx} \tag{3.24}$$

where C depends on reward bounds and the discount factor, and ϵ_{approx} represents function approximation error.

Algorithm 1 MARQ: Decision-making and Move Selection

Input: Public Observations O, Board State S, DQN weights θ, AAREN weights ϕ, Temperature T
Output: Optimal Action a^*

$\mathcal{H}_t \leftarrow \text{UpdateAggregation}(O, \phi)$
$\mathcal{B}_t \leftarrow \text{GetSoftmaxBeliefs}(\mathcal{H}_t)$
$X_t \leftarrow \text{Concatenate}(S, \mathcal{H}_t)$
$Q(s, \cdot) \leftarrow \text{RainbowInference}(X_t, \theta)$
$\mathcal{A} \leftarrow \text{LegalMoves}(S)$
for all $a_i \in \mathcal{A}$ **do**
  if $\text{IsAttack}(a_i, S)$ **then**
    $target_pos \leftarrow \text{TargetSquare}(a_i)$
    $P \leftarrow \mathcal{B}_t(target_pos)$
    $V_{exp} \leftarrow \sum_{r \in \mathcal{R}} P(r) \cdot \mathbb{U}(\text{Attacker}, r)$
    $V_{final}(a_i) \leftarrow \lambda Q(s, a_i) + (1 - \lambda) V_{exp}$
  else
    $V_{final}(a_i) \leftarrow Q(s, a_i)$
  end if
end for
if Training/Self-Play and $T > 0$ **then**
  $\pi(a_i) \leftarrow \frac{\exp(V_{final}(a_i)/T)}{\sum_{a \in \mathcal{A}} \exp(V_{final}(a)/T)}$
  return $a^* \sim \pi(\cdot)$
else
  return $a^* \leftarrow \arg\max_{a_i \in \mathcal{A}} V_{final}(a_i)$
end if

Implicit Representation Accuracy Bounds

Representation accuracy improves with additional observations:

$$\mathbb{E}[\text{Accuracy}(\mathcal{B}_t)] \geq 1 - \exp(-\alpha \cdot N_{obs}(t)) \tag{3.25}$$

where $N_{obs}(t)$ is the cumulative number of informative observations and α is a positive constant.

Strategy Convergence in Self-Play

The league training methodology converges to ϵ-Nash equilibria under standard monotonic improvement assumptions:

$$\lim_{T \to \infty} \|\pi_T - \pi^*\| \leq \epsilon \tag{3.26}$$

for some $\epsilon > 0$ depending on the exploration parameters and opponent diversity. The convergence depends on the diversity of the opponent pool and the promotion thresholds that control which agents enter the main pool.

The experimental results in Chapter 5 are consistent with these bounds. The Rainbow agent's loss continued to decrease over 75,000 episodes (Finding 13), and the win rate in self-play reached 49.88% (Finding 7), though the system was not run long enough to test convergence to approximate Nash equilibrium.

3.10 Algorithmic Evolution and Architecture Ablations

The final MARQ architecture emerged from systematic experimentation and the refinement of several reinforcement learning components. Multiple established algorithms were evaluated and replaced because of performance bottlenecks or insufficient strategic depth in the Stratego domain.

3.10.1 Value Optimization and Exploration.

Early iterations used a normal DQN with ϵ-greedy exploration. This approach proved to be a challenge early on for Stratego's sparse rewards and led to a desync of loss to seing model improvement though winrate and reward accumlated. The implementation was upgraded to a Rainbow DQN architecture incorporating Noisy Networks for a more dynamic approach to exploration. The standard DQN configuration was then replaced with a Dueling Head architecture, which improved value

estimation accuracy in states where multiple actions carry similar utility. To mitigate overfitting to deterministic opponent policies during self-play, the system also employs *Boltzmann Exploration* (Temperature Scaling). The opponent's action selection uses a temperature-scaled softmax over Q-values, injecting controlled stochasticity to produce a more robust training curriculum.

3.10.2 History and Memory Representation.

Initial attempts to manage hidden information in Stratego relied on explicit Probabilistic Belief States, a direct representation using another dqn to estimate the piece value. These states introduced prohibitive computational complexity and memory overhead. They required $O(N^2)$ updates that throttled training throughput. The framework moved to an implicit belief system powered by the AAREN architecture. Long Short-Term Memory cells for trajectory embeddings were also replaced by AAREN's attention-based recurrence, which provided more stable gradient flow over long sequences and better handled the non-Markovian nature of piece identity inference.

3.10.3 Architectural Refinements.

The view of the agent and how it sees the enviroment had several iterations, moving from standard deep convolutional stacks to a dedicated ResNet architecture. Testing identified 6 Residual Blocks as the optimal depth. This configuration provides full receptive field coverage of the 10x10 board while maintaining low inference latency. The optimizer was transitioned from standard ADAM to ADAMW to improve weight decay and policy loss stability.

3.10.4 Strategic Initialization and Efficiency.

The piece placement phase was initially modeled using an agentic SetupAgent trained through deep reinforcement learning. This approach introduced exploitability risks and computational overhead. The project transitioned to a non-agentic, fixed Heuristic Setup Agent with randomized flag positioning. The change resolved the exploitability concern and provided substantial computational savings. The agentic setup required approximately 2 seconds per episode; the heuristic method completes in under 100 milliseconds. This optimization effectively halved the total training compute requirement. The architectural iterations and the theoretical justifications are systematically documented, linking each component's theoretical necessity directly to empirical validation.

Chapter 4

Methodology

This chapter presents the development and evaluation of the MARQ framework, covering the system architecture, training procedure, and the metrics used to compare agent performance against heuristic and self-play baselines.

4.1 Project Planning

This research followed a rapid prototyping method for the design and construction of the MARQ framework. Development ran from September 2025 to February 2026 and covered building the game environment and training the agent. Algorithmic evaluation and ablation testing ran from March to April 2026.

4.2 System Development

System development consists of two components: the game environment and the MARQ framework. The game environment provides the simulation in which agents operate and train. The MARQ framework produces the trained agent whose performance is evaluated against heuristic baselines and through self-play.

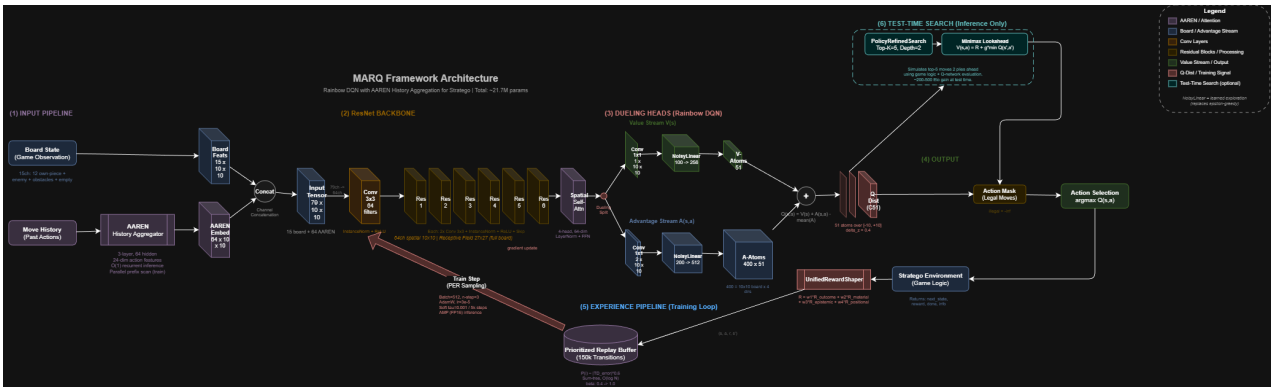


Figure 4.1: MARQ framework architecture.

4.2.1 Piece Value Computation

The reward function requires a quantitative assessment of piece value to guide learning. This section presents an analytical method for computing piece values based on probability theory, as a deterministic alternative to simulation-based estimation.

Combat Dominance Score

The Combat Dominance Score (CDS) represents the probability that a given piece type defeats a uniformly random opponent piece. Let $\mathcal{P}$ denote the set of all piece types and n_i the count of piece type i in standard Stratego. The CDS for piece p is defined below.

$$\text{CDS}(p) = \frac{\sum_{i \in \mathcal{P}} \mathbb{1}_{p \succ i} \cdot n_i}{N} \tag{4.1}$$

where $N = \sum_{i \in \mathcal{P}} n_i = 40$ is the total piece count, $\mathbb{1}_{p \succ i}$ is an indicator function equal to 1 if piece p defeats piece i according to Stratego combat rules, and 0 otherwise. The combat rules specify that higher-ranked pieces defeat lower-ranked pieces, with specific exceptions. The Spy defeats the Marshal when attacking, and the Miner defeats Bombs.

Survival Rate

The Survival Rate (SR) measures the expected probability of a piece surviving an attack from a uniformly random attacker. Let $\mathcal{M} \subset \mathcal{P}$ denote the set of movable piece types (excluding Flag and Bomb). The SR for piece p is calculated as follows.

$$\mathrm{SR}(p) = \frac{\sum_{a \in \mathcal{M}} \nVdash_{p \succeq a} \cdot n_a}{\sum_{a \in \mathcal{M}} n_a} \tag{4.2}$$

The indicator function equals 1 if piece p survives an attack from piece a.

Table 4.1: Computed piece values with combat and survival metrics.

Piece	Value	CDS	SR
Marshal	8.4	0.825	0.939
General	8.0	0.800	0.939
Colonel	7.5	0.750	0.879
Major	6.7	0.675	0.788
Captain	5.7	0.575	0.667
Lieutenant	4.8	0.475	0.545
Miner	4.3	0.400	0.273
Sergeant	3.8	0.375	0.424
Bomb[†]	2.0	—	—
Spy	1.1	0.050	0.000
Scout	1.0	0.050	0.030
Flag[‡]	0.1	—	—

[†]Immovable defensive piece; eliminates any non-Miner attacker.

[‡]Immovable objective piece; capture results in game loss.

Auxiliary Value Components

Certain pieces have capabilities that other than their combat rank. Table 4.2 enumerates these bonuses.

Table 4.2: Strategic ability bonuses for special-capability pieces.

Piece	Bonus	Description
Miner	2.0	Capable of removing Bombs
Spy	1.5	Defeats Marshal when initiating attack
Scout	1.0	Multi-square movement capability
Marshal	0.5	Highest combat rank

4.2.2 AAREN and Game History

The challenge in Stratego is the partial observability. The framework uses the same architectural approach as DeepNash. In deepnash the technique used was a U-Net Convolutional Torso with Residual

Blocks and Skip-Connections [46]. The Attention-as-a-Recurrent-Neural-Network (AAREN) module aggregates history and generates embeddings which then is passed to the CNN and Resnet.

Action Feature Extraction. For each step observed opponent action, AAREN extracts a 24-dimensional feature vector encoding movement features, position features, behavioral indicators, and temporal features.

4.3 Model Training

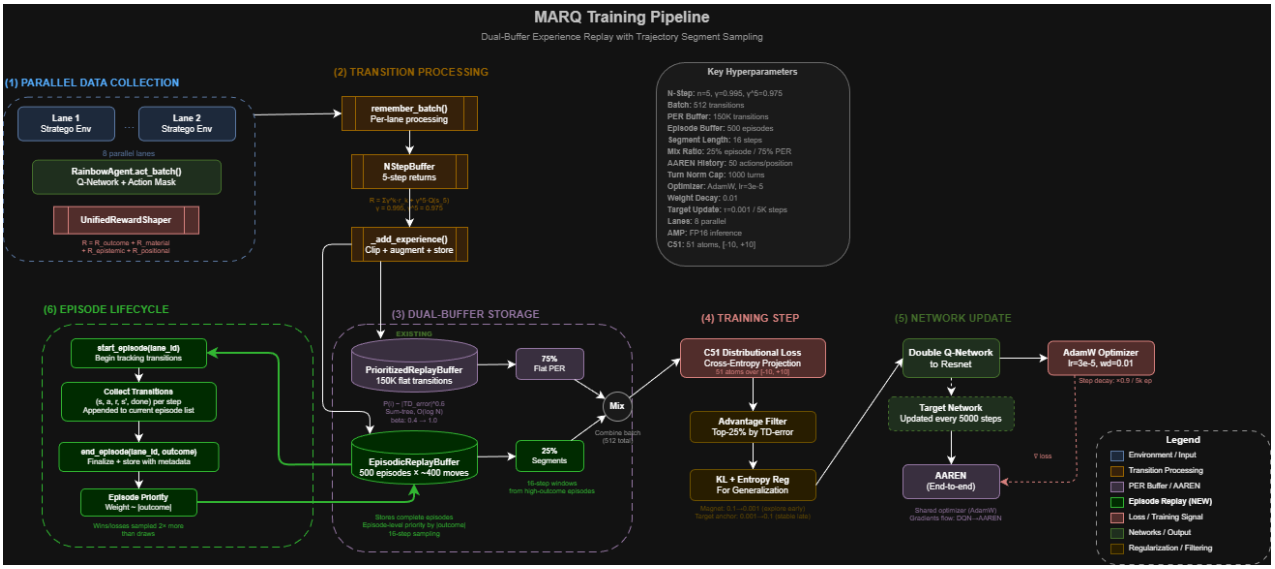


Figure 4.2: MARQ training architecture.

Training the MARQ agent is computationally intensive due to the large state-action space and the need for accurate belief state representations over extended game horizons.

Rainbow DQN Architecture. The MARQ system uses the six components of Rainbow DQN [26]. Categorical DQN (C51) [8] handles value distribution estimation. Noisy Networks [61] drive exploration. A Dueling Network architecture [38] decouples state values from action advantages. Double DQN chooses the bootstrap action with the online network and takes its value from the target network. Prioritized experience replay samples transitions in proportion to their loss, and

n-step returns ($n = 5$) carry rewards several moves back in one update. The noisy layers introduce state-dependent stochasticity, removing the need for manual epsilon-greedy scheduling.

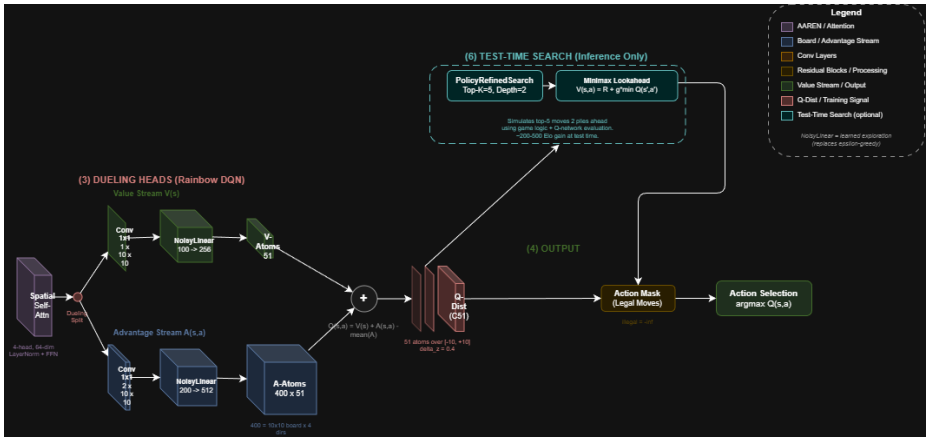


Figure 4.3: Dueling network heads.

Network Structure. The Q-network features a ResNet backbone for spatial feature extraction. An initial convolutional layer feeds into six residual blocks. This configuration provides a sufficient receptive field to evaluate global board state interactions.

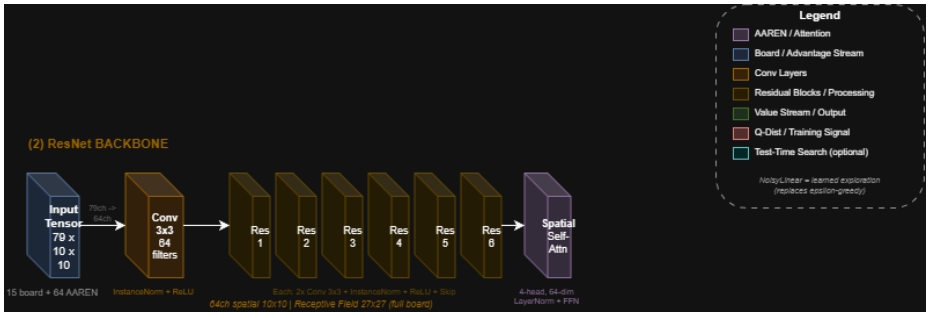


Figure 4.4: ResNet backbone.

A spatial self-attention layer follows the ResNet backbone. It captures relationships between pieces across the full board, even at long range. This layer feeds into the dueling heads, allowing the network to weight distant piece relationships in value estimation.

The 79-channel state representation is partitioned into two functional sets, as summarized in Table 4.3.

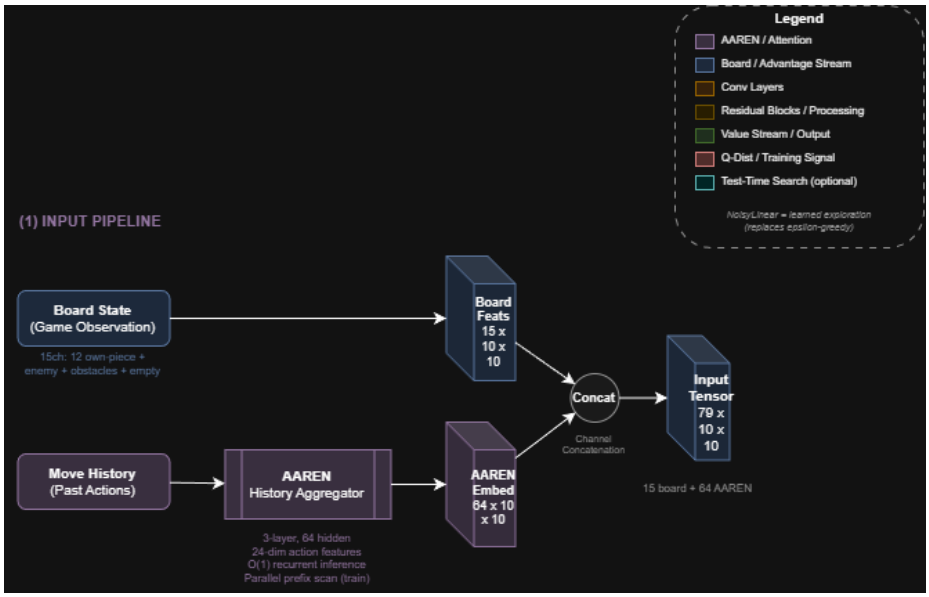


Figure 4.5: 79-channel input representation.

The board feature channels provide deterministic knowledge of the agent's own piece positions and observed enemy locations. The AAREN embedding channels encode learned representations of hidden enemy piece identities based on observed behavior. These embeddings are dense learned vectors, not explicit probability distributions. AAREN learns them by predicting the type of an enemy piece when a battle reveals it. The Q-network learns how to use the embeddings, but its loss does not train AAREN (see the Gradient Bridge below). The AAREN embedding channels are populated in all training phases; the agent always receives embeddings from the history aggregator rather than ground truth one-hot encodings. The board feature channels carry different levels of enemy information depending on the observability setting. During full observability (Phase 1), the board tensor reveals all enemy piece identities, providing a strong signal in the piece-specific board channels. During partial observability phases, unrevealed enemy pieces appear only as a "hidden piece" presence marker (affecting Channel 12, the enemy mask), and the agent must rely on AAREN embeddings to infer their true identities. An earlier version suffered from a bug where the hidden piece marker was filtered out of Channel 12, blinding the network to the presence of enemy pieces.

Training Hyperparameters. The training configuration specifies a learning rate $\alpha = 3 \times 10^{-5}$ with AdamW optimizer and weight decay $\lambda = 0.01$, and a discount factor $\gamma = 0.995$ to capture

Table 4.3: State representation input channels.

Channel	Category	Description
Board Features (Channels 0–14)		
0–11	Own Pieces	Binary spatial maps indicating positions of each piece type (Flag, Spy, Scout, Miner, Sergeant, Lieutenant, Captain, Major, Colonel, General, Marshal, Bomb)
12	Enemy Mask	Binary map indicating positions of all visible enemy pieces
13	Obstacles	Binary map of lake squares (impassable terrain)
14	Empty Squares	Binary map of unoccupied traversable positions
AAREN Embedding Features (Channels 15–78)		
15–78	Learned Embeddings	64-dimensional learned embedding vectors per board position, produced by the AAREN history aggregator from observed opponent action sequences. AAREN learns them from a supervised loss on revealed pieces; the Q-network reads them as inputs.

long-term strategic consequences. A batch size of 512 transitions and a replay buffer capacity of 150,000 transitions with prioritized sampling are used. Soft target network updates ($\tau = 0.001$) occur every 5,000 steps. Model updates run every 2 steps across 8 parallel lanes with 5-step returns for deeper credit assignment in long-horizon games. The AAREN history aggregator retains up to 50 actions per position in its training buffer to preserve early-game movement patterns across 200 to 500 move games. Turn normalization is aligned with the maximum game length of 1,000 turns. Horizontal flip augmentation exploits the game's symmetry to improve sample efficiency. Training proceeds for 35,000+ episodes with checkpoints saved every 1,000 episodes.

Configuration after the thesis experiments. The settings in this chapter are those of the experiments reported in Chapter 5. The training system was revised after them (Appendix C). The later formal runs use a 30-plane observation (32 from formal_6 on, with two planes marking the hidden enemy pieces that have moved) with a lossless 3,600-action encoding and fog of war in every stage. They use $\gamma = 0.999$, a batch of 128, a replay capacity of 75,000 and a hard copy into the target network every 1,000 updates. They run 16 parallel lanes and start from heuristic demonstrations. In their first phase the AAREN planes are switched off, so those runs test the Rainbow learner without AAREN. Their results are reported in Appendix C, not in Chapter 5.

Computational Optimizations. Mixed-precision inference accelerates training throughput. During action selection, network forward passes operate in FP16 precision via `torch.amp.autocast`, reducing memory bandwidth requirements and leveraging Tensor Core acceleration on NVIDIA GPUs.

Integrating the recurrent AAREN module into the replay pipeline presents a significant bottleneck. Full sequential reconstruction of action histories during replay increases iteration time from 15s to 60s. To maintain high throughput, MARQ stores pre-computed embedding snapshots in the replay buffer, detached from AAREN's computation graph. This gave a fourfold speedup. The thesis code also added a "Gradient Bridge" to the stored snapshots during replay: an epsilon-scaled projection of a zero vector, $\epsilon \cdot W_{proj}(\mathbf{0})$, meant to give the DQN loss a gradient path back to AAREN. The projection is affine, so $W_{proj}(\mathbf{0}) = \mathbf{b}_{proj}$ and the term is the layer's bias alone. The DQN loss therefore reached only one of AAREN's 21 parameter tensors, and none that depends on the move history. In the thesis experiments AAREN was trained, in effect, only by its supervised signal from piece reveals. The current code drops the bridge and trains AAREN by that supervised loss alone (Appendix C).

Multi-Phase Curriculum. Training follows a multi-phase curriculum. Phase 1 establishes foundational rules under full observability. As win rates stabilize, the curriculum introduces mixed observability, requiring the agent to use AAREN-based inference before full information occlusion is enforced.

Phase 1 runs for 5,000 to 10,000 episodes against a progressively challenging opponent mix: initially random opponents, then basic heuristic agents, and finally a multi-factor heuristic opponent. Phase advancement requires $\geq$95% win rate against random opponents, $\geq$75% against the basic heuristic, and $\geq$60% against the strategic heuristic. All win-rate metrics are computed relative to decisive games (wins and losses) rather than total episodes to account for the high frequency of early-stage draws.

Phase 2 (Partial Observability) disables full observability and requires reliance on AAREN-generated embeddings for 5,000 to 10,000 episodes. The opponent distribution allocates 40% to frozen heuristic agents and 60% to the strategic heuristic. Phase advancement requires embedding-based inference accuracy $\geq$75% and an overall win rate $\geq$60% relative to decisive games.

Phase 3 (Self-Play) trains against a mixture of delayed agent copies (60%) and strategic heuristic

opponents (40%) for 5,000 to 15,000 episodes to discover novel strategies while preventing policy collapse. Phase 4 (League Training) activates the full opponent distribution with 40% historical league agents, 30% strategic heuristic, 20% greedy agents, and 10% self-play. This phase also activates independent parallel-learning (MARL) for Agent 2, transitioning the opponent from a fixed policy pool to a continuously updating student agent. Phase 5 (Scenario Drills) introduces specialized board configurations every 1,000 episodes for targeted skill training.

4.3.1 Test-Time Search

The trained Q-network provides a strong action-selection policy through single-pass inference. Tactical performance in partially observable environments improves with a formal *Expectamax* search, originally proposed by Michie [43] and recently analyzed in complex game environments by Novikov and Yanovskyi [43]. This algorithm extends standard minimax by incorporating stochasticity through "Chance Nodes" that represent the hidden identity of opponent pieces.

The search refinement uses the piece-identity probabilities generated by the AAREN history aggregator to calculate expected utility of potential moves. For a given state s and a set of legal moves $\mathcal{A}(s)$, the refined value $V^*(a_i)$ is computed as follows:

1. **Max Node evaluation.** The agent evaluates each candidate move a_i using its positional prior $Q_{dqn}(s, a_i)$.

2. **Chance Node summation.** If a_i results in an attack on position j, the identity of the target piece is treated as a chance node. The expected combat utility $\mathbb{E}[U_{combat}]$ sums the utility of all possible outcomes, weighted by their probabilities:

$$\mathbb{E}[U_{combat}] = \sum_{r \in \mathcal{R}} P(\text{rank}_j = r|\mathcal{H}) \cdot \text{Utility}(rank_{attacker}, r) \tag{4.3}$$

   where $P(\text{rank}_j = r|\mathcal{H})$ is the AAREN-derived probability that the target piece at position j has rank r, and $\mathcal{H}$ denotes the observed interaction history.

3. **Move Selection.** The final decision combines the positional prior and the expected tactical outcome:

$$a^* = \arg\max_{a_i \in \mathcal{A}(s)} (\lambda \cdot Q_{dqn}(s, a_i) + (1 - \lambda) \cdot \mathbb{E}[U_{combat}]) \tag{4.4}$$

The Expectamax logic prevents the agent from committing high-value pieces to squares with high probabilities of containing Bombs or superior ranks, even when the raw Q-network output is optimistic about the board position. The exact summation over rank probabilities grounds the risk assessment in observed opponent behavior.

4.3.2 Deployment

The best-performing MARQ model weights are saved after training and validation. During evaluation, exploratory noise is deactivated, and the model selects the action with the highest predicted Q-value. Each evaluation run uses the same checkpoint to maintain consistency across test conditions.

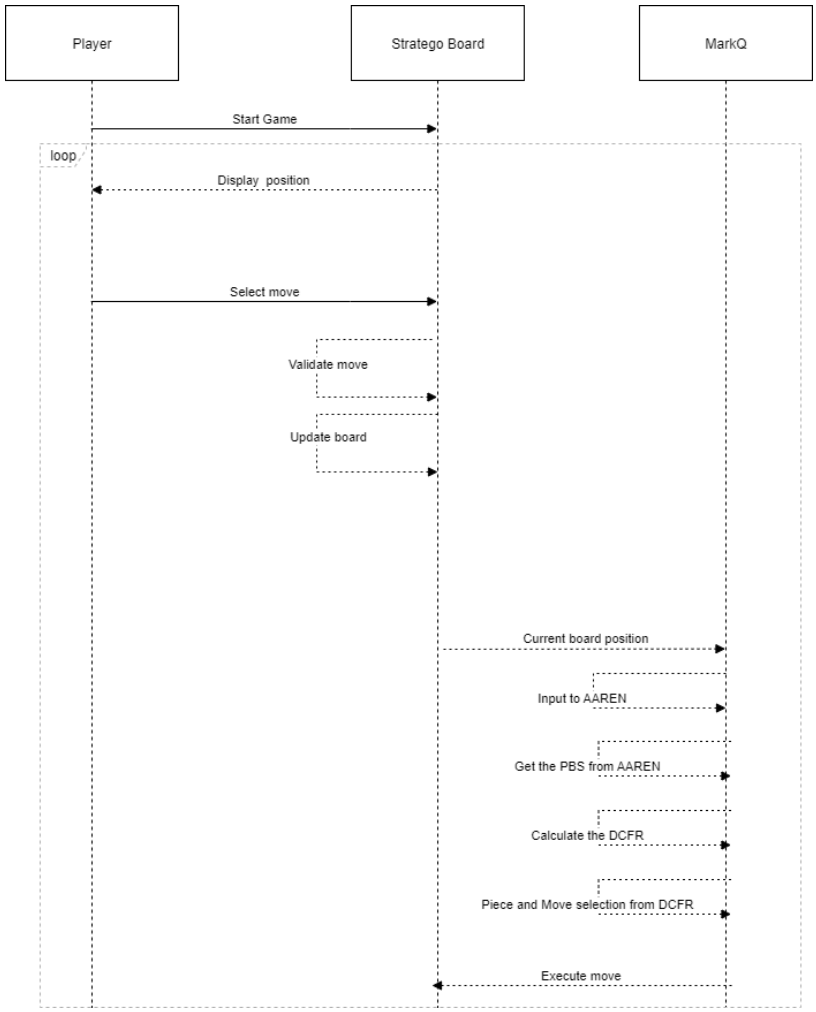


Figure 4.6: Game sequence diagram.

The game loop runs until one agent captures the opposing flag. On each turn, the active agent receives the current board state and updates the AAREN module, which produces learned embeddings from observed action history. The DQN evaluates Q-values for all legal actions given this embedding-enhanced state representation and selects the action with the highest expected value.

4.4 Game Environment

The game environment is a digital implementation of Stratego developed in Python and Pygame. It manages the board state, enforces game rules, handles combat resolution, and provides the observation interface for agent perception. All agent-vs-agent experiments in this study use this environment.

4.4.1 Evaluation

The evaluation tracks learning progress and agent performance through agent-vs-agent testing. Win rate measures the proportion of decisive games won against each opponent tier. Average game length, computed as the mean number of moves per game, indicates decision efficiency. Loss curves show the progression of distributional and value losses over episodes. They show whether the critic fits its own targets, not whether the policy improves, so they are read next to the win rates rather than in their place. The Rainbow DQN agent is compared with a standard DQN baseline. The two configurations differ in more than one component at a time, so the comparison shows the effect of the combination. Isolating each component needs an ablation that removes one at a time, which has not yet been run (Chapter 6). Empirical findings are organized thematically into designated subsections to clearly connect each architectural modification to its resulting behavioral outcome.

Developing the MARQ framework is iterative. Each version represents an improvement over the last. Appendix B shows the initial results from 1,000 episodes. During this early stage, the agent is still exploring strategies. The early win rates show many draws as pieces move without decisive engagement. With Noisy Networks, the agent learns how much to explore on its own, adapting better as training continues.

Chapter 5

Results and Discussion

5.1 Normal DQN Performance Ceiling

The multi-phase curriculum concluded with 37,213 episodes of self-play training. The normal DQN agent used an LSTM-based history encoder and trained against random, greedy heuristic, league, and self-play opponents. Table 5.1 reports the results.

Table 5.1: Normal DQN performance after 37,213 episodes of self-play training.

Opponent Type	**Episodes**	**Wins**	**Win Rate**	**Draw Rate**
Greedy Heuristic	9,319	1,418	15.2%	41.3%
Self-Play	9,565	2,370	24.8%	36.8%
Random	9,294	2,260	24.3%	37.7%
League	9,035	2,231	24.7%	38.2%
Aggregate	**37,213**	**8,279**	**22.2%**	**38.5%**

The training data contains six findings, each linked to an architectural limitation.

5.1.1 Win Rate Plateau

Agent1's cumulative win rate stayed near 22% after 2,000 episodes, it stayed more or less the same over the next 35,000. The 100-episode rolling average swung between 15% and 30% with no deviation.

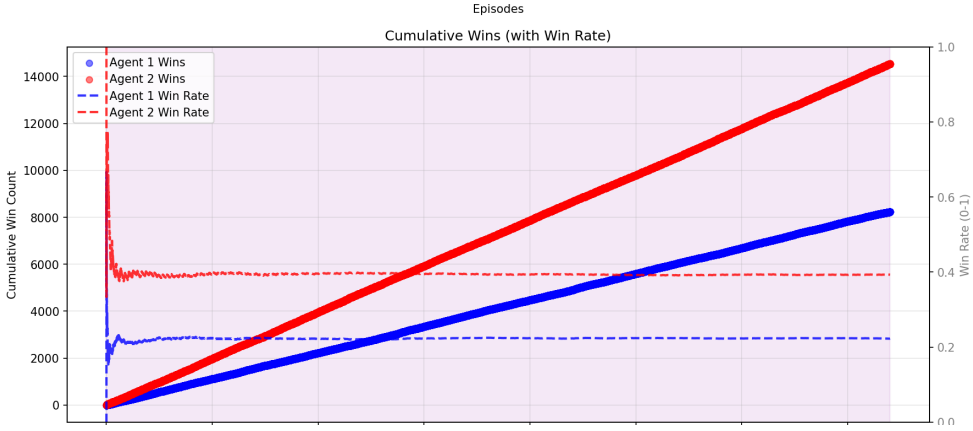


Figure 5.1: Cumulative win count and win rate over 37,213 episodes. The win rate plateaus early and remains flat, indicating policy stagnation.

5.1.2 Loss–Performance Decoupling

The TD error fell to roughly 0.001 however, the win rate did not follow. The network learned to predict the value of passive, draw-seeking play accurately. It did not discover better strategies.

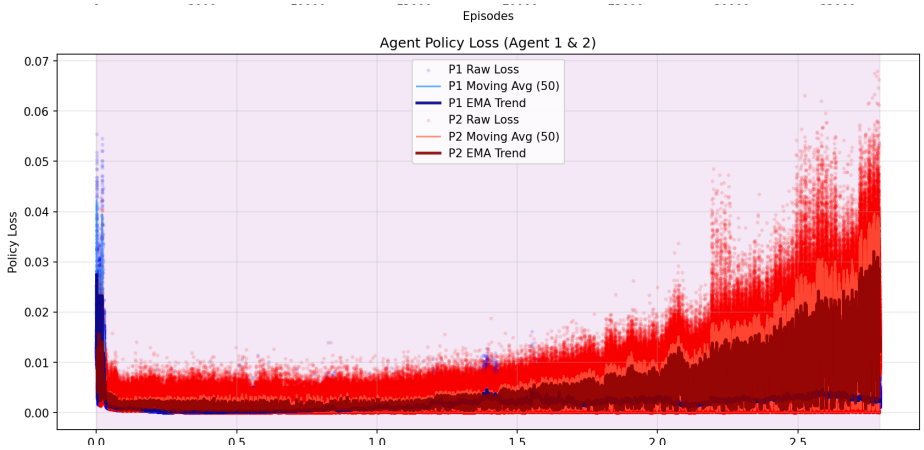


Figure 5.2: Policy loss (TD error).

5.1.3 Q-Value Overestimation

Average Q-values climbed from 0.2 to 1.2. Win rates did not move with the rising value estimates.

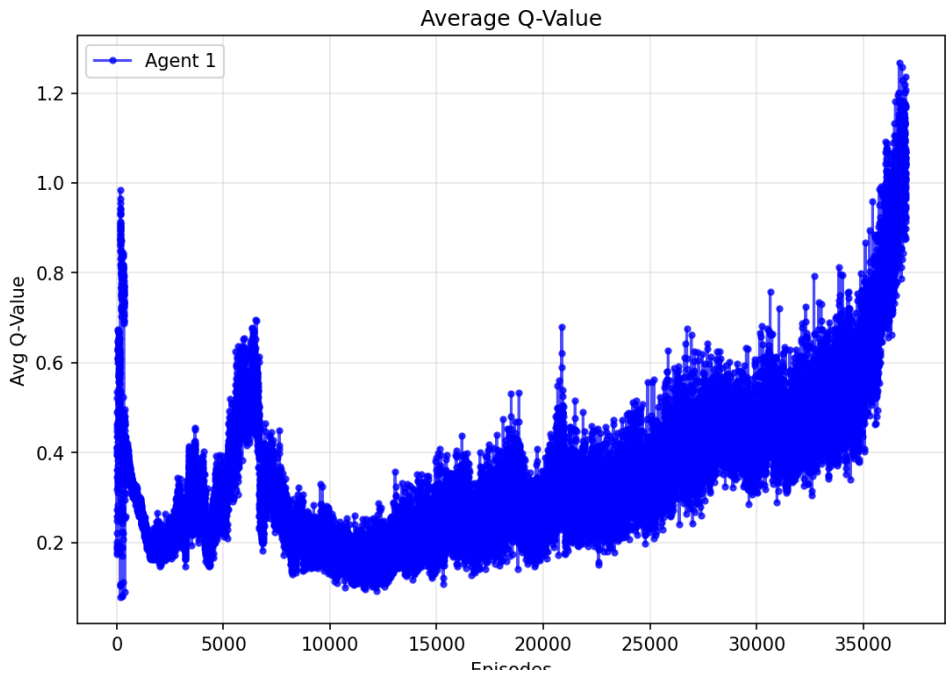


Figure 5.3: Average Q-values climb.

5.1.4 LSTM Prediction Accuracy Ceiling

The LSTM piece-identity module accuracy stayed around 18–22%, a 2.2–2.6× gain over the 8.3% random baseline but is still far short of reliable tactical inference. The improvement stopped after episode 5,000 and stayed at that level.

5.1.5 Opponent-Specific Performance Asymmetry

Performance varies by opponent given randomly in the training process. Win rate against the greedy heuristic was 15.2%; in self-play it reached 24.8%. The policy fit its own behavioral distribution and did not adapt to the others opponent types.

5.1.6 Draw Dominance

The game results was mostly draws. Draws accounted for 38.5% of outcomes, far above the 22.2% win rate. The agent learned to shuffle pieces passively and avoid uncertain engagements. It minimized short-term losses at the expense of long-term winning probability, lacking the long horizon planning.

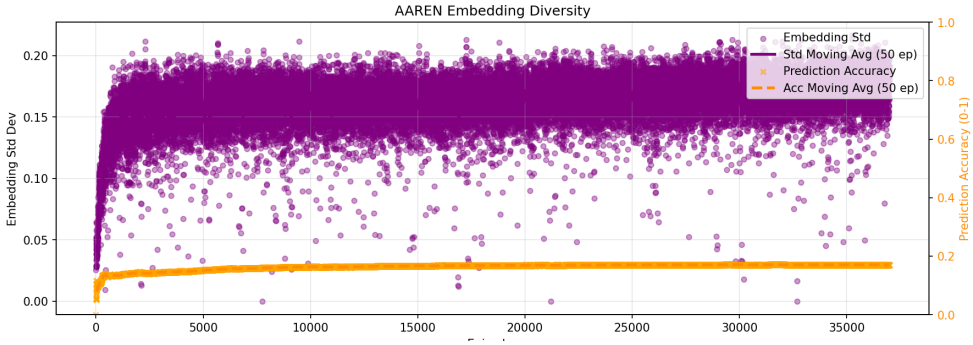


Figure 5.4: Piece-identity prediction accuracy.

5.2 Behavioral Patterns from Training Logs

5.2.1 Passive Shuffling Convergence

The agent grew conservative. Average episode length rose throughout training as it avoided combat and cycled pieces between adjacent squares. Scalar value collapse (Finding 2) explains why the Q-network converges to the lowest-variance action class, playing safe moves instead of winning moves.

5.2.2 Opponent-Specific Adaptation Failure

Table 5.1 shows the agent overfitting to its own play style, not generalizing to other agents, since the drop-off of performance against greedy opponents is significant (15.2%). The greedy heuristic has the lowest win rate (15.2%) due to its rank-agnostic attack pattern directly punishing passive shuffling. The heuristic set of instructions attacks the nearest piece, forcing repeated engagements that the agent's risk-averse policy is designed to avoid. When forced into combat, the agent unreliable piece-identity inference needed to choose favorable matchups fails, so each engagement is approximately a coin flip weighted by the opponent's rank distribution. The result is a consistent loss rate that the agent win with further passivity.

Self-play produced the highest win rate (24.8%), but this figure can be misleading. Both agents share the same weights and therefore the same behavioral tendencies. When two equally passive and cautious policies face each other, games resolve through symmetric attrition rather than strategic superiority. The apparent 24.8% win rate reflects the probability that stochastic tiebreakers favor one copy over another, not genuine tactical competence. The high draw rate in self-play (36.8%)

confirms that most games end in mutual passivity.

Random opponents produced a 24.3% win rate, nearly identical to self-play despite being the weakest opponent class. A random policy does not exploit the agent's weaknesses, but it also provides no consistent pattern for the agent to learn from. Transitions generated against random play carry high variance and low signal, contributing gradient noise without driving policy improvement. The similar win rates against random and self-play opponents indicate that the agent has not learned to exploit weak play, further evidence of policy stagnation.

League opponents (24.7%) are a historical snapshots of the agent past versions. Because the agent's policy plateaued early (Finding 1), these snapshots are nearly indistinguishable from the current policy. A league of stagnant policies cannot force adaptation and the fixed ϵ-greedy decay adds to this problem by providing no mechanism for adapting exploration to different opponent since the agent has stopped trying to find different strategy.

5.2.3 Loss-Reward Divergence

The TD loss dropped from 0.05 to 0.001 by episode 37,000. Average episode reward stayed flat near 0.08, a possible confusion in the reward signal. The network learned to predict outcomes of its existing passive play with high precision, knowing that a safe move will net a safe outcome. With the bottoming ϵ-greedy, it did not search for strategies with higher expected returns.

5.2.4 LSTM Accuracy

The piece prediction accuracy hit 20% after episode 5,000 and stagnated for the rest of the training. One correct inference in five is not enough to evaluate combat outcomes reliably, so the agent tend to avoid combat altogether

5.3 Draw Dominance

Draws reached 38.5% of all games, and the win rate was only 22.2%. This rate matters not just as a performance number but as a drag on learning itself, a symptom of a bigger problem.

The scalar value collapse (Finding 2) made the model play safe rather than optimal. A scalar Q-value cannot represent multi-modal outcome distributions. A move with 40% win probability and

60% loss probability (expected value ≈ -0.2) looks worse than a certain draw (-0.3) to a mean-based estimator. The draw-producing move gets the higher Q-value even though it is strategically inferior.

5.4 Architectural Comparison: Unified versus Dual-Model Pipeline

A different approach was use from the initial architecture, which was using two separate networks, a RLNN SetupAgent for piece placement and a DQN for movement. That dual-model design was abandoned, due to the computation overhead and signal interference between the two networks. The current unified pipeline uses a single DQN for all movement and a randomized heuristic for placement. Table 5.2 compares both.

Table 5.2: Dual-model versus unified architectural pipelines.

Aspect	**Dual-Model (Setup + Attack)**	**Unified DQN**
Architecture	Separate C51-based SetupAgent for placement, separate DQN for attack	Single DQN for movement; heuristic setup with randomized flag
Setup Latency	Approximately 2 seconds per episode	Under 100 milliseconds per episode
Training Overhead	Doubled compute requirement (setup and attack training phases)	Halved total training compute
Exploitability	Deterministic setup patterns learned by opponents	Randomized placement resists opponent exploitation
Draw Pattern	Draws due to gradient interference between competing objectives	Draws due to risk-averse policy under uncertainty

Two lessons emerged from this transition. The SetupAgent did not converged on deterministic placements, randomizing setup while lowering the TD-error of the model a mismatch in result. Placing flag in front of the lineup and trapping big pieces behind the bomb. A randomized heuristic eliminated this vulnerability and cut setup latency by 20×.

The dual-model design also suffered from gradient interference. Setup gradients rewarded defensive formations while attack gradients rewarded aggressive piece use. Optimal defense often places pieces in positions bad for attack. Removing the setup phase from the learning loop resolved this conflict.

Both architectures converged on draw-heavy outcomes. In the dual case, gradient interference

between objectives prevented coherent learning. In the unified case, scalar value collapse and poor exploration drove the agent into passive shuffling. The draw problem is architectural and rooted in the long horizon of the game, not an unforeseen accident.

5.5 Architectural Bottleneck Analysis

Each of the findings above can be explained by a known structural property of the DQN architecture. The explanations are hypotheses that the later sections test; they do not rule out other causes, such as the reward design (Appendix C).

5.5.1 Q-Value Overestimation

Finding 3 is a known property of single-estimator Q-learning. The network selects and evaluates actions with the same weights:

$$y_t = r_t + \gamma \max_{a'} Q_\theta(s_{t+1}, a') \tag{5.1}$$

Noisy estimates plus a max operator produce a positive bias, reducing the ability of the agent to learn from its mistake [26]:

$$\mathbb{E}\left[\max_a (Q^*(s,a) + \epsilon_a)\right] \geq \max_a Q^*(s,a) \tag{5.2}$$

The bias grows with noise variance and action-space size $|\mathcal{A}|$. Stratego positions can have over in theory a 220 to 240 legal moves, amplifying the effect. Double DQN splits selection from evaluation across two estimators, provably reducing this bias [26].

5.5.2 Scalar Value Collapse

Finding 2 (loss convergence without performance gain) follows from distributional RL theory. The standard DQN learns a scalar $\mathbb{E}[Z(s,a)]$ and minimizes:

$$\mathcal{L} = \mathbb{E}\left[\left(r + \gamma \max_{a'} Q(s',a') - Q(s,a)\right)^2\right] \tag{5.3}$$

In Stratego the return distribution $Z(s,a)$ is multi-modal [26]: wins near $+1$, losses near -1, exchanges in $[-0.5, +0.5]$, draws near 0. A scalar mean collapses all modes into one number that

may not match any real outcome. The loss converges when this mean is estimated correctly, even if the policy cannot tell aggressive high-variance moves from passive low-variance ones.

5.5.3 LSTM Information Bottleneck

Finding 4 follows from sequential compression. The LSTM encodes observations $o_1, \ldots, o_T$ through a 64-dimensional hidden state:

$$h_t = \text{LSTM}(o_t, h_{t-1}) \tag{5.4}$$

A Stratego game runs 200 to 500 moves. The hidden state must track belief estimates for 40 opponent pieces across 12 types. A 64-dimensional vector can hold enough information in theory, but every successive nonlinear update risks losing early observations. Information from move k must survive $T - k$ transformations to reach the final representation. Gating helps, but retention still degrades with sequence length [22].

AAREN sidesteps this bottleneck. It computes weighted aggregations directly over the full history with $O(1)$ per-timestep complexity:

$$h_t = \frac{a_t}{c_t} = \frac{\sum_{k=1}^{t} v_k \exp(s_k - m_t)}{\sum_{k=1}^{t} \exp(s_k - m_t)} \tag{5.5}$$

No sequential compression, no information decay. The network can attend to any prior timestep directly.

5.5.4 Exploration Collapse

The normal DQN relies on a hard-coded ϵ-greedy exploration schedule that decays from 1.0 down to 0.01. Once it hits this floor, the schedule stops adapting. From that point on, the agent explores uniformly at a flat 1% rate regardless of the game state or its confidence in the current Q-values.

Stratego has over 10^{535} possible states [46]. A uniform random exploration rate of 1% is a blunt mechanism that will almost never stumble on the complex sequences of moves required to discover optimal strategies. The agent becomes trapped in local optima because its exploration policy cannot recognize which parts of the state space require further investigation. This limitation necessitates a mechanism like Noisy Networks [26], which replaces hard-coded randomness with learnable parameters that concentrate exploration where the value function is most uncertain.

5.5.5 Sparse Reward Propagation

The normal DQN cannot develop long-horizon strategic behavior (Findings 1, 5, 6), a limitation compounded by single-step TD learning. Terminal reward information at step T propagates backward through the Bellman backup at a rate set by the discount factor γ:

$$Q(s_k, a_k) \leftarrow Q(s_k, a_k) + \alpha \left(r_k + \gamma Q(s_{k+1}, a_{k+1}) - Q(s_k, a_k)\right) \tag{5.6}$$

For a decisive game outcome occurring at step T in a game of length 300, a mid-game decision at step 100 receives an effective signal of γ^{200}. With $\gamma = 0.99$, this yields $0.99^{200} \approx 0.134$, meaning that only 13.4% of the terminal reward signal reaches mid-game decisions through single-step bootstrapping. Multi-step returns (n-step TD) [26] propagate terminal signals n times faster by directly incorporating rewards over n consecutive steps:

$$y_t^{(n)} = \sum_{k=0}^{n-1} \gamma^k r_{t+k} + \gamma^n Q(s_{t+n}, a_{t+n}) \tag{5.7}$$

This reduces the effective bootstrap depth and accelerates the propagation of sparse terminal rewards to earlier decision points.

5.6 Empirical Validation: Rainbow DQN with AAREN

The bottleneck analysis above predicted that each failure mode has a specific architectural fix. The following data mining analysis tests those predictions empirically. Three training datasets were analyzed: the same normal DQN at 15,302 episodes, the Rainbow DQN with AAREN at 9,778 episodes of self-play, and a separate extended Rainbow run of 75,047 episodes against heuristic opponents. Table 5.3 presents the head-to-head comparison. The complete training progress charts appear in Appendix B (Figures B.12–B.19).

Three limits apply to every comparison in this section. First, the runs differ in more than the architecture: the opponents, curriculum, reward and run length differ too. Each configuration was also trained once, so a difference cannot be attributed to a single component. Second, the self-play figures come from the training games, in which the agent plays a copy of itself, not from evaluations against fixed opponents. Third, the average Q-value logged by the Rainbow trainer was the maximum over all 3,600 actions. Most of those actions are illegal and never trained, and Appendix C shows

this statistic read about +2 while the returns were about −1. The Q-values below therefore do not measure the values of the moves that were played.

Table 5.3: Self-play performance comparison between Vanilla DQN with LSTM and Rainbow DQN with AAREN.

Metric	**DQN+LSTM (15k)**	**Rainbow+AAREN (9.5k)**
Total Episodes	15,302	9,778
Total Steps	10,306,433	5,169,892
Win Rate (%)	24.97	49.88
Draw Rate (%)	50.04	6.69
Loss Rate (%)	24.99	43.43
Steps per Win	2,697	1,060
Avg Episode Length	1,345	1,055
Flag Capture Ratio (%)	18.79	8.88
Loss–Win Rate Corr.	0.051	−0.787
Mean Q-Value	0.33	0.70
Model Size (per ckpt)	∼39 MB	∼433 MB

5.6.1 Win Rate

The Rainbow agent won 49.88% of its self-play games at 9,778 episodes, against 24.97% for the DQN at 15,302 episodes. In self-play this number is set mostly by the draw rate. When both sides play the same policy, the decisive games split about evenly between them, as they did in both runs: the DQN won 24.97% and lost 24.99%, and the Rainbow agent won 49.88% and lost 43.43%. The win rate doubled because the share of decisive games nearly doubled (from 50% to 93%), not because the agent became stronger than its opponent. Showing that the Rainbow agent plays better than the DQN would need games between the two, or against the same fixed opponents.

5.6.2 Draw Elimination

The normal DQN drew 50.04% of its games, finishing the 1500 moves per agent in an episode. The Rainbow agent drew 6.69% over 1500 moves. It lost more games outright (43.43% versus 24.99%), but it produced definitive outcomes. An agent that draws half of its games has learned to avoid playing passively and conservative in calculating outcomes in the short term. Section 5.1.6 (Draw Dominance) attributed the draws to scalar value collapse. These runs cannot tell which change removed them, because the configurations differ in more than the value head. The effect also did

not carry over to the 75k run against heuristic opponents, which drew 77.02% of its games (Table 5.4).

A 43.43% loss rate alongside a 6.69% draw rate could suggest that the Rainbow agent takes excessive risks. In self-play, however, every decisive game has one winner and one loser, and both are copies of the same agent. The 93.31% decisive outcomes mean that the policy forces resolutions rather than stalling. They say nothing about whether its play is reckless or sound against an opponent that is not itself. The self-play games were also shorter (1,055 moves on average, against 1,345 for the normal DQN).

5.6.3 Loss–Performance Correlation

The correlation between windowed policy loss and win rate was 0.051 for the normal DQN and -0.787 for the Rainbow agent. Two curves that both drift with training time correlate largely through that common drift, whatever their relation. The later formal_4 run (Rainbow DQN, AAREN off) shows how far this goes. Over blocks of 100 episodes its loss and training win rate correlated at -0.49, but the correlation fell to -0.14 once a linear trend was removed from both. Its loss fell from 3.63 to 3.05 while its greedy evaluations against fixed opponents did not improve (Appendix C). The -0.787 was not computed on detrended curves, and in self-play the win rate mostly reflects the draw rate (Section 5.6.1). The coefficient therefore does not show that the Rainbow loss tracks the quality of play. The decoupling of Section 5.1.2 appears in the Rainbow learner as well.

5.6.4 Sample Efficiency

The Rainbow agent required 1,060 environment steps per win compared to 2,697 for the normal DQN. At the episode level, the Rainbow agent won once every 2.0 episodes while the normal DQN won once every 4.0, meaning the DQN architecture stumbled its way to a win due to the maximum step is 3000 steps per episode. The Rainbow model consumed 5.17 million total steps; the normal DQN consumed 10.31 million. Steps per win is the total number of steps divided by the number of wins, and in self-play the win rate rose with the share of decisive games (Section 5.6.1). The metric therefore reflects fewer draws and shorter games more than it reflects learning more from each sample.

5.6.5 AAREN and LSTM Accuracy

The AAREN module achieved 24.23% piece identity inference accuracy, compared to 17.74% for the LSTM-based PBS module. Three things limit what this gap shows.

First, the 8.3% baseline of Section 5.1.4 is a uniform guess over 12 types, but the types are not equally common. Scouts are 8 of the 40 pieces (20%) and 8 of the 33 movable ones (24%). A predictor that always names the most common type would therefore score about 20 to 24%, depending on which pieces battles reveal. Until both accuracies are compared with that majority baseline on the same reveals, 24.23% is not yet evidence of inference from behaviour.

Second, the two accuracies come from different runs, with different opponents, lengths and networks. Third, in the thesis code AAREN read the padding step of every history shorter than the longest in its batch instead of its last real step, and the DQN loss did not train it (Chapter 4, Gradient Bridge). Section 5.1.4 (LSTM Accuracy Ceiling) predicted that attention would help, but this result neither confirms nor refutes that prediction.

The Q-values cannot be compared either. The LSTM Q-values settled near 0.18 and the Rainbow ones reached 0.84, but the Rainbow trainer's average Q was a maximum over mostly untrained actions.

5.6.6 Learned Exploration Active

The NoisyNet entropy rose steadily from 0.034 to 0.049 throughout Rainbow training, adapting to the need for exploration during the training as the agent have not converge to a solution. The normal DQN has a fixed entropy of zero (Section 5.1.5, Exploration Collapse), due to the constant decay of ϵ-greedy. Noisy Networks replaced frozen ϵ-greedy decay with adaptive, state-dependent exploration.

5.6.7 Extended Training at 75,000 Episodes

A separate Rainbow DQN with AAREN training run reached 75,047 episodes against heuristic opponents. This run used a different curriculum configuration and did not advance to self-play, so it cannot be directly compared to the self-play results above. It does, however, validate two architectural properties over a longer horizon. Table 5.4 summarizes its metrics.

Table 5.4: Rainbow DQN with AAREN extended training (75k episodes) against heuristic opponents.

Metric	**Value**
Total Episodes	75,047
Win Rate	11.66%
Draw Rate	77.02%
vs Smart Heuristic WR	11.74% (38,462 games)
vs Frozen Heuristic WR	11.59% (43,331 games)
Policy Loss (late)	2.84
Q-Value (late)	1.84
AAREN Accuracy	20.78%
Flag Capture Ratio	99.96%
Avg Episode Length	378

Loss Convergence Over Long Horizons

The distributional loss fell from 3.89 to 2.84 across 75,000 episodes. The normal DQN showed no comparable long-term loss reduction; its loss either plateaued or inflated in late training. A C51 loss of $\ln 51 \approx 3.93$ is what a prediction spread evenly over the 51 atoms scores, so the run started from an almost uniform prediction. The falling loss shows the critic fitting its own targets more closely. A gradual increase in win rate was seen across the second half of the run, but the run's win rate against heuristic opponents was 11.66% overall. As in formal_4 (Section 5.6.3), the loss can fall while the agent keeps losing, so the falling loss alone is not evidence that extended training improves play.

Flag Pursuit Behavior

The average episode length in the 75k run settled near 378 steps, compared to 1,345 for the normal DQN and 1,055 for the Rainbow self-play run. The shorter episodes indicate that the agent ended games faster. Of its wins, 99.96% came by capturing the opponent flag rather than by attrition. The normal DQN won by flag capture only 18.79% of the time, relying instead on piece depletion. The logged average Q-value grew from near zero to 1.84. With a win worth +1, a value of 1.84 at an 11.66% win rate cannot be the value of the moves played; it is the maximum over mostly untrained actions described above. The attention layer may help the agent locate the flag, but no run has isolated its contribution.

The 75k run is also the more informative measure of strength. In self-play a win rate near 50%

follows from symmetry (Section 5.6.1). Against fixed heuristic opponents, the agent won about one game in nine (11.66%) and drew 77.02% across 75,047 episodes. It did as well against the smart heuristic (11.74%) as against the frozen one (11.59%). The later formal runs (Appendix C) also struggled against these opponents. formal_4 still lost as many games as it won against a random opponent at episode 3,000, mostly because it attacked revealed pieces it could not beat.

5.7 Evidence for Architectural Necessity

Each component in the thesis title maps to a specific failure mode observed in the normal DQN baseline. Table 5.5 summarizes this mapping and the evidence so far from the Rainbow DQN runs.

Table 5.5: MARQ title components mapped to normal DQN failure modes and the Rainbow DQN evidence so far.

Title Component	**Observed Failure Mode**	**Empirical Evidence**
Multi-Agent	Symmetric MARL Nash lock-in	P1 and P2 win rates were 25.1% and 24.9% in the normal DQN. In Rainbow self-play they were 49.88% and 43.43%, still close, as self-play implies; what changed is that draws fell from 50% to 6.69% (Section 5.6.1).
Rainbow DQN	Q-value overestimation, scalar value collapse, exploration death, sparse reward propagation	Self-play draw rate dropped from 50% to 6.69% (Section 5.6.2), but the 75k run drew 77%. NoisyNet entropy rose throughout training (Section 5.6.6). The loss–win correlation ($r = -0.787$) was not detrended and does not show coupling (Section 5.6.3).
AAREN	Sequential information bottleneck	AAREN accuracy 24.23% versus LSTM's 17.74% (Section 5.6.5), not yet compared with a majority-class baseline of about 20–24%.
Imperfect Information	Risk-averse draw-seeking policy	The self-play draw rate fell from 38.5% (Section 5.1.6) to 6.69% (Section 5.6.2). Against heuristic opponents the 75k run drew 77.02% and won 11.66%, 99.96% of its wins by flag capture (Section 5.6.7).

The combined findings are consistent with the MARQ architecture, but they do not yet establish what each component contributes. Each component targets a distinct bottleneck. Draw dominance

(Section 5.1.6) is plausibly a compound effect of scalar value collapse, exploration death and sparse reward propagation. Under the combined changes the self-play draw rate fell from 50.04% to 6.69% (Section 5.6.2). No ablation separates the effects of those changes, however, and the run against heuristic opponents still drew 77% of its games.

5.8 Supporting Evidence from Related Work

These limitations are consistent with findings elsewhere in deep RL for imperfect information games.

Pérolat et al. (2022) solved full-size Stratego with DeepNash [46] using game-theoretic regularization (R-NaD), not Q-value maximization. DeepNash shows Stratego is solvable by deep RL, but only when the architecture explicitly handles non-stationarity and strategic diversity. The Ataraxos system [50] extended this to no-press Diplomacy with KL-divergence stabilization, a mechanism similar to MARQ's dynamic damping but absent from the normal DQN baseline.

Hessel et al. (2021) showed in the Rainbow DQN study that Double DQN, PER, Dueling Networks, Noisy Nets, C51, and multi-step returns each contribute additive gains [26]. Removing any single component degraded performance. No one fix is enough.

5.9 Implications for MARQ

The analysis attributes the normal DQN's performance ceiling to identifiable architectural properties. Table 5.6 maps each finding to its theoretical cause, the MARQ component meant to address it, and how far the data so far supports that.

"Not tested" means that no run isolated the component. Overestimation would need the values of the moves played compared with the returns that followed. The trainer's logged Q was a maximum over mostly untrained actions, and the win rate of Section 5.6.1 does not measure bias. The steps-per-win figure of Section 5.6.4 reflects the draw rate, not how far rewards propagate.

The normal DQN baseline in this chapter served as a reference point. In self-play the Rainbow agent with AAREN produced far fewer draws (6.69% against 50.04%) and shorter games, and its exploration noise stayed active. Against heuristic opponents it won 11.66% of 75,047 games. Two questions remain open. What each component contributes needs ablations evaluated against fixed opponents. Whether AAREN supplies information the observation does not needs runs with and

Table 5.6: Observed limitations mapped to theoretical causes, MARQ components, and validation status.

Finding	**Theoretical Cause**	**MARQ Component**	**Status**
Q-Value Overestimation	Maximization Bias	Double DQN / Dueling	Not tested
Loss–Perf. Decoupling	Scalar Value Collapse	C51 Distributional RL	Not supported (Sec. 5.6.3)
LSTM Accuracy Ceiling	Sequential Bottleneck	AAREN Attention	Inconclusive (Sec. 5.6.5)
Exploration Collapse	Fixed ϵ-greedy	Noisy Networks	Supported (Sec. 5.6.6)
Reward Propagation	1-step Bootstrapping	n-step Returns	Not tested
Draw Dominance	Compound Effect	Full MARQ Pipeline	Self-play only (Sec. 5.6.2)

without it, compared on the same evaluations. The formal runs of Appendix C begin that work, so far with AAREN switched off.

Chapter 6

Conclusion and Recommendations

6.1 Conclusion

The experiments presented in this thesis set out to determine whether the performance ceiling of a normal DQN agent in Stratego arises from insufficient training or from structural properties of the architecture itself. The evidence supports the latter interpretation. Across 37,213 episodes of curriculum training, the normal DQN exhibited a stable 22% win rate, monotonically decreasing TD loss with no corresponding performance gain, Q-value overestimation, piece prediction accuracy frozen near 20%, and a 38.5% draw rate driven by passive piece shuffling. These observations are not independent failures. They form a coherent pattern: scalar value collapse prevents the agent from distinguishing aggressive high-variance moves from passive low-variance alternatives, exploration decay eliminates the mechanism for discovering new strategies, and the LSTM information bottleneck denies the agent the opponent model needed to evaluate combat outcomes.

The Rainbow DQN with AAREN targeted each of these bottlenecks with an architectural change. C51 distributional value estimation replaced scalar Q-learning. Noisy Networks replaced frozen ϵ-greedy exploration with adaptive, state-dependent noise, which stayed active throughout training. AAREN replaced the LSTM history encoder; its piece-identity accuracy was 24.23%, against 17.74% for the LSTM. Multi-step returns shortened the effective bootstrap depth for sparse terminal rewards. In self-play the combined changes cut the draw rate from 50.04% to 6.69% and shortened the games. The self-play win rate rose from 24.97% to 49.88%, but in self-play both sides are the same agent,

so that rise reflects the decisive share of games rather than greater strength (Section 5.6.1).

The transition from draw-dominated outcomes to decisive games is the most consequential behavioral shift observed in the data. The normal DQN converged on a policy that minimized short-term risk at the cost of long-term winning probability. The Rainbow agent stopped doing so in self-play. Against fixed heuristic opponents, however, the extended 75,047-episode run won 11.66% of its games and drew 77.02%. Of its wins, 99.96% came by flag capture, and its average episode lasted 378 steps. Whether the agent learned to identify and pursue the flag, or only ended the games it won by that route, these figures cannot tell.

These results connect each architectural component to a failure mode of the baseline. They do not yet measure each component's contribution, for three reasons. The configurations differ in more than one component, and no ablation was run. Several of the measures used (the loss–win correlation, the logged Q-values, the self-play win rate) turned out not to measure what they were taken to measure (Chapter 5). The following sections present the individual components, the broader methodological contributions, and the remaining limitations.

6.2 Recommended Architectural Enhancements

6.2.1 Value Estimation

The Q-value overestimation and scalar value collapse identified in the training analysis directly motivated two Rainbow DQN enhancements formalized in the MARQ architecture. The Dueling Network decomposes action values into state-value and advantage streams. This design suits Stratego's action space, where many positional moves share similar strategic value and only a small subset of actions (attacks, flag-adjacent moves) carry distinct utility. The C51 distributional head provides multi-modal value estimation that differentiates risky decisive actions from safe passive alternatives. Scalar Q-values provably cannot make this distinction.

The empirical results are consistent with this design but do not isolate it. The correlation between loss and win rate shifted from 0.051 (normal DQN) to -0.787 (Rainbow DQN with AAREN). Both curves drift with training time, though, and in the later formal_4 run the correlation shrank from -0.49 to -0.14 once that drift was removed, while the loss fell without any gain in evaluation (Section 5.6.3). The self-play draw rate dropped from 50.04% to 6.69%, but the 75k run against heuristics drew 77.02%. Whether multi-modal value estimation by itself resolves the draw-seeking

trap needs an ablation that changes only the value head.

6.2.2 LSTM to AAREN Transition

The LSTM prediction accuracy ceiling of approximately 18 to 22% was the most direct evidence for the AAREN module's necessity. The LSTM-based history encoder has concrete advantages over a full DRQN architecture in replay compatibility, per-position granularity, and gradient isolation. The sequential information bottleneck inherent in any recurrent encoder, however, limits representational capacity over long game horizons. AAREN preserves all structural advantages of the separated encoder design while replacing the recurrent computation with attention-based history aggregation. This substitution addresses the compression bottleneck without modifying the replay buffer infrastructure or the DQN backbone.

AAREN achieved 24.23% piece identity inference accuracy compared to 17.74% for the LSTM, a 6.5 percentage point difference. This does not yet validate the transition. Always naming the most common piece type would score about 20 to 24%, the two runs differed in more than the encoder, and the thesis code took AAREN's embedding at a padding step (Section 5.6.5). The separated encoder design itself is a useful structure. Independent hidden states for each of the 100 board positions produce a spatially structured $(64, 10, 10)$ embedding tensor that preserves piece-level temporal information. A monolithic DRQN hidden state would conflate action histories across all 40 opponent pieces into a single vector, losing spatial specificity. The per-position design also maintains full compatibility with standard experience replay by storing compact embedding snapshots alongside 15-channel board tensors, avoiding the sequence burn-in cost required by full DRQN architectures.

6.2.3 Exploration Recovery

The Noisy Net sigma remained at zero throughout the entire normal DQN training run. This confirmed that the ϵ-greedy exploration schedule exhausted its utility without the agent discovering strategies beyond the initial local optimum. Noisy Networks parameterize exploration as learnable per-weight perturbations and provide state-dependent exploration that adapts to the agent's uncertainty rather than following a predetermined decay schedule.

The Rainbow agent's NoisyNet entropy rose steadily from 0.034 to 0.049 throughout training, so the learned exploration noise did not collapse. How much this exploration contributed to the change

in outcomes is not known without an ablation.

6.2.4 Long-Range Credit Assignment

Single-step TD learning attenuates terminal rewards by $\gamma^{200} \approx 0.134$ for mid-game decisions. Five-step returns ($n = 5$), as specified in MARQ, shorten the effective bootstrap depth and push strategic outcomes back to earlier decision points faster. The steps-per-win figure (1,060 versus 2,697 for the normal DQN) does not test this. In self-play it follows the draw rate (Section 5.6.4), so whether multi-step returns accelerated reward propagation remains to be measured.

6.3 Computer Science Contributions

The following contributions apply to reinforcement learning systems beyond the Stratego domain. Stratego is the test domain, but the techniques are not domain-specific and apply to other partially observable or adversarial settings.

6.3.1 Five-Phase Curriculum Design

The curriculum transitions agents from full to partial observability in stages. Observability drops progressively while opponent difficulty rises and reward shaping decays. The intent is to mitigate the cold-start problem, so that agents do not need to learn basic mechanics and hidden-information reasoning at the same time. The later runs found a cost to this structure (Appendix C). Changing the observation and the shaping weight between phases made the network relearn the game at each switch. The current opponent schedule therefore keeps fog of war, the turn limit and the shaping fixed, and varies only the opponents.

6.3.2 Embedding Snapshots for Replay Compatibility

Hybrid architectures face a tension between replay accuracy and speed. Reconstructing full history sequences at replay time gives correct gradients but costs 4× throughput. Storing detached snapshot embeddings keeps throughput, but the DQN loss can then no longer train the encoder. The thesis code tried to restore that path with a "gradient bridge", adding $\epsilon \cdot W_{proj}(\mathbf{0})$ to the stored embeddings. That term equals the projection's bias, so it reached only that one parameter tensor and nothing

that depends on the history (Chapter 4). The design that works is a separately supervised encoder: the snapshots are plain inputs to the Q-network, and the encoder is trained by its own loss on revealed pieces. Any encoder that is meant to learn from the value loss should be checked by listing which of its parameters actually receive a gradient.

6.3.3 Expectamax Search Integration

At test time, the system blends the DQN's positional Q-values with combat utilities derived from AAREN's rank-probability predictions through Expectamax search. This grounds neural value estimates in tactical reality and blocks overconfident attacks on hidden pieces. The approach transfers to any partially observable domain that combines learned value functions with probabilistic state inference.

6.3.4 Potential-Based Reward Shaping with Phase Annealing

PBRS guarantees policy invariance while providing dense signals during early training. A multiplicative schedule tapers the shaping coefficient to zero across curriculum phases, so the final training phase runs on terminal rewards alone. No manual cutoff is needed. The design avoids the reward-farming problem common in additive shaping.

6.4 Limitations and Future Work

The validation presented in this thesis was bounded by the training infrastructure, opponent diversity, and compute budget available during the research period. The following limitations qualify the conclusions drawn above and identify directions where additional work is needed.

The Rainbow DQN with AAREN produced far fewer draws than the normal DQN baseline in self-play. The two were never evaluated against each other or against the same fixed opponents, and against heuristic opponents the Rainbow agent won 11.66% of its games. Performance against opponents beyond the current heuristic and self-play pool may reveal gaps in strategic reasoning that the existing evaluation does not expose.

Combining C51, Noisy Nets, Dueling Networks, AAREN, and n-step returns in a multi-agent self-play loop introduces interaction effects that single-component theory does not predict. The extended 75k training run remained in the partial observability phase against heuristic opponents

for its entire duration, suggesting that curriculum promotion thresholds require further tuning for long-horizon training stability.

The full MARQ pipeline is heavier than the normal DQN. Each checkpoint occupies approximately 433 MB compared to 39 MB, an 11-fold increase. While sample efficiency improved (5.17 million steps versus 10.31 million for the normal DQN), wall-clock time per episode is higher, reducing total episode count under a fixed compute budget.

Ablation studies are needed. The data does not yet show that the combined architecture plays better than the normal DQN, nor what each component contributes. Future work should test each component in isolation, evaluated in greedy games against the same fixed opponents, with enough games for a difference to exceed its noise. AAREN in particular needs paired runs with and without it, and a majority-class baseline for its accuracy. Beyond fixed opponents, the agent's exploitability should be measured by training a fresh exploiter against the frozen policy. A deterministic greedy policy can be exploited in a game of hidden information, and methods built for such games (NFSP, PSRO, R-NaD) add an averaged or regularized policy for that reason.

Appendix A

Gantt Chart

This section includes the images of the Gantt chart that will represent the timeline for the whole duration of this study.

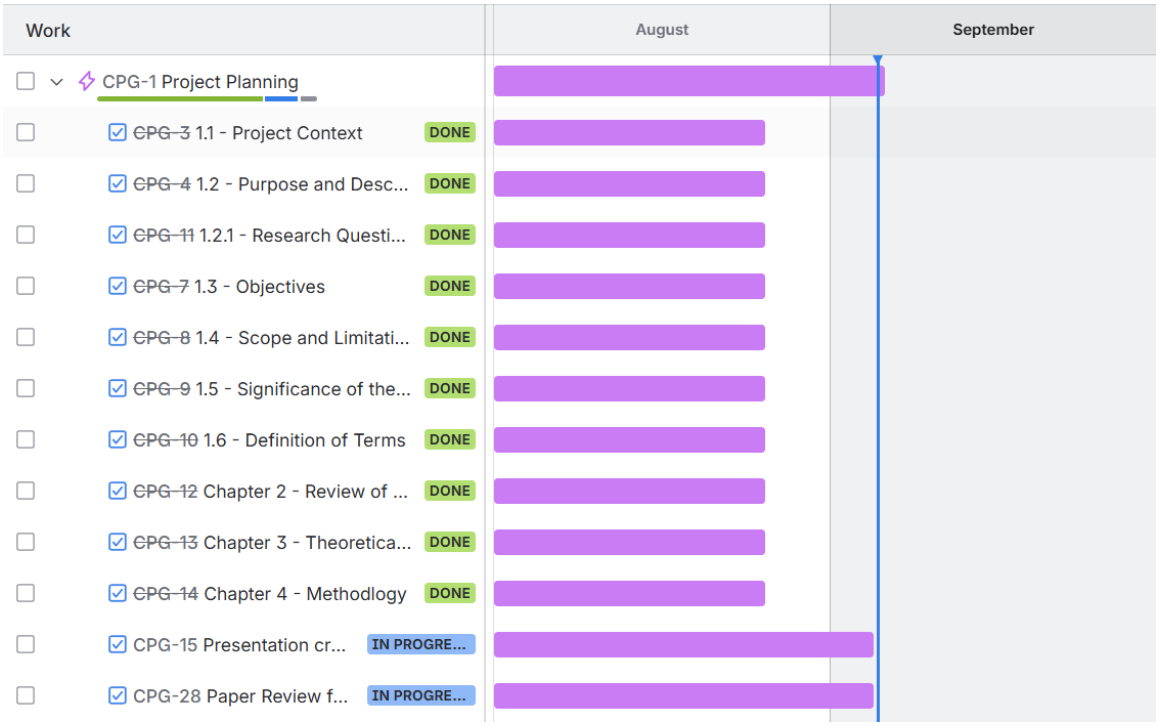


Figure A.1: Timeline: August 1 - September 5



Figure A.2: Timeline: September 11 - November 30



Figure A.3: Timeline: October 27 - January 27

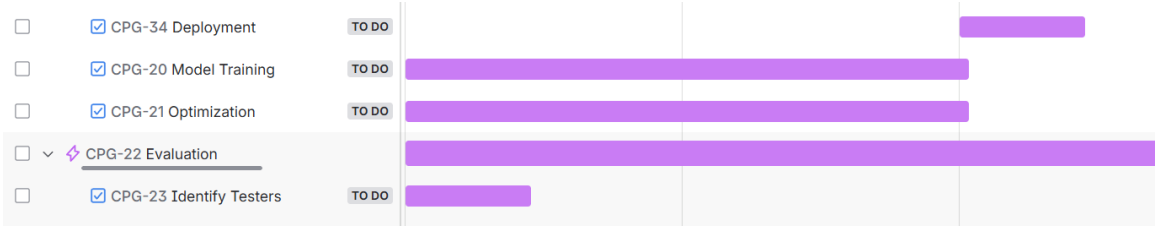


Figure A.4: Timeline: December 1 - February 14



Figure A.5: Timeline: February 15 - May 4

Appendix B

System UI and Training

This section includes the images of the system UI and Training.

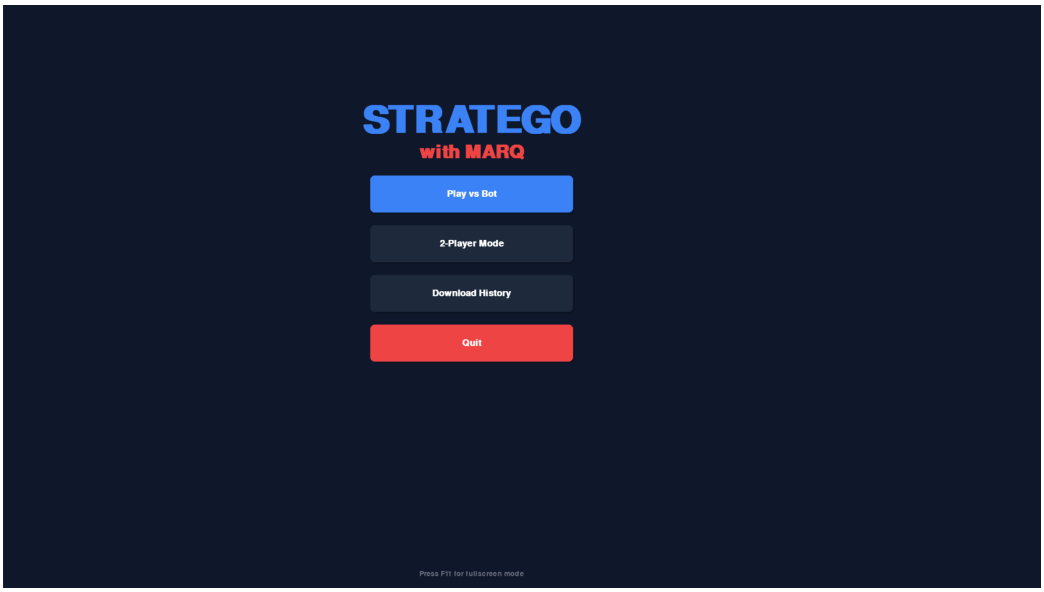


Figure B.1: Game Menu



Figure B.2: Game Rules and Instructions

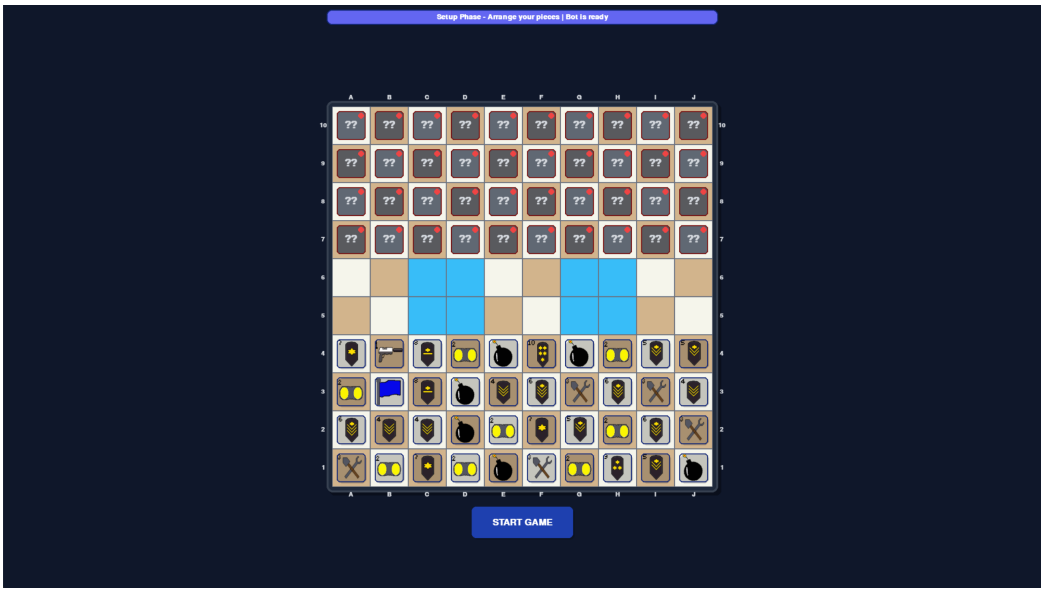


Figure B.3: Setup Phase

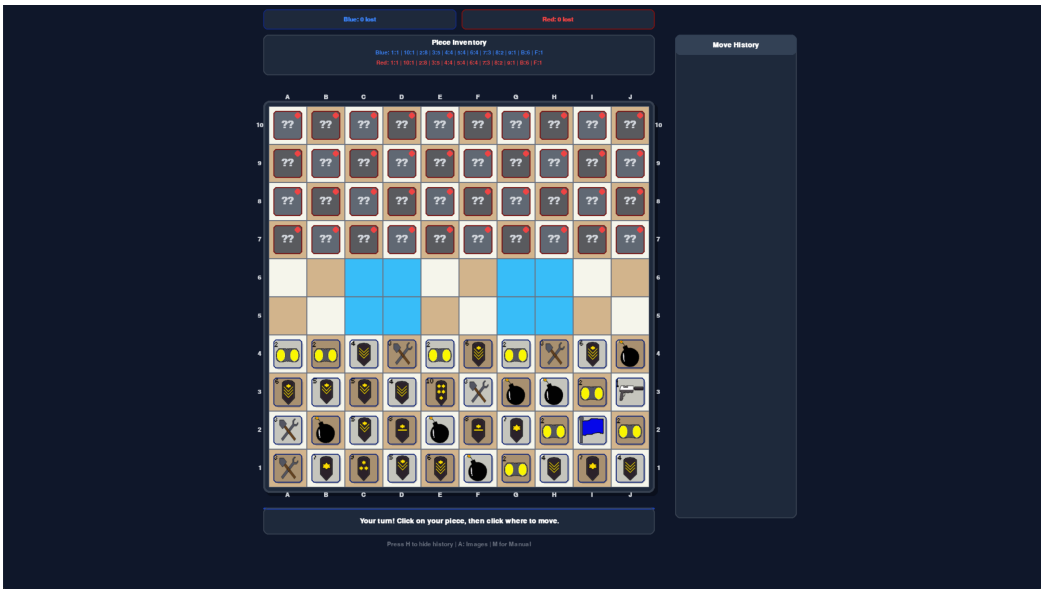


Figure B.4: Game Start

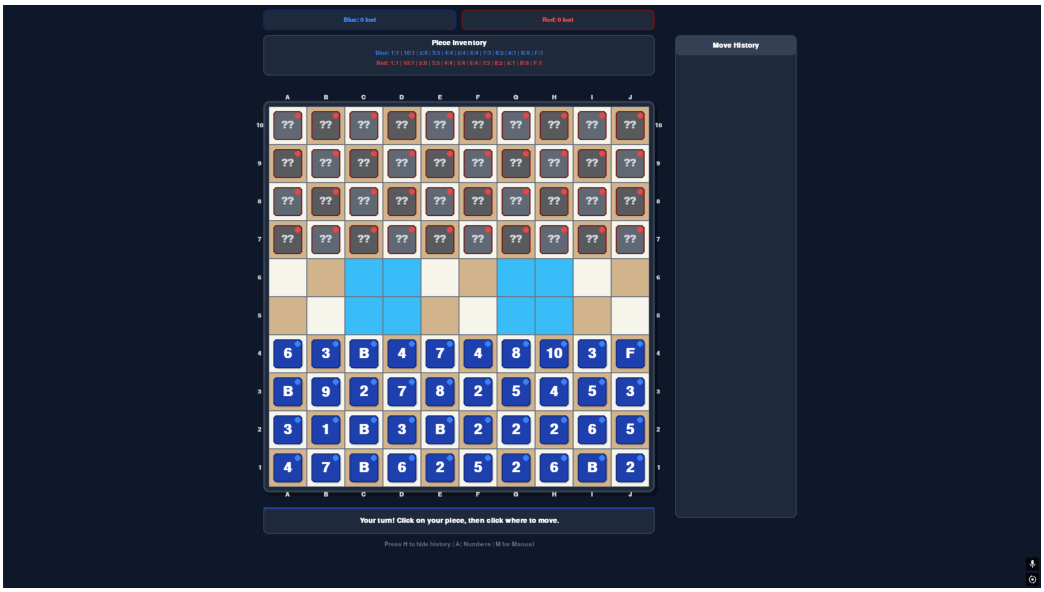


Figure B.5: Change image of pieces to numbers

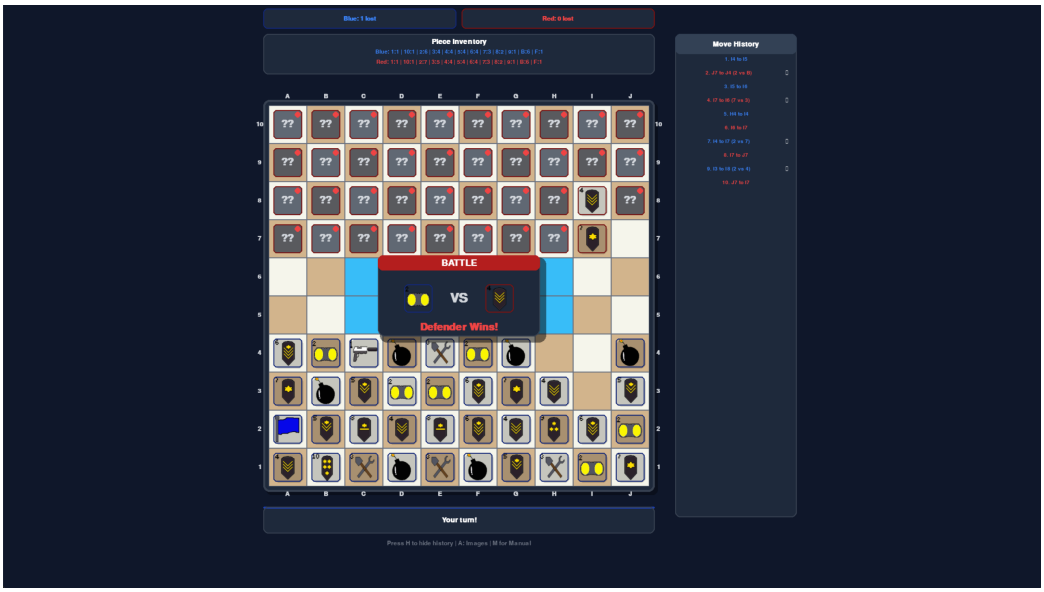


Figure B.6: Battle and History

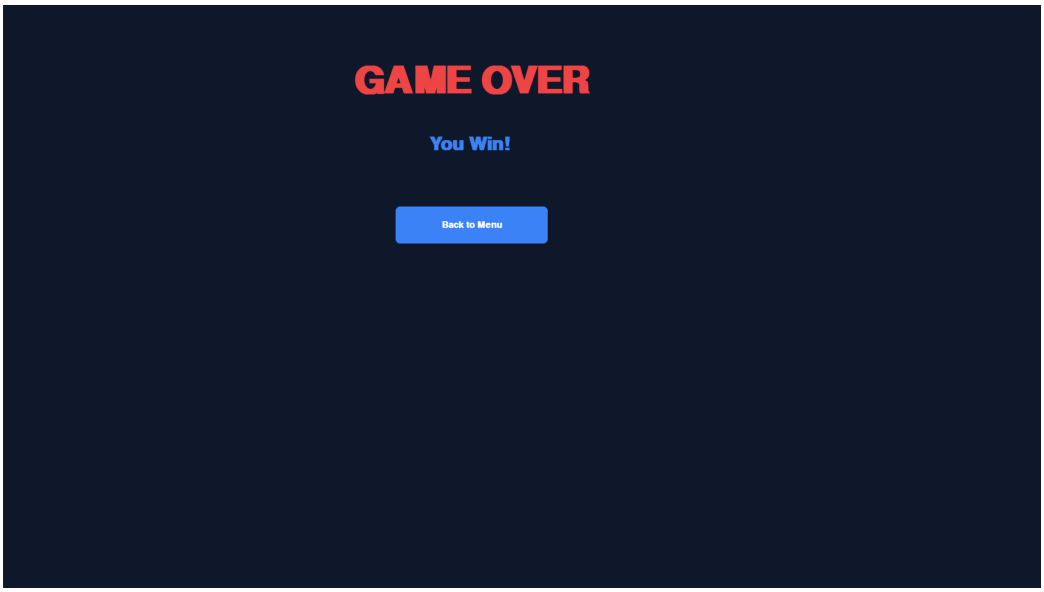


Figure B.7: Victory Screen

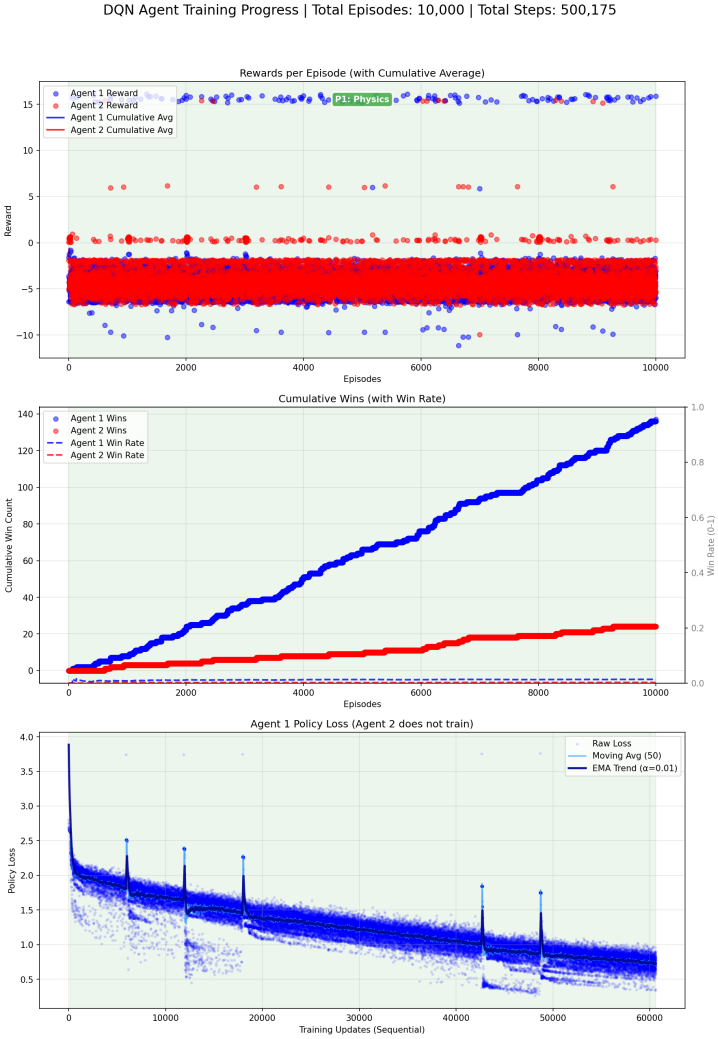


Figure B.8: Training Progress: Episode 10,000

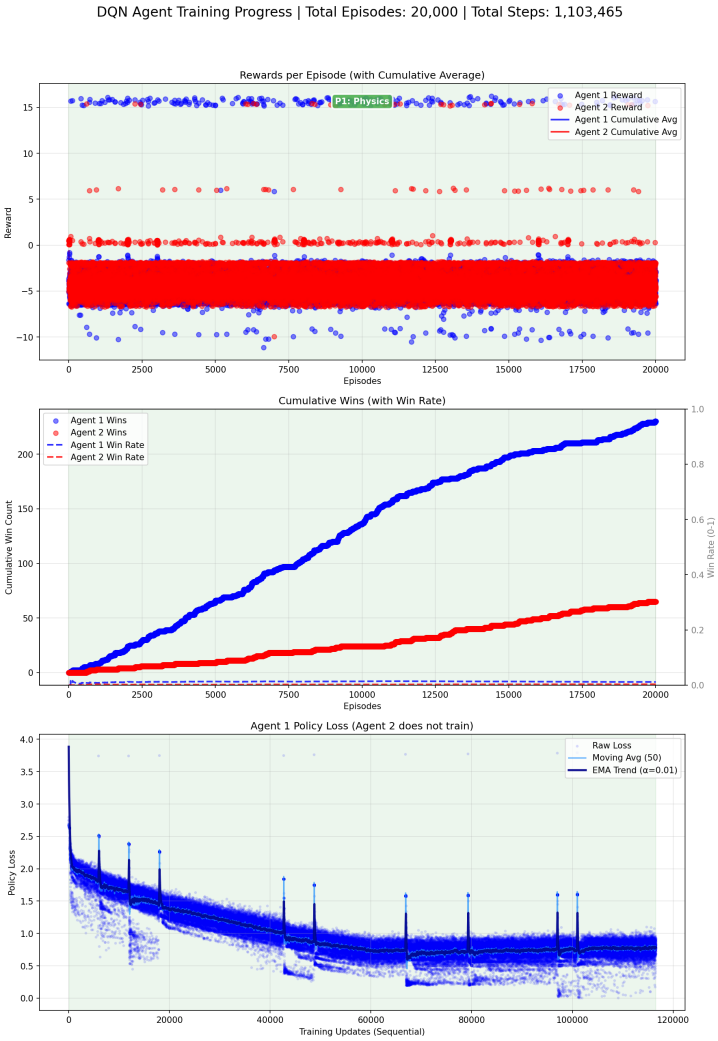


Figure B.9: Training Progress: Episode 20,000

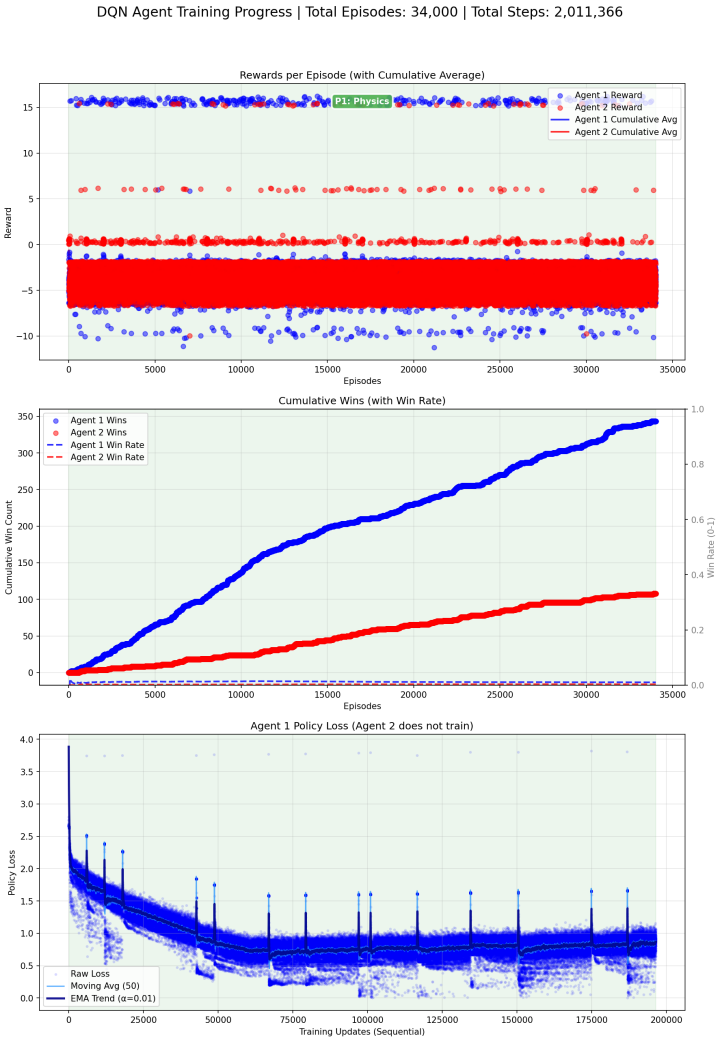
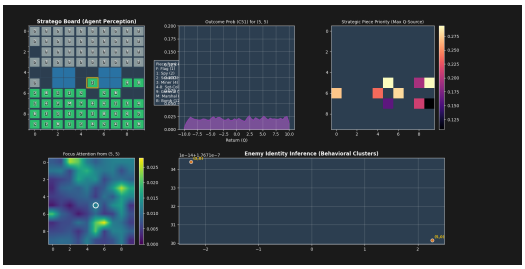


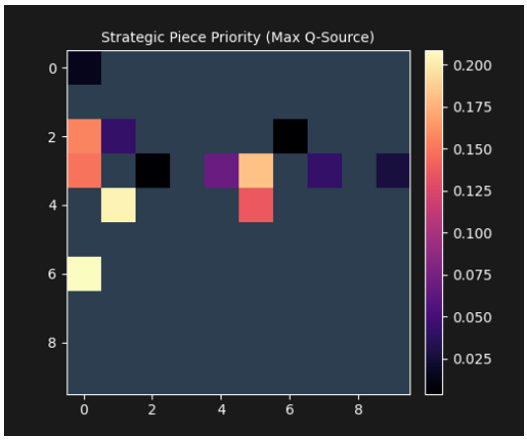
Figure B.10: Training Progress: Episode 34,000

B.1 Interpretability Dashboard

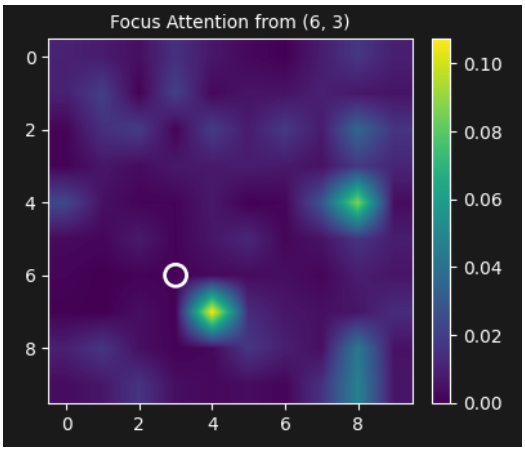
Figure B.11 shows the real-time interpretability dashboard, which projects the agent's internal representations into four visual panels: the C51 return distribution, a Q-value heatmap, spatial attention weights, and PCA-reduced AAREN embeddings.



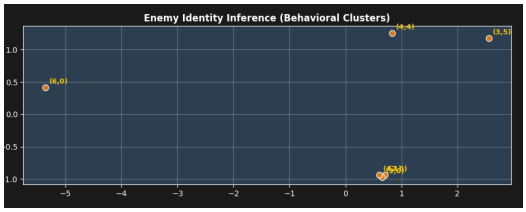
(a) Dashboard Overview



(b) Q-Value Heatmap



(c) Spatial Attention



(d) Latent Trajectories

Figure B.11: Interpretability dashboard snapshots.

B.2 Architecture Comparison Training Charts

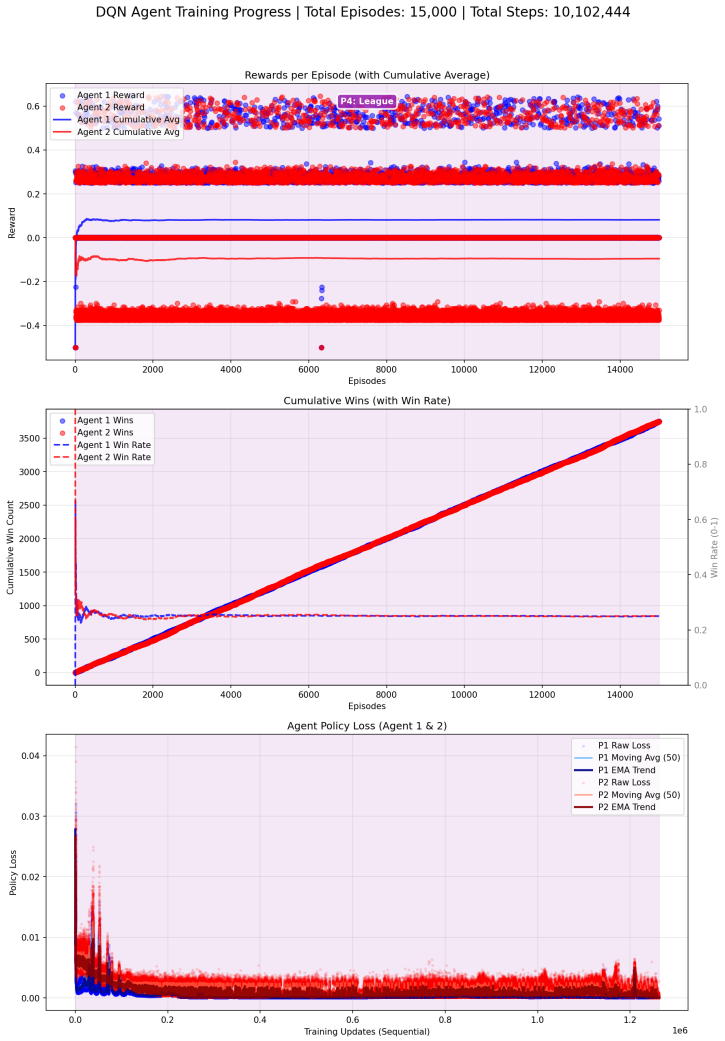


Figure B.12: DQN+LSTM Training Progress at 15,302 Episodes

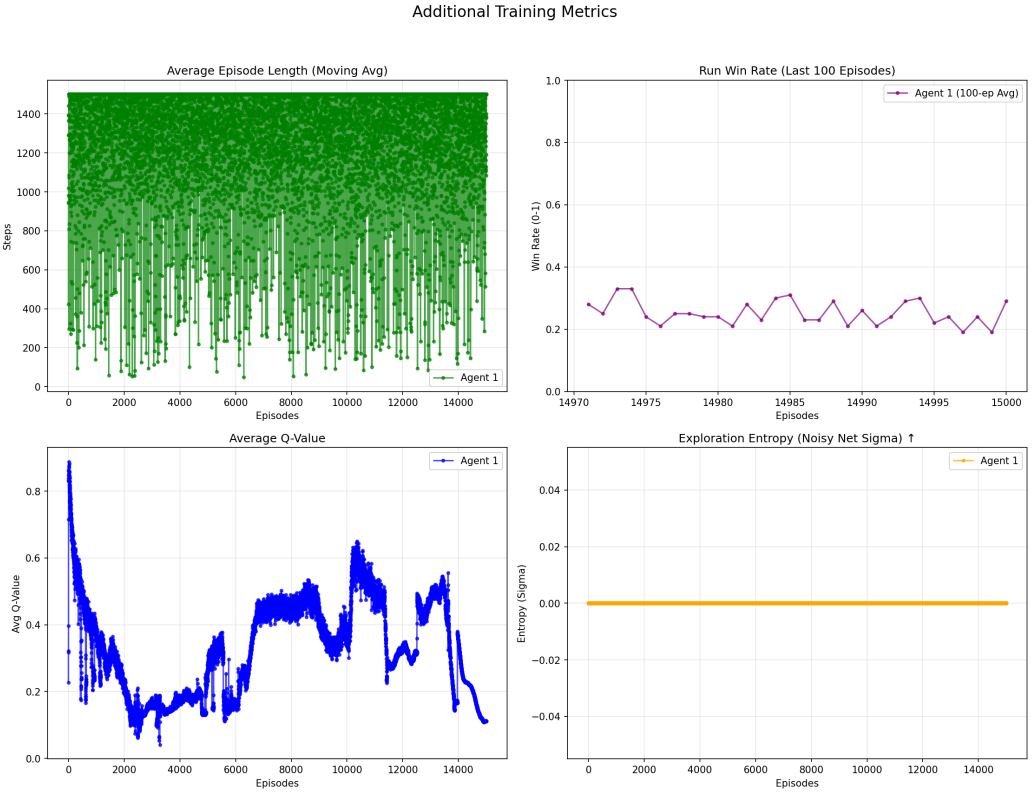


Figure B.13: DQN+LSTM Additional Metrics at 15,302 Episodes

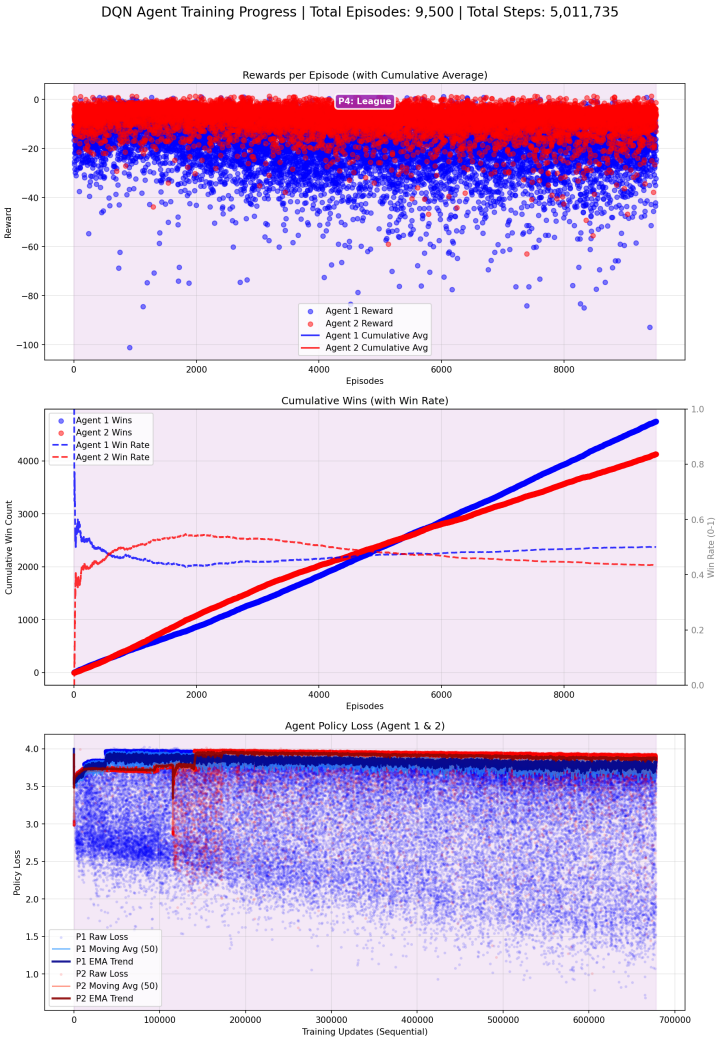


Figure B.14: Rainbow DQN+AAREN Training Progress at 9,778 Episodes

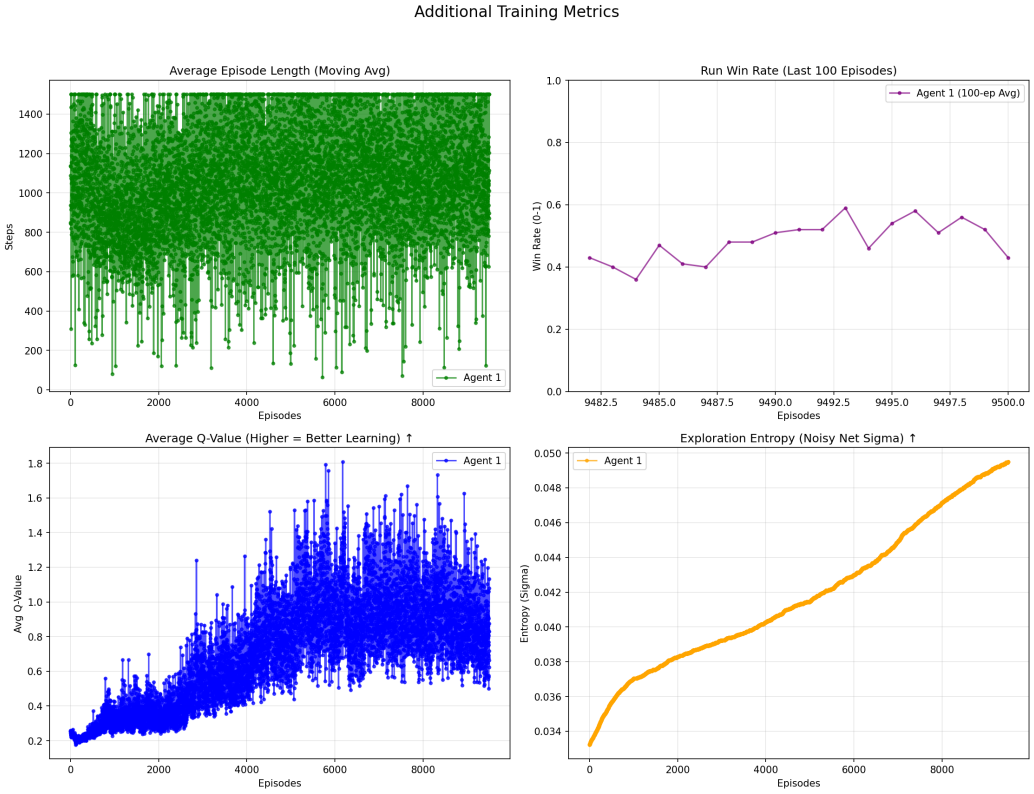


Figure B.15: Rainbow DQN+AAREN Additional Metrics at 9,778 Episodes

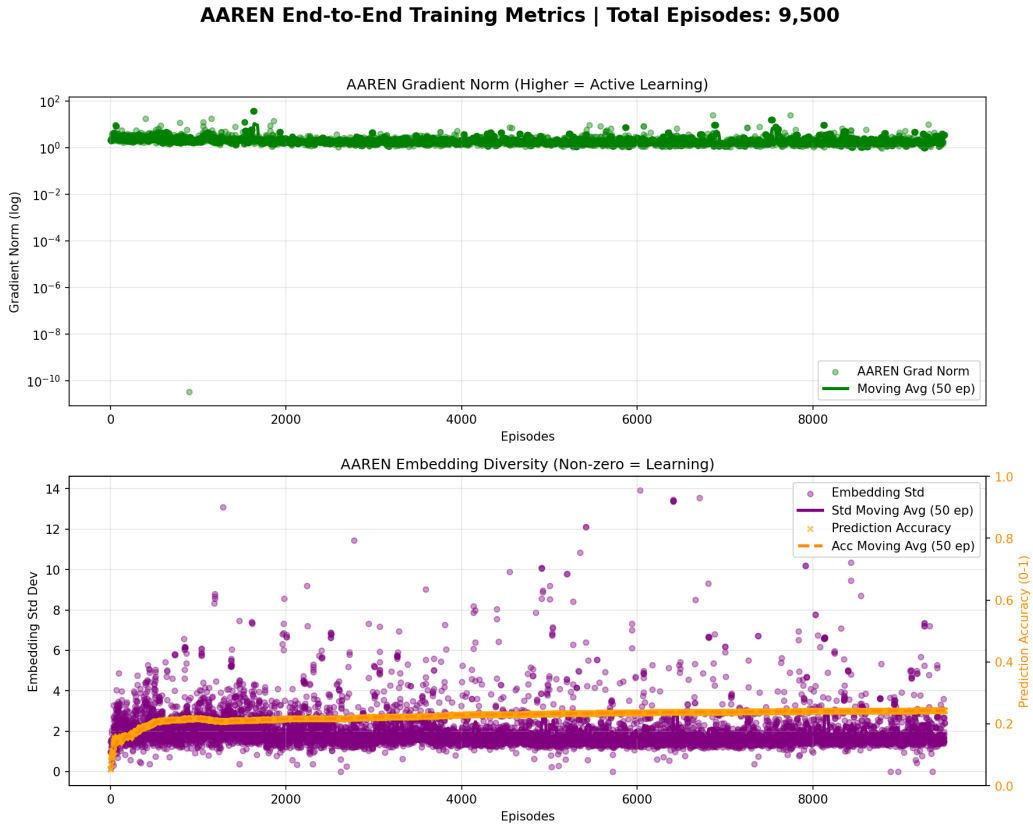


Figure B.16: AAREN End-to-End Training Metrics at 9,778 Episodes

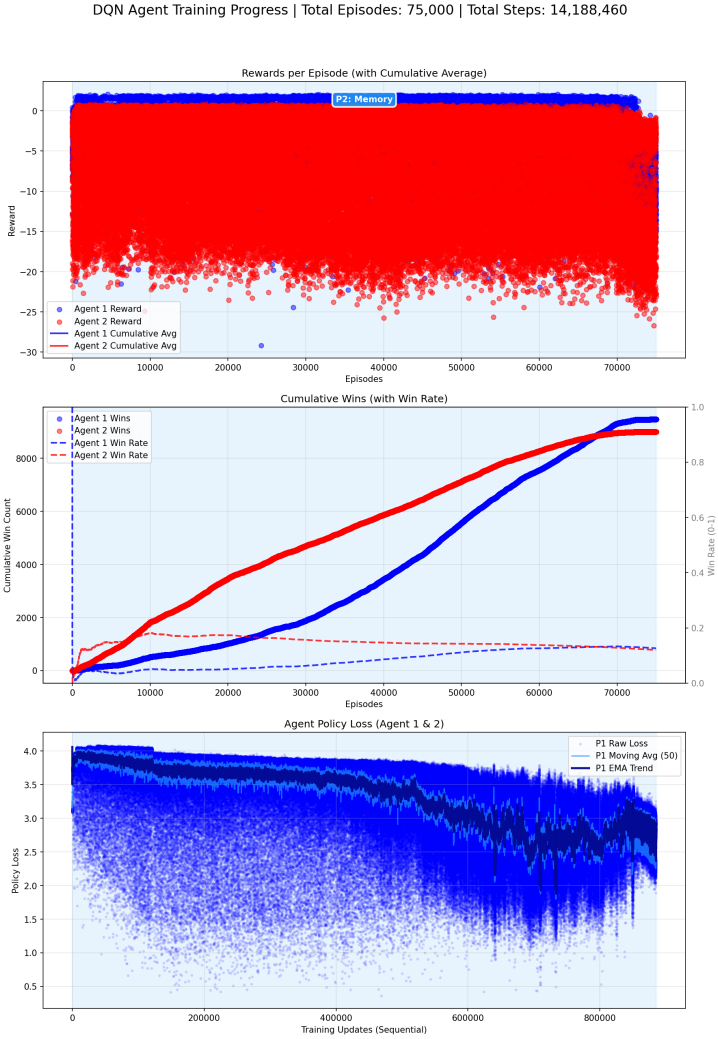


Figure B.17: Rainbow DQN+AAREN Extended Training Progress at 75,047 Episodes

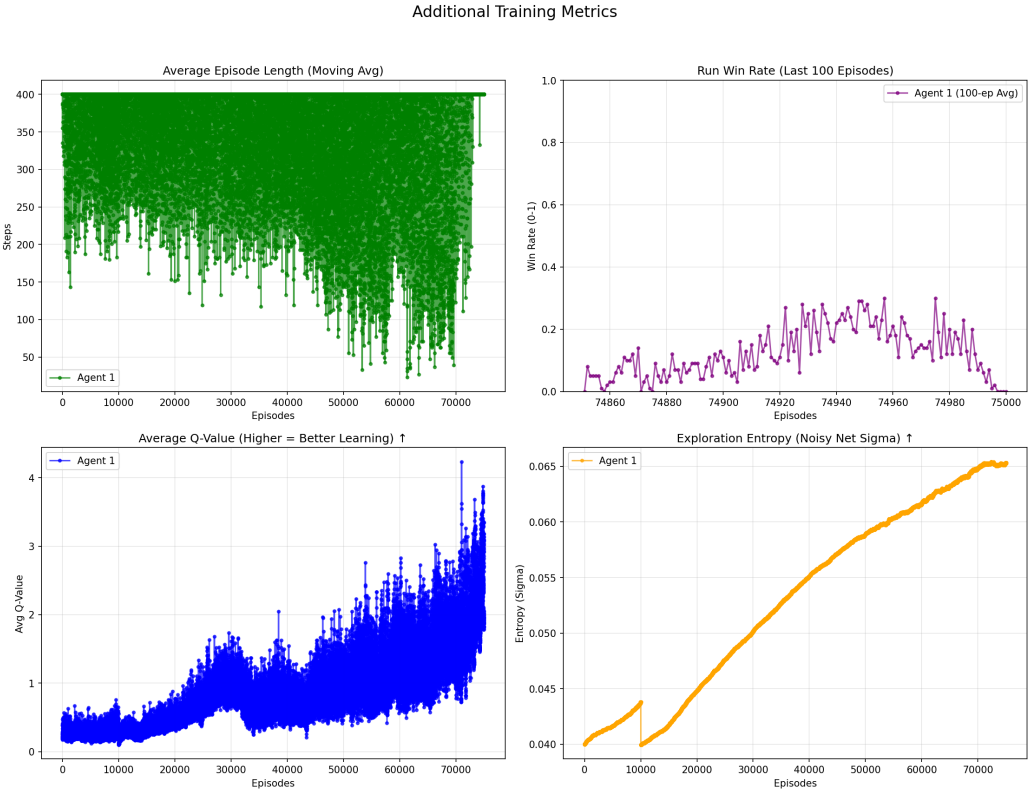


Figure B.18: Rainbow DQN+AAREN Extended Additional Metrics at 75,047 Episodes

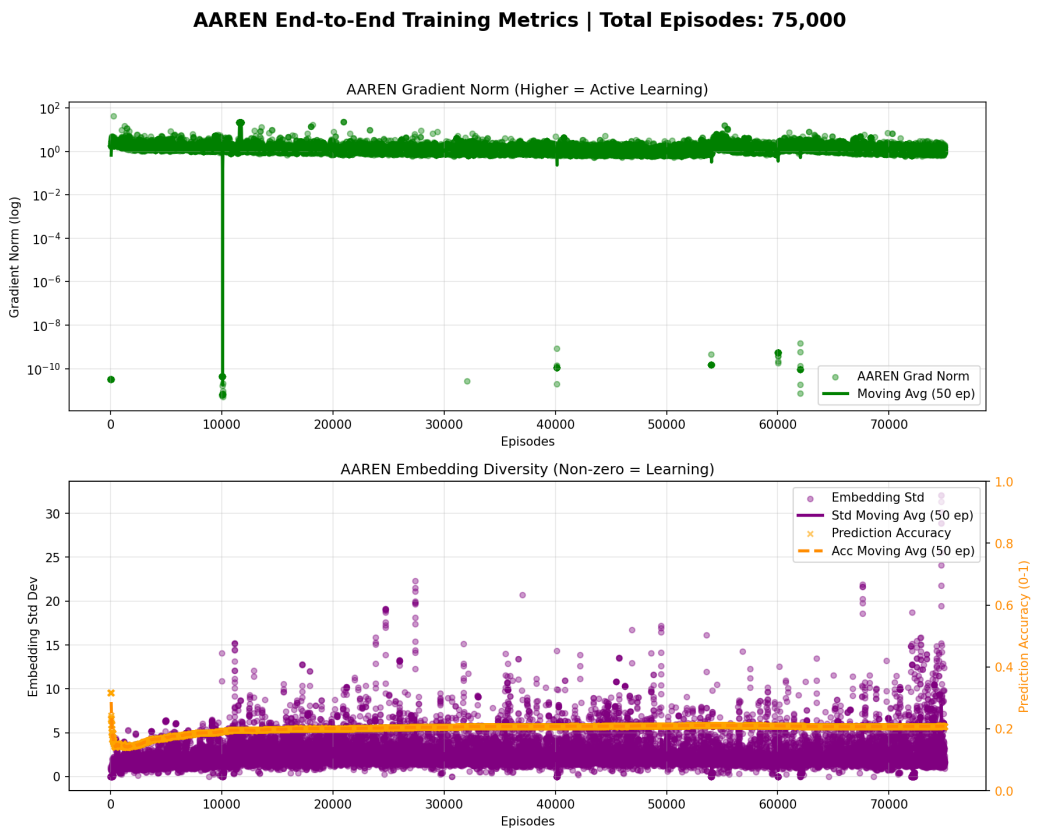


Figure B.19: AAREN End-to-End Training Metrics at 75,047 Episodes

B.3 AI Transparency, Inclusion, and Originality of Intent

The development of the MARQ framework for Stratego was governed by intentionality and human-led architectural design. While large language models and generative AI have become ubiquitous in modern software development, their role in this research was strictly limited to tasks such as grammatical refinement, high-level brainstorming, and code debugging for modules and networking. Every core component of the MARQ system, including the AAREN history aggregator and the ResNet-based policy heads, was researched, evaluated, and implemented with the intent of mastering imperfect information games in the Stratego domain.

Appendix C

Implementation Log and Lessons Learned

This appendix records the changes made to the training system after the thesis experiments, each with the evidence that motivated it and what it taught. Entries are kept in the order the work was done, and every entry follows the same pattern: the change, why it was made, and the lesson. New entries are added at the end.

C.1 Narrowing the C51 support to ± 2

Change. The value support of the distributional head [7] was narrowed from $[-10, 10]$ to $[-2, 2]$ (51 atoms, so 0.08 per atom instead of 0.4).

Why. A return is the outcome ($+1$, -1 or the draw reward -0.5) plus the potential-based shaping $\gamma\Phi(s') - \Phi(s)$ [42], which telescopes over a game to $-\Phi(s_0)$. With $\Phi \in [-0.4, 1]$ every return therefore lies in $[-2, 1.4]$; the returns observed in the formal_2 run spanned -1.54 to 0.71. The ± 10 support was inherited from the thesis configuration, whose shaping was not potential-based. On it, the shaping of a capture (about 0.02 to 0.05) was a tenth of an atom: the head could hardly represent the difference between a good and a bad exchange.

Lesson. The support of a distributional critic is part of the reward design, not a default. It should be derived from the bounds of the return (here the telescoping of the shaping makes that exact) and checked against the returns a run actually produces.

C.2 Measuring the average Q-value on the actions taken

Change. The logged "average Q" is now the expected value of the action that was taken in each sampled replay transition.

Why. The thesis metric took the maximum over all 3600 actions. Nearly all of them are illegal in any given state and are never trained, so the maximum was an artefact: it read about $+2$ while the returns were about -1.

Lesson. With a large, mostly illegal action space, any statistic taken over all actions measures the untrained part of the network. Diagnostics must be restricted to actions that can occur.

C.3 Diagnosing a flat value function

Observation. After thousands of episodes of the formal_2 run the agent still drew most games, and its Q-values barely distinguished good moves from bad ones.

Diagnosis. Two probes separated the possible causes. A *clean-signal probe* trained the same network on synthetic transitions in which only the combat outcome determined the reward; it learned the difference quickly, so the learner itself (loss, target, optimiser) was sound. A *same-state probe* then compared, in real positions, the values of a winning and a losing attack available in the same state. In the formal_3 run at episode 1250 the winning attack was preferred in 68% of such positions, but by a margin of 0.003, against a true difference of about 0.05. In real training the combat signal is small next to the noise: a winning attack is worth $+0.019$ of shaping and a losing one -0.033, while an ordinary move's shaping varies with a standard deviation of 0.02, and the outcome arrives hundreds of moves later.

Lesson. When a reinforcement learner does not learn, first test the learner on a problem whose answer is known; if it passes, the problem is in the signal (reward scale, support, credit assignment), not in the optimisation. Probes that compare actions *within the same state* are far more informative than averages over states.

Operational note. A run that ends with exit code 1 and no Python traceback was killed from outside the process (here by memory pressure), not by a bug in the training code.

C.4 Making the training loop faster

Changes.

- **Batched evaluation.** The evaluation games now run side by side, with one network call per side per round instead of one call per position. An evaluation of $40 + 40$ games went from 363 s to 143 s.
- **Cheaper replay.** The C51 loss needs the distribution of the action taken only, and the target needs that of the Double-DQN action only. The network now returns raw logits; the two needed actions are gathered first and normalised afterwards, instead of normalising all 3600×51 outputs. One update went from 128.7 ms to 97.5 ms.
- **More lanes at a constant replay ratio.** The loop used to perform at most one gradient update per iteration, so adding parallel games silently lowered the number of updates per move. It now performs one update per `REPLAY_UPDATE_INTERVAL` learner moves however many lanes moved, which allowed 16 lanes instead of 8 (acting cost per move fell from 2.0 ms to 1.33 ms).

Lesson. Profile before optimising: replay was half of the step time and evaluation was serial, while the environment was below 10%. And a "speed" setting such as the number of lanes can change the learning algorithm (here the replay ratio) unless the loop is written so that it cannot.

C.5 Removing the curriculum's environment changes

Change. The five-phase curriculum no longer changes the game. Every stage is played under fog of war, with the same turn limit (1500), the same no-battle draw rule (200 plies, also used in evaluation and the GUI) and the same, constant shaping weight. The stages remain only as an opponent schedule: random and scripted opponents first, then the stronger heuristic, then self-play and the league, advancing on evaluation gates as before.

Why. The thesis curriculum [9] changed the environment under the network at every transition. Phase 1 had full observability (the agent saw every enemy rank), and the shaping weight fell from 1.0 to 0.5, 0.2 and finally 0. Each of these changes the optimal values of the states the network has already learned: a value learned with the enemy's ranks visible is wrong under fog of war, and cutting the shaping weight changes every return. The network must then relearn what it knew, with the replay memory still full of transitions from the old rules, which is the setting in which catastrophic forgetting [33] occurs. Annealing the shaping also buys nothing: potential-based shaping leaves the optimal policy unchanged [42], so there is no bias to remove. A smaller inconsistency was the draw rule: training drew after 100 plies without a battle and evaluation after 200, so the agent was graded under a rule it had not trained under.

Lesson. A curriculum should make the *task* easier (weaker opponents), not change the *rules* the values refer to. Anything the network's targets depend on (observability, reward, termination) must stay fixed across stages, and evaluation must use the same rules as training.

C.6 Cold start from demonstrations

Change. Before its first game the agent now learns from games in which the scripted heuristic plays its side, following Deep Q-learning from Demonstrations (DQfD) [27]; the idea of a short supervised "cold start" before reinforcement learning is also central to recent reasoning-model training [17].

1. The heuristic plays 100 games against the random and heuristic opponents under the training rules. Its transitions are recorded exactly as the agent's own are (move plus reply, PBRS shaping plus outcome, 5-step returns, mirror augmentation) into a separate demonstration

buffer that the agent's own experience never overwrites. Each one also keeps the legal moves of its state.

2. The network is pre-trained on the demonstrations alone (5000 updates).

3. During training 10% of each batch comes from the demonstrations, falling linearly to 0% by episode 5000.

4. Demonstrated transitions also get the large-margin loss of DQfD,

$$J_E(Q) = \max_{a \in \mathcal{A}(s)} \big[Q(s,a) + \ell(a_E, a)\big] - Q(s, a_E), \qquad \ell(a_E, a) = \begin{cases} 0 & a = a_E \\ 0.1 & a \neq a_E \end{cases}$$

with the maximum over the legal moves $\mathcal{A}(s)$ only, added to the C51 loss with weight 1.

The earlier "imitation" option, which let the heuristic choose 10% of the agent's moves in its first 2000 games, is switched off.

Why. A network trained from zero starts from random play, which against these opponents mostly ends in draws: at episode 1000 the formal_3 run drew 32 of 40 evaluation games against the random opponent. Its first thousands of games therefore carry almost no outcome signal. The imitation option did not fix this: the heuristic's moves entered the replay memory like the agent's own, so the C51 loss learned what they were worth but nothing told the network to prefer them over the moves that were not played. The margin loss does exactly that. In a unit test with equal returns for every move, the C51 loss alone left the greedy move at chance, while with the margin loss the demonstrated move became the greedy one in at least 90% of states. The margin is 0.1 (1.25 atoms of the ± 2 support), comparable to the value of a capture; DQfD used 0.8 with rewards clipped to ± 1. The demonstration share decays because the heuristic is a weak demonstrator, not an expert, and should not cap the agent.

Result. The 100 demonstration games took about three minutes; the heuristic won about half of them (52 wins, 26 losses, 22 draws in one collection) and they produced 38 742 transitions, mirror copies included. Pre-training costs about 107 ms per update, so the 5000 updates take about nine minutes. Whether the cold start speeds up learning has not yet been measured in a full run.

C.7 The formal_3 run and the start of formal_4

formal_3. The formal_3 run trained from zero with the ±2 support but still with the curriculum's environment changes and without the cold start. It was stopped at episode 3500, never having left Phase 1. Its evaluations (40 greedy games per opponent every 500 episodes) did not improve; against the random opponent they got worse:

Episode	vs random (W/L/D)	vs heuristic (W/L/D)
500	8/14/18	1/37/2
1000	7/1/32	1/28/11
1500	6/2/32	1/32/7
2000	0/4/36	0/29/11
2500	5/8/27	2/30/8
3000	3/5/32	0/35/5
3500	1/3/36	0/35/5

The agent learned to avoid losing to the random opponent mainly by drawing, and it never won more than 2 of 40 games against the heuristic. Its games carried almost no outcome signal, which is the problem the cold start addresses.

formal_4. formal_4 starts from zero with every change above: the opponent schedule without environment changes, the cold start, the asynchronous learner and 16 lanes. Its cold start played the 100 demonstration games in 84 s (the heuristic won 56, lost 24 and drew 20) and stored 35 544 transitions. The 5000 pre-training updates took 271 s (54 ms per update with the GPU not shared with another run), and the large-margin loss fell to 0.068, below the margin of 0.1.

formal_4, first 1000 episodes. The evaluations (40 greedy games per opponent) were, next to formal_3 at the same episodes:

	formal_4		formal_3	
Episode	vs random	vs heuristic	vs random	vs heuristic
500	16/16/8	10/17/13	8/14/18	1/37/2
1000	14/22/4	8/21/11	7/1/32	1/28/11

(wins/losses/draws). Against the heuristic formal_4 won 8 to 10 games where formal_3 won one, and it no longer drew most games against the random opponent. But it lost more than half of them instead, almost always by losing all its movable pieces, and it did not improve from episode 500 to 1000, nor by episode 2500 (vs random 12/23/5, 12/25/3 and 15/22/3 at episodes 1500, 2000 and 2500; vs heuristic 4/14/22, 5/19/16 and 6/16/18). formal_4 also ran about 1.4 times as fast (1000 episodes in 45 minutes, against about 64 for formal_3).

Diagnosis: attacks on revealed pieces. A probe played 30 greedy games against the random opponent under the training rules and classified every attack by whether the attacked piece's rank was already known to the attacker:

Player 1	attacks per game	hidden target: won / lost	revealed target: won / lost
heuristic (the demonstrator)	39.1	39% / 47%	77% / 0%
formal_4, episode 250	41.7	45% / 45%	26% / 62%
formal_4, episode 500	41.0	42% / 43%	27% / 63%
formal_4, episode 1000	39.8	43% / 42%	28% / 63%
formal_3, episode 1000	20.2	36% / 48%	22% / 72%

Against hidden pieces the agent does as well as the heuristic. Against pieces whose rank it can see (the ranks of revealed enemy pieces are in the observation, and a test checks that they reach the network) it loses most of its attacks, while the heuristic never attacks a known piece it cannot beat. The cold start transferred how *often* the heuristic attacks (about 40 attacks per game, twice formal_3's), but not *which* known pieces it attacks, and this was so from episode 250 on, so it was never learned rather than learned and forgotten.

Test: pre-training on the demonstrations alone. To see whether more imitation would teach the rule, a separate copy of the network was pre-trained on a fresh set of 100 demonstration games (35 198 transitions) with the cold start's losses, for 20 000 updates instead of 5000, and probed in the same way. The agreement is the share of demonstration states in which the network's greedy legal move is the heuristic's:

Updates	agreement	margin loss	revealed target: won / lost	hidden target: won / lost
0	8.4%	–	–	–
5000	63.1%	0.072	28% / 60%	41% / 45%
10 000	75.7%	0.051	28% / 60%	44% / 42%
20 000	88.2%	0.027	31% / 62%	42% / 44%

The network came to reproduce the heuristic's move in 88% of the demonstration states, yet in its own games it lost as many attacks on revealed pieces as before. It also did worse against pieces it could see than against hidden ones, as a player would that picks its targets without regard to their visible rank: the enemy pieces that are revealed are mostly ones that survived an earlier battle, so they are stronger than average. Fitting the demonstrations, even closely, did not produce the rule that makes the heuristic's attacks good. (The agreement is measured on the demonstrations the network was trained on, so part of it may be memorisation.)

Test: does the value of an attack respond to the target's rank? In 300 real positions (random play, fog of war) where a piece of rank Miner to General could attack an adjacent enemy piece, the observation was edited at the target square only, showing the target as a revealed Scout (the attack wins), a revealed Marshal (the attack loses) or as hidden, and the network's value of that attack was compared across the three. The same edit was also applied while reading the value of a quiet move far from the target, as a control.

Network	Q vs Scout	Q vs Marshal	Q vs hidden	Scout − Marshal (positive in)
untrained	−0.023	−0.025	−0.022	+0.002 (51%)
formal_3, episode 3500	−0.642	−0.644	−0.628	+0.001 (46%)
formal_4, episode 2500	+0.055	+0.048	+0.187	+0.007 (50%)
pre-trained, 20 000 updates	−0.140	−0.172	+0.074	+0.032 (59%)

The control moved by less than 0.01 in every case. The networks trained by reinforcement value an attack on a revealed Scout and on a revealed Marshal the same, no more apart than the untrained network does, although one wins a piece and the other loses one. They do respond to *whether* the target is revealed (formal_4 values attacking a hidden piece at +0.19 and a revealed one at about +0.05), but not to *which* rank is revealed. The pre-trained network shows a small preference (+0.03),

with a spread across positions (standard deviation 0.12) four times as large. The environment keeps a revealed piece's rank visible when it moves, and the ranks are in the observation, so the information is there: it is the training signal that has not made the networks use it. This is the combat signal of the earlier diagnosis: the shaping difference between winning and losing an exchange (a few hundredths) is no larger than the variation of an ordinary move's shaping, and the outcome comes hundreds of moves later.

Measurement: the combat signal of the potential. The shaping of Player-1 transitions (its move plus the reply, as the trainer builds them; terminal transitions left out) was measured in 30 games against the random opponent, with the heuristic and with a random player as Player 1, for the current potential and for its material term alone, $\Phi_{\text{mat}} = (\text{own} - \text{enemy material})/148$:

	current Φ	material only
quiet move, mean / sd (random Player 1)	+0.0008 / 0.015	0.0000 / 0.0001
quiet move, mean (heuristic Player 1)	+0.0065	0.0000
attack on a revealed piece, won − lost (random)	0.043	0.061
in standard deviations of a quiet move	2.9	about 430
in atoms of the ±2 support	0.53	0.76
attack on a hidden piece, won − lost (heuristic)	0.093	0.058

Under the current potential an ordinary move is not neutral: the proximity and penetration terms reward advancing, so for the heuristic a quiet move is worth +0.0065 on average, and beating a revealed piece only +0.011. Every 5-step return also adds four more transitions of this variation (standard deviation about 0.02 each) to the difference between a won and a lost exchange of about 0.04. With the material term alone, a move without a battle is worth exactly nothing (up to the discount), so the only shaping left in a return is that of the battles in it, and a won exchange differs from a lost one by 0.06, about three-quarters of an atom. The information and proximity terms also explain why the networks prefer attacking hidden pieces: revealing a piece raises the information term whatever the result.

Lesson. A shaping potential with several heuristic terms makes every move carry reward, and that variation is noise to the decision the agent most needs to learn. Before tuning a learner, measure

the reward difference between the decisions that matter, relative to the variation of the rest of the reward stream, and in the units the critic resolves.

Lesson. Imitation transfers the frequent, easy-to-see features of a demonstrator's play (here, attacking a lot) before the conditional rule behind them (attack a known piece only if it is weaker), and a high agreement with the demonstrator does not show that the rule was learned. A demonstration is worth checking for the specific decisions it was meant to teach, with a probe that classifies those decisions in the agent's own games, not only by win rate or by agreement on the demonstrations.

Lesson. An evaluation that only counts wins hides how a policy fails: formal_3's win rate against the random opponent stayed low throughout, but its draws rising from 18 to 36 of 40 showed that it was converging to passive play rather than failing to learn at all.

Lesson. Demonstrations stored as ordinary experience teach values, not preferences; to make a value learner imitate, the loss must rank the demonstrated action above the alternatives. And with a large, mostly illegal action space, that ranking has to be restricted to the legal moves, or it spends its effort pushing down actions that can never be played.

C.8 A material-only potential (formal_5)

Change. The weights of the shaping potential's four terms (material, penetration, information, proximity) became a setting, `POTENTIAL_WEIGHTS`, and were changed from $(0.4, 0.15, 0.1, 0.35)$ to $(1, 0, 0, 0)$:

$$\Phi(s) = \frac{\text{own material} - \text{enemy material}}{148},$$

with a hidden enemy piece still counted at the mean army rank of 3.7. Its range is $[-1, 1]$, and the support test now reads the range from the weights. Φ is 0 at the start of a game (both armies have 148 material, the hidden one counted at the mean), so the shaping of a whole game telescopes to zero, and a return still lies within the outcome reward plus or minus 1, inside the ± 2 support. Everything else is formal_4's configuration, including the cold start and the asynchronous learner. formal_4 was stopped at episode 3000; its last evaluation had 17 wins, 17 losses and 6 draws against the random opponent and 3 wins, 24 losses and 13 draws against the heuristic, with no gain since episode 1500.

Why. The measurement above: under the old potential an ordinary move carried a shaping reward that varied as much as the difference between winning and losing an exchange on a revealed piece, and the network did not learn to tell those exchanges apart. With material only, a move without a battle carries no shaping, and a won exchange is worth about three-quarters of an atom more than a lost one. The cost is that nothing rewards advancing or scouting any more until a battle happens; whether the agent still finds the enemy, or stalls into no-battle draws, is what formal_5 has to show.

Result, first 1000 episodes. The cold start went as in formal_4 (the heuristic won 58, lost 20 and drew 22 of the 100 demonstration games; the margin loss fell to 0.071). The evaluations were 20/16/4 and 9/30/1 against the random opponent and 3/18/19 and 9/19/12 against the heuristic at episodes 500 and 1000 (wins/losses/draws), against 16/16/8 and 14/22/4, 10/17/13 and 8/21/11 for formal_4. The agent did not stall: no-battle draws were 74 and 94 of the training games in the two blocks of 500, and games lasted 340 to 346 moves. It still lost mostly by running out of movable pieces (197 and 198 of 500 games), as formal_4 did. So far the material-only potential has not changed how the agent loses. Two evaluations 500 episodes apart also differed by 11 wins out of 40 against the same opponent, which shows how wide the noise of a 40-game evaluation is.

Result, to episode 2750. formal_5 was stopped at episode 2750 to start formal_6 (Section C.15) with the fixes found in the meantime. Its later evaluations were 22/15/3, 17/22/1 and 16/20/4 against the random opponent and 9/24/7, 8/20/12 and 7/14/19 against the heuristic at episodes 1500, 2000 and 2500. formal_4 had 12, 12 and 15 wins against the random opponent and 4, 5 and 6 against the heuristic at the same episodes. The material-only potential won a few more games against the heuristic, but by less than the noise of 40-game evaluations (about 3 wins), and neither run passed a gate. The potential was not the bottleneck it was suspected to be; the faults of the next sections were found while formal_5 ran.

Result, the revealed-piece probes. The two probes of formal_4 were repeated on the networks of formal_5 (episode 2750) and formal_4 (episode 3000). Both have 30 input planes, so they were run under the formal_6 code with the agent built for 30 planes and given the first 30 planes of the new encoding, whose layout is unchanged. Each played 30 greedy games as Player 1 against the random opponent:

	formal_5, ep. 2750	formal_4, ep. 3000	heuristic
games won / lost / drawn	17 / 13 / 0	14 / 16 / 0	23 / 4 / 3
attacks on revealed pieces	378	339	270
won / tied / lost	28 / 9 / 63%	30 / 11 / 59%	77 / 23 / 0%
attacks on hidden pieces, won / lost	43 / 43%	44 / 42%	39 / 47%

A network whose checkpoint failed to load (a fresh one, through the silent fallback of `load_model`) lost 66 to 67% of its attacks on revealed pieces in the same games, so on this decision both trained networks are close to an untrained one. In the rank-swap test (300 positions, the target shown as a revealed Scout or a revealed Marshal, everything else unchanged) formal_5's value of the attack rose by +0.0016 for the Scout, positive in 49% of positions, and formal_4's by +0.0011, positive in 51%, against a spread of the attack's value across positions of 0.06 to 0.09. With a won exchange worth about three-quarters of an atom more than a lost one in every return that contains it, 2750 episodes of material-only shaping did not make the network read the revealed rank at all. A reward signal that clear and that frequent should have been learnt, so the cause has to be sought in the learner or in what reaches it rather than in the reward. None of the faults of the next sections has yet been shown to explain it.

Lesson. When a change aimed at one failure leaves that failure exactly where it was, measure the failure directly before reading the win rates: the probe settled in minutes what three noisy evaluations could not. And a diagnostic that loads a checkpoint must fail loudly when the load fails; here a silent fallback to a fresh network gave plausible-looking numbers.

C.9 Asynchronous learner

Change. The gradient updates now run on a background thread with its own CUDA stream while the main loop chooses moves and steps the parallel games. This is the actor-learner split of Ape-X [28] and IMPALA [21], also used for large-scale asynchronous reinforcement learning of language models [24], here within a single process:

- The replay ratio is unchanged: the loop still owes one update per `REPLAY_UPDATE_INTERVAL` learner moves and hands them to the learner. Once more than `ASYNC_MAX_LAG` = 2 updates

are owed it waits (back-pressure), so the policy that acts is at most three updates behind the one being trained.

- Acting reads a copy of the network that is refreshed after every update. The learner must not share its network with the actor: the optimiser changes the weights in place, and NoisyNet acting switches the network's noise and train/eval mode.
- The replay memory, which both threads use, is behind a lock. Because the learner runs on its own CUDA stream, it waits for the actor's stream before sampling and finishes its reads before releasing the lock; otherwise a transition evicted by the actor could be freed by the allocator while the learner's kernels were still reading it.
- Anything that reads or replaces the learner's weights or optimiser state (checkpoints, copying the network into the second agent, the learning-rate schedule) pauses the learner first, and an error in the learner thread is re-raised in the training loop.

Result. In two otherwise identical short runs (8 lanes, the cold start on, measured from the first gradient update over about 10 000 learner moves, on a GPU shared with another training run) the synchronous trainer made 43.4 learner moves per second and the asynchronous one 59.3, i.e. 1.37 times as fast. Both performed exactly 1313 gradient updates, so the replay ratio was the same.

Lesson. Asynchrony is only safe if every piece of shared state is accounted for: the weights (a separate acting copy), the replay memory (a lock), GPU memory on two streams (stream ordering and synchronisation), and the replay ratio (back-pressure). The last one matters most for reproducibility: without back-pressure an asynchronous learner changes the algorithm, not just its speed.

C.10 What the falling loss of formal_4 did and did not show

Observation. Over the 3000 episodes of formal_4 the C51 loss of Player 1 fell steadily while its evaluations did not improve. Per block of 500 training episodes (about 11 500 gradient updates each):

Episodes	mean loss	mean Q (action taken)	mean return	training games won (random / heuristic)
0–500	3.63	−0.01	−0.66	34% / 16%
500–1000	3.42	−0.02	−0.59	39% / 18%
1000–1500	3.32	−0.06	−0.63	34% / 18%
1500–2000	3.21	−0.14	−0.60	36% / 20%
2000–2500	3.11	−0.22	−0.57	39% / 21%
2500–3000	3.05	−0.28	−0.51	43% / 22%

The greedy evaluations over the same span won 40, 35, 30, 30, 38 and 43% of the 40 games against the random opponent and 25, 20, 10, 13, 15 and 8% against the heuristic (episodes 500 to 3000). The run had the AAREN history planes switched off (`AAREN_ENABLED` false: 30 input planes), so it says nothing about AAREN.

Reading. The loss is the cross-entropy between the predicted return distribution and a target built from the network's own target copy; a distribution spread evenly over the 51 atoms scores $\ln 51 = 3.93$, about where the cold start left it (3.93). Most of the fall is the network making its distributions sharper and moving its values towards what it actually gets: the mean Q of the actions taken went from −0.01 to −0.28, towards the mean return of about −0.55, i.e. it learned more accurately that it was losing. Across blocks of 100 episodes the loss and the training win rate correlated at −0.49, but the loss correlated with time at −0.97; once a linear trend was removed from both, the correlation was −0.14. The training games (with NoisyNet exploration on) did improve somewhat against the random opponent, from 34% to 43% of about 215 games per block, roughly two standard errors; the greedy evaluations of 40 games did not show it.

Lesson. A falling TD or distributional loss shows that the critic fits its own moving targets better, not that the policy plays better; with a pessimistic, well-calibrated critic it falls while the agent keeps losing. Two quantities that both drift with training time correlate whatever their relation, so a loss–performance correlation should be computed on detrended (or differenced) curves before it is read as coupling. Progress has to be judged on evaluations against fixed opponents, with enough games for the difference to exceed its noise.

C.11 Correcting the claims of the thesis chapters

Change. The Methodology and Results chapters were revised where the code or the data did not support what they said. No reported number was changed; the readings of them were.

- AAREN was described as trained end to end with the Q-network through a "gradient bridge", $\epsilon \cdot W_{proj}(\mathbf{0})$ added to the stored embeddings. An affine layer maps the zero vector to its bias, so the DQN loss reached only one of AAREN's 21 parameter tensors, and none that depends on the move history. AAREN was trained, in effect, only by its supervised loss on revealed pieces, and the text now says so. In the thesis code its embedding was also taken at the padding step of every shorter history rather than at the history's last real step.

- The self-play win rate of 49.88% (against 24.97% for the DQN) was read as the agent doubling its strength. In self-play both sides are the same policy, and in both runs the decisive games split almost evenly (24.97/24.99% and 49.88/43.43%). The win rate doubled because the decisive share did (50% to 93%), and steps per win follows from that too.

- The loss–win correlation of -0.787 was read as the loss becoming a reliable proxy for play. It was computed on raw windowed curves. formal_4 shows the raw correlation (-0.49) shrinking to -0.14 once a linear trend is removed from both curves, while its evaluations did not improve.

- AAREN's 24.23% accuracy was compared only with a uniform 8.3% baseline over 12 types. Scouts are 8 of the 40 pieces and 8 of the 33 movable ones, so always naming the most common type would score about 20 to 24%.

- The logged average Q-values (0.70, 0.84, 1.84) were read as value resolution. They were the maximum over all 3600 actions, the artefact of the second entry of this appendix.

The validation table now reads "Not tested", "Not supported", "Inconclusive", "Supported" or "Self-play only" instead of "Validated" for every row. The Methodology chapter now lists all six Rainbow components and separates the thesis configuration from that of the formal runs.

Why. Re-reading formal_4 against the chapters (previous entry). That run had AAREN switched off, yet the chapters presented the falling loss and the self-play results as validation of both the Rainbow learner and AAREN.

Lesson. A metric from symmetric self-play measures the draw rate, not strength; a correlation between two curves that both drift with time measures the drift; an accuracy needs the majority-class baseline, not the uniform one; and a claimed gradient path should be checked by listing which parameters actually receive a gradient. Each of these was cheap to check and reversed a claimed validation.

C.12 Replay priorities from the cross-entropy

Observation. The prioritized replay [26] sets a transition's priority to $(\ell + 10^{-6})^{0.6}$, with ℓ the C51 loss of that transition, which is the cross-entropy $-\sum_i m_i \log p_i$ between the projected target m and the prediction p. The cross-entropy is the KL divergence plus the entropy of the target, $\mathrm{CE} = \mathrm{KL}(m \,\|\, p) + H(m)$. Rainbow prioritizes by the KL divergence, the part that measures how wrong the prediction is. $H(m)$ is large for every non-terminal transition, because its target is the network's own diffuse prediction one step on. It is at most $\ln 2$ for a terminal one, whose target is the outcome itself, split over the two nearest atoms.

Measurement. formal_4's network of episode 3000 played 32 greedy games (against the random opponent and the heuristic, 8 won, 16 lost and 8 drawn). They were recorded through the trainer's 5-step and mirroring pipeline, which gave 13 328 transitions; the rewards came from the current material-only potential. For each transition:

Transitions	count	CE	$H(m)$	KL
terminal	320	4.15	0.51	3.64
non-terminal, return with a battle	6790	3.76	3.73	0.02
non-terminal, no battle	6218	3.42	3.40	0.02

With the cross-entropy, the 95th percentile of the priority was 1.2 times the 5th. The terminal transitions, 2.4% of the memory, held 2.6% of the total priority, so they were sampled as often as under uniform replay. With the KL divergence the ratio would be 10.3. The terminal transitions would hold 40% of the priority, about 16 times their share.

The terminal KL of 3.64 also shows that, one move before the end of a game, the network's distribution put almost no mass near the actual outcome. Those 2.4% of transitions are the only

ones whose target is the game result rather than the network's own estimate. Under the cross-entropy priority the replay did nothing to make them count more.

A second, smaller fault: the 2.5-fold priority boost for battles is decided by the newest step of a 5-step window. It therefore lands on the transition that *starts* four moves before the battle, not on the one that starts with the attack. The mirrored copy of a transition gets no boost at all.

Change. Made for formal_6 (Section C.15); at the time of this entry, none. The proposed fix is to compute the priority from $\mathrm{CE} - H(m)$, and to decide the battle boost by the first step of the window. Whether this changes play is not yet measured in a run.

Lesson. A priority must measure surprise, not the size of the loss. With a distributional loss the two differ by the entropy of the target, which here was almost the whole loss and made prioritized replay uniform. The distribution of the priorities is cheap to check (the ratio of high to low percentiles, and the share held by the transitions that carry the outcome), and it should be checked once whenever the loss changes.

C.13 An audit of the rest of the Rainbow learner

Observation. Before changing the replay priorities, the rest of the learner was read and, where a doubt could be measured, probed on formal_4's network of episode 3000.

- *Checked and correct:* the C51 projection, the Double DQN target over legal moves, the dueling combination, the factorised NoisyNet layers, the per-lane 5-step buffers (mirroring is applied after the n-step sum, so the two never mix), the flushing of open returns at the end of a game, the move-plus-reply transitions, the asynchronous learner's locking and weight copy, and the replay ratio (about 8 samples per stored transition, the usual DQN ratio).

- *Instance normalisation and constant planes.* The backbone normalises every channel over the board, and planes 27–29 (observability, no-battle counter, game progress) are constant over the board, so they might have been normalised away. They are not. In 300 fog-of-war positions, raising the no-battle counter from 0 to 0.99 lowered the network's value by 0.15 on average, as it should with a draw costing -0.5, and game progress by 0.11. Removing one own piece

changed it by 0.02. A fresh network learnt a target from the counter plane equally well with and without the normalisation layers (correlation 0.99 in both cases).

- *Dropout in the target.* The attention block has dropout 0.1 and the target network is never put in evaluation mode, so its targets carry dropout noise. Dropout alone moved the value of the greedy move by a standard deviation of 0.015, more than the NoisyNet noise (0.010), and about a third of the spread of the values across a position's legal moves (0.043).

- *How much NoisyNet explores.* With the acting noise on, the move played differed from the greedy move in 4 to 6% of positions (100 and 300 positions, 8 and 20 noise samples).

Change. Made for formal_6 (Section C.15); at the time of this entry, none. The proposed changes are the evaluation mode for the target network (or no dropout in the attention block) and the priority fix of the previous section.

Lesson. A suspicion about an architecture is cheap to test: perturb the input it concerns and see whether the output moves, or check whether a fresh network can fit a target that depends only on it. Here the plausible suspicion (normalisation erasing the counters) was wrong, and two quieter faults (an always-on dropout in the target, and priorities that do not prioritise) were real.

C.14 What AAREN can add to the observation

Why it is off. The formal runs set `AAREN_ENABLED` to false, so the network sees the 30 observation planes only. No document records the reason; the code enforces two constraints. The cold start refuses AAREN, because a demonstration has no history of its own and zero planes would teach the network what a game without history looks like. The asynchronous learner also refuses it, because AAREN's supervised training shares the network. The plan was to make the Rainbow learner work first and change one thing at a time.

Measurement. The heuristic played 225 games as Player 1, alternately against the random opponent and the heuristic. A history aggregator tracked Player 2's moves and labelled them at battles exactly as the trainer does.

- *The "has moved" bit.* Of Player 1's attacks on a piece that had never moved, 41% (1472 attacks, against random) and 49% (1028, against the heuristic) hit a Bomb or the Flag. Of its attacks on a piece that had moved, none did (2503 and 1712 attacks). The observation does not contain this bit, and the game state does not record which pieces have moved.

- *Piece type from the history.* AAREN labels only pieces that moved at least twice. Its 6216 labelled reveals were split by game, 80% for training and 20% for testing, and a fresh AAREN was trained for 4000 batches. On the 1272 test reveals it named the type correctly in 35.6% of cases. Always naming the most common type (Lieutenant) gave 16.3%, and "a long move means a Scout, otherwise the most common type" gave 23.9%. Its accuracy was the same against the random opponent (34.9%) as against the heuristic (36.0%). Since the random opponent moves without regard to rank, most of what AAREN reads must be where a piece stood in the heuristic setup, rather than how it behaves. A setup prior is real information against this opponent pool, but it would shrink against varied setups.

Reading. AAREN does carry information the observation lacks: it beats both simple rules on held-out games. The largest single piece of history information, however, is whether a hidden piece has moved at all. That bit separates a 45% chance of attacking a Bomb or the Flag from none, and it needs one observation plane rather than a 64-plane embedding. The thesis accuracy of 24.23% is close to the long-move rule here. Whether AAREN helps the *agent* is a separate question that only paired runs can answer, with and without it, on the same evaluations.

Change. Made for formal_6 (Section C.15); at the time of this entry, none. Proposed order: add a "has moved" plane for enemy pieces (and mark a Scout revealed by a long move), then compare AAREN on and off. The cold start and the asynchronous learner would have to support AAREN for that comparison.

Lesson. Before adding a learned module to supply missing information, find out what the information is and how much of it a hand-made feature already captures. Here one bit carried most of it, and a module trained on revealed pieces never sees the unmoved ones that bit is about.

C.15 Moved-piece planes and three learner fixes (formal_6)

Change. Four changes, made together before formal_6:

- *Two observation planes, 32 in all.* The environment now records, for every square, whether the piece on it has moved and whether it has ever moved more than one square at once, which only a Scout can do. Both facts are public: the opponent sees every move. The flags travel with the piece, including when it wins a battle, and are cleared when it is taken. Plane 30 marks the enemy pieces of unknown rank that have moved, so they are not a Bomb or the Flag. Plane 31 marks those that have made a long move, so they are Scouts. The GUI bot and the browser player build the same planes, and tests replay games in both and compare them plane by plane with the training environment.

- *Replay priorities from the KL divergence.* A transition's priority is now $(\mathrm{KL}(m \,\|\, p) + 10^{-6})^{0.6}$, with $\mathrm{KL} = \mathrm{CE} - H(m)$, as in Rainbow [26], instead of the cross-entropy. The loss itself is unchanged.

- *The battle boost on the battle move.* The 2.5-fold priority of a battle now goes to the 5-step transition whose *first* move was the attack. The mirrored copy of a transition now gets the same boosts as the original, for battles and for wins.

- *No dropout in the target.* The target network is put in evaluation mode whenever it is copied, so its targets carry no dropout. Its NoisyNet noise stays on, as in Rainbow.

Everything else is formal_5's configuration: material-only potential, cold start, asynchronous learner, 16 lanes, 20 000 episodes. The first convolution now reads 32 planes, so checkpoints of formal_4 and formal_5 do not fit the new network; formal_6 starts from zero like them.

Why. The three preceding sections. The "has moved" bit separated a 41 to 49% chance that an attacked hidden piece was a Bomb or the Flag from none, and the observation did not contain it. Under the cross-entropy priority the terminal transitions, the only ones whose target is the game result, were sampled no more often than under uniform replay; under the KL divergence they would hold about 16 times their share of the priority. The battle boost of the thesis code landed four moves before the attack, and dropout moved the target value of the greedy move by more than the NoisyNet noise did.

Result, cold start. The cold start ran as before: the heuristic won 50, lost 27 and drew 23 of its 100 demonstration games (39 406 transitions), and 5000 pre-training updates took the loss from 4.13 to 3.95 and the margin loss to 0.076. The checks set for formal_6 were: whether attacks on unmoved hidden pieces, and so losses to Bombs, become rarer than in formal_4 and formal_5; whether the Q-value one move before the end of a game moves towards the outcome (formal_4's network put almost no mass there, a KL of 3.64); the spread of the priorities in the replay memory, which should now be wide rather than flat; and whether the network finally reads the rank of a revealed piece (the rank-swap test and the share of attacks on revealed pieces it loses, 59 to 63% for formal_4 and formal_5 against 0% for the heuristic).

Result, to episode 9000. The greedy evaluations of stage 1 (episodes 500 to 5000, 40 games each) won 17, 18, 10, 18, 11, 23, 14, 18, 15 and 13 games against the random opponent and 6, 5, 6, 13, 2, 7, 11, 8, 4 and 5 against the heuristic: no trend, and no better than formal_4 or formal_5. No gate was passed. At episode 5000 the stage's episode cap moved the opponent schedule to stage 2, whose training opponents are the frozen and the smart heuristic only; the random opponent and the plain heuristic drop out. The evaluations against the heuristic then won 7, 6, 7, 1, 5, 4, 1 and 7 of 40 (episodes 5500 to 9000). In the training games of stage 2 the agent won about 17% against the frozen heuristic and drew about 80% against the smart one, and games grew from about 350 to 550 moves, most of them ending in no-battle draws.

The probes of the previous runs were repeated on the networks of episodes 5000 and 9000, in 30 greedy games against the random opponent (and 300 positions for the rank-swap test):

	ep. 5000	ep. 9000	heuristic
games won / lost / drawn	11 / 15 / 4	0 / 3 / 27	23 / 4 / 3
attacks per game	34	9	39
attacks on revealed pieces lost	66%	73%	0%
rank-swap: Scout − Marshal, positive in	48%	33%	–
share of attacks on unmoved hidden pieces	27%	0%	31%
of which Bomb or Flag	44%	–	40%

formal_5's network (episode 2750, 30 planes) put 30% of its attacks on unmoved hidden pieces, 37% of them Bombs or the Flag. So by episode 5000 the moved-piece planes had not changed whom the agent attacks, and it still did not read the revealed rank (the Scout and Marshal versions of an

attack differed by −0.001 on average). By episode 9000 it no longer attacked unmoved pieces at all, but it attacked hardly anything: it drew 27 of 30 games against the random opponent, which formal_6 had beaten 11 times at episode 5000. Its value of an attack was about −0.45 whatever the target, close to the −0.5 of a draw.

The C51 loss reported by the trainer fell from 2.47 (first 500 episodes) to 0.38 (last 500), far below the 3.0 to 3.7 of formal_4 and formal_5. The reported loss is the cross-entropy weighted by the importance-sampling weights of the prioritized replay, and those weights change with the priorities, so the number is not comparable across the change of priority and shows nothing about play on its own.

Reading. Two separate things. In stage 1, the moved-piece planes and the learner fixes changed nothing that the probes measure: the network still does not learn the outcome of a battle from what it can see, the failure of formal_4 and formal_5. In stage 2, a change of opponents alone, with the environment unchanged, turned an agent that beat the random opponent about a third of the time into one that only draws: the agent trained against a pool in which it mostly draws (the smart heuristic) or loses (the frozen one), its values moved to the value of a draw, and it stopped attacking. This is the forgetting that the removal of the curriculum's environment changes was meant to avoid, caused here by the opponent schedule that was kept.

Lesson. An episode cap that moves an agent to the next stage of an opponent schedule when it has not passed the gate turns the schedule into a fixed timetable, and dropping the weak opponents from the pool removes the games the agent can win; with only draws and losses left, a value learner learns that nothing it does matters and plays for the draw. Keep the earlier opponents in the pool, and evaluate against them throughout, so that forgetting shows up when it starts. (formal_7 kept them, and the collapse happened anyway, so dropping them was not the cause: Section C.17.)

Lesson. When a run is stopped to fix several faults at once, the next run cannot say which fix mattered; it can only say whether they did together. Here that trade was accepted because each fix restores the standard algorithm rather than adding something new, and the paired comparison that matters more (AAREN on and off) still has to be run with one difference.

C.16 Why the network does not learn the rank of a revealed piece

Observation. In every formal run the agent lost most of its attacks on enemy pieces whose rank it could see (59 to 73%), and its value of an attack did not depend on that rank (Section C.15). Three explanations were possible: the information does not reach the training targets, the network cannot represent the rule, or the training signal is too weak for the learner to pick up. Three probes on formal_6's network of episode 5000 separate them.

Measurement 1: the targets. The network played 32 greedy games (6 won, 22 lost, 4 drawn) through the trainer's pipeline, which stored 11 426 transitions. Each one was classed by its first move; an attack on a revealed piece was classed won, tied or lost by the battle rule. For the non-terminal ones:

First move	count	share	5-step reward	$\mathbb{E}[m]$	network Q
quiet move	9524	83.4%	−0.016	−0.110	−0.108
attack on a hidden piece	1304	11.4%	−0.002	−0.035	−0.017
attack on a revealed piece, won	146	1.3%	+0.023	−0.040	−0.079
attack on a revealed piece, lost	388	3.4%	−0.027	−0.104	−0.074

The targets do carry the outcome of the exchange: their means differ by 0.064 between won and lost attacks. The network's Q differs by -0.005. The agent also chose attacks on revealed pieces that it lost 2.7 times as often as ones it won. Each target distribution m is wide, with a standard deviation of 0.77 for these transitions, because it carries the uncertainty of an outcome hundreds of moves away. The target *means* of one class varied much less across transitions (0.13).

Measurement 2: capacity. A fresh network of the same size was trained on 3267 positions from these games. In each, one attack on an adjacent enemy piece was chosen, and the target square was edited to a revealed piece of random rank. The C51 target was a function of the battle outcome $o \in \{1, 0, -1\}$ of that attack alone. Every batch of 128 consisted of such attacks. The measure is the gap between the network's values of the attack when the target is shown as a Scout (a win) and as a Marshal (a loss), over 300 held-out positions:

Target	l.r.	Scout > Marshal	gap (ideal)
$0.5\,o$, no noise	$3 \cdot 10^{-5}$	100%	0.44 (1.00)
$0.5\,o$, no noise	10^{-3}	100%	0.97 (1.00)
$0.07\,o$ plus sampled noise of s.d. 0.77	$3 \cdot 10^{-5}$	68%	0.003 (0.14)
the same with signal $0.35\,o$	$3 \cdot 10^{-5}$	94%	0.023 (0.70)
the same with 12 "attack wins / ties / loses" planes	$3 \cdot 10^{-5}$	72%	0.005 (0.14)
distributions of s.d. 0.77, mean $0.07\,o$ plus noise of s.d. 0.13	$3 \cdot 10^{-5}$	99%	0.028 (0.14)
the same plus a squared error on the mean	$3 \cdot 10^{-5}$	99%	0.052 (0.14)

(After 2000 batches. "Scout > Marshal" is the share of held-out positions in which the attack is valued higher with the target shown as a Scout.) With a clean target the network learnt the rule within 1000 batches, even at the run's learning rate, so it can read a revealed rank and compare it with the attacker's. Planes that state the outcome of every possible attack outright hardly helped, so finding the relation is not the difficulty either. What decides is the size of the signal relative to the width of the target. The win–loss difference of the material potential is under one atom of the support (0.08 of ± 2 over 51 atoms) and a twelfth of the target's width. Under the cross-entropy, a shift of the mean that small changes the loss by little, so it is learnt slowly. With targets shaped like the real ones, 2000 batches of nothing but these attacks learnt a fifth of the gap. Adding a squared error between the predicted and target means doubled that.

Reading. The rank information reaches the network, and the network can use it. The training signal for using it is small and diluted. Attacks on revealed pieces were 5% of the stored transitions, so the real learner sees about 6 in a batch of 128. Those 6 compete with everything else the network fits, against a target network that changes every 1000 updates. The probes show the signal is learnable when concentrated. That dilution and the moving targets account for the rest is an inference, not a measurement. This is also why the earlier fixes did not change the behaviour. The material-only potential made the signal clean, but no larger than a fraction of an atom. The moved-piece planes and the priority fixes do not touch it. Nor would AAREN: it supplies information about hidden pieces, while this failure concerns pieces that are already revealed.

Change. The first and third were made for formal_7 (Section C.17). The options the probes support were, by cost: an expected-value loss beside the C51 loss (it doubled the learnt gap); a larger weight on the material potential with a wider value support (a five-fold signal got the ordering right in 94% of positions); and a priority boost for attacks on revealed pieces, which raises their share of the batch.

Lesson. A distributional critic learns differences in the mean slowly when they are small next to the width of the return distribution, even when they are exact. Before relying on a reward term to teach a decision, compare that term's difference between the right and the wrong choice with the spread of the targets. Then check, on a fresh network trained on that decision alone, how much of the difference it learns in a realistic number of updates.

C.17 A mean loss, revealed attacks in the batch, and the opponent pool (formal_7)

Change. formal_6 ended at episode 11 680, when its process was killed from outside (its log shows no error; it was started from the desktop app's shell, and formal_7 is started detached from it). Its evaluations against the heuristic from episode 9500 on were 5, 6, 8, 5 and 5 wins of 40, with no gate passed. formal_7 starts from zero with formal_6's configuration and three changes:

- *A loss on the mean.* The squared error between the means of the predicted and the target distribution, $\left(\sum_i p_i z_i - \sum_i m_i z_i\right)^2$ with weight 1, is added to each transition's C51 cross-entropy. Both are weighted by the importance-sampling weight. The priorities are still the KL divergence. The loss reported by the trainer now includes this term, so it is not comparable with that of earlier runs.

- *Attacks on revealed pieces in the batch.* A transition whose first move attacks an enemy piece of known rank, and its mirror image, keeps four times its priority through every priority update. The importance-sampling weights leave this factor uncorrected, so these transitions count four times as much in the loss, not just four times as often. At about 5% of the replay, they become about 17% of each batch.

- *The opponent pool after stage 1.* Stages 2 and 3 keep the random opponent and the heuristic

in their pools beside the frozen and smart heuristics (or the self-play snapshot). The usual weighting (more games against the opponents it beats less often) still gives the random opponent at least 15%. Every evaluation now also plays the random opponent, so forgetting shows as soon as it starts. Stage 1 and the episode caps are unchanged.

Why. Section C.16. The win–loss difference of an exchange is present in the targets but is a twelfth of their width, so the cross-entropy learns it slowly. A mean loss doubled the learned gap in the probe. The attacks that carry the difference were 5% of each batch. Section C.15: once the stage-1 cap moved formal_6 to a pool it could only draw or lose against, it stopped attacking.

Result, to episode 2500. The cold start ran as in formal_6 (the heuristic won 53, lost 27 and drew 20 of the 100 games; 5000 pre-training updates, loss 4.17 to 3.94 with the mean term, margin loss 0.074). Wins in the 40-game evaluations, with formal_6 at the same episodes in brackets:

Episode	500	1000	1500	2000	2500
random	16 (17)	13 (18)	12 (10)	23 (18)	17 (11)
heuristic	4 (6)	4 (5)	10 (6)	7 (13)	7 (2)

Summed over the five evaluations, formal_7 won 81 games against the random opponent to formal_6's 74, and 32 against the heuristic to 32. One evaluation of 40 games has a standard error of about 3 wins at these rates, so stage 1 shows no difference yet. That is expected: two of the three changes act on stage 2 and later, and the mean loss acts on a signal the evaluations measure only indirectly. The test is still the rank-swap probe and the share of revealed attacks lost, and, after episode 5000, whether the evaluations against the random opponent hold up.

Result, the revealed-rank probe at episodes 2500 and 3500. Each checkpoint of both runs played 32 greedy games (same seed). The attacks on revealed pieces in them were classed by the battle rule, and the rank swap of Section C.16 was run on one fixed set of 300 positions for all four checkpoints:

	formal_6, 2500	formal_7, 2500	formal_6, 3500	formal_7, 3500
attacks on revealed pieces	478	484	512	470
won / tied / lost	26 / 10 / 64%	32 / 6 / 62%	30 / 9 / 62%	29 / 8 / 63%
Scout > Marshal	65%	64%	66%	67%
mean gap	0.021	0.020	0.032	0.022
$\mathbb{E}[m]$ gap, won − lost	0.088	0.111	0.101	0.115

By episode 3500 (about 650 000 updates) the two runs do not differ. Both still lose about two-thirds of the attacks they choose on pieces whose rank they can see, and both rank the Scout above the Marshal in two-thirds of the positions, with a gap of a fifth of the target's 0.1. The targets carry the difference as before. So the mean loss and the four-fold priority have not yet made the network learn it faster in the real run. The probe had predicted twice the learning speed on a batch made only of these attacks. In the run they are about a sixth of the batch, and the targets move. The value of every attack was also lower in formal_7 at episode 3500 (-0.22 to -0.26, against -0.05 to -0.12 in formal_6). That checkpoint comes after the resume, which emptied the replay of its own games, so the cause is not clear.

Result, to episode 5500: stage 2 collapses as before. Stage 1 ended at its cap of 5000 episodes. Its last evaluations were 15, 18 and 17 wins of 40 against the random opponent and 7, 6 and 6 against the heuristic (episodes 4000 to 5000), as in formal_6. 500 episodes into stage 2, the greedy agent drew 37 of 40 games against the random opponent, won none and lost 3. It won 7 of 40 against the heuristic. Episodes slowed from about 1.9 to 6.9 seconds, as the games ran on to the draw rules. The random opponent was in the stage-2 pool: it got 112 of the 500 training games. The agent won the first of them about half the time and then stopped winning against it. So keeping the opponents it can beat in the pool did not prevent the collapse; Section C.15's explanation, that the agent forgot them, is wrong.

Only the opponents change at the stage switch: the game, its limits and the shaping stay the same. A reading that fits the numbers is that stage 2 made avoiding battle the better policy. Against the frozen and smart heuristics, three-quarters of the stage-2 pool, the agent won about 15% and lost about 70% of its training games. A game it fought out was then worth about $0.15 - 0.70 = -0.55$, no more than a draw (-0.5). An exchange it lost also cost material shaping at once, while avoiding battle cost nothing until the game ended. The observation does not say which opponent the agent

is playing, so the same passive policy was played against the random opponent. The underlying weakness is the one measured above: it loses two-thirds of the attacks it chooses on pieces whose rank it can see. Until that changes, fighting does not pay against a competent opponent.

Lesson. When a curriculum moves the agent to opponents it mostly loses to, check the value of a draw against the value of playing on. If the draw is worth as much, the agent will learn to draw, and it will do so against every opponent it cannot tell apart from them.

The process was killed at about episode 3000 by a console window-close event (Section C.20). It was resumed from the checkpoint of episode 2750 with the networks, the target network, the optimiser and the curriculum state, but not the replay buffer. On resuming, the trainer played 100 new heuristic games (48 won, 24 lost, 28 drawn; 36 214 demonstration transitions) to refill it, and skipped the pre-training updates. Its own transitions of episodes 1 to 2750 are gone, so for a while after the resume the replay holds a larger share of demonstrations than it did before. Results after episode 2750 are of a resumed run.

Lesson. When a probe shows that a learner cannot pick up a small but exact signal, the fix should act on that signal, not on the agent's inputs. Two levers do: the loss, so the signal weighs more against the noise it competes with, and the data, so the cases that carry it are a larger share of each batch.

C.18 What the agent has to learn, and in what order

Observation. Read together, the formal runs show the agent stuck at one step of a sequence of skills that Stratego asks for. Each skill pays off only through the one before it, and the opponent schedule had assumed skills the agent did not have.

1. *The rules.* Which moves are legal. The action mask supplies this; nothing has to be learned.
2. *Engagement.* Moving pieces forward and making contact, instead of shuffling until a draw rule ends the game. Learned in every run since the draw was given a cost: in stage 1 the agent attacks, and it wins 30 to 58% of its evaluation games against the random opponent, about 40% on average.

3. *Comparing two known ranks.* Attacking a revealed piece it beats and leaving or fleeing one that beats it. This is the whole game under full information, and it needs no inference. It is *not* learned: from formal_4 to formal_7 the agent lost 59 to 73% of the attacks it chose on revealed pieces, and its values barely depend on the rank it sees (Sections C.16 and C.17). An agent with this skill should beat a random opponent nearly every time, because the random opponent offers its pieces at random. Winning 40% of those games is itself a sign that the skill is missing.

4. *Inference about hidden pieces.* A piece that has moved is neither Bomb nor Flag. A piece that moved more than one square is a Scout. A low piece can be sent to make a strong one reveal itself. formal_6 gave the network these facts as planes 30 and 31. This skill pays off only through skill 3: knowing more about a piece is worth something only to an agent that attacks the right pieces once it knows.

5. *Managing information.* Keeping its own strong pieces hidden, protecting revealed ones, and the Spy against the Marshal. It pays off by denying the opponent skills 3 and 4.

6. *Strategy.* Finding and defending the Flag, tempo, the setup and the endgame. Only the outcome rewards it.

Reading. The reward signal for each skill weakens down the list. The material potential rewards skill 3 at once, as each exchange is won or lost. Skill 4 gets no immediate reward; it is paid later, through better exchanges. Skill 6 is paid only by the outcome, hundreds of moves later. This is why skill 3 is the bottleneck. It has the most direct signal of the skills that have to be learned, and everything above it is built on it. (Why that signal is not learned is still open. It is not only its size: formal_8 made it two and a half times larger in the targets, and the network still did not respond to the rank; Section C.19.)

The opponent schedule assumed the skills in a different order. Stage 2's frozen and smart heuristics play skills 3 to 5 well, so playing them presumes the agent can already compare ranks. The stage-1 gates measured win rates against random and the heuristic, which mix every skill together. And the episode cap moved the agent on whether or not it had passed them. An agent without skill 3 that meets opponents with it loses most exchanges. It can then do no better than draw, which is what formal_6 and formal_7 learned to do.

Lesson. Name the skills a game requires, the order in which their rewards depend on each other, and a direct measure of each. Then advance a curriculum on the measure of the skill the next stage needs, not on a win rate that mixes them all. The direct measure of skill 3 is the share of attacks on revealed pieces that the agent loses in its own games. The evaluations report it from formal_8 on.

C.19 A five-fold material signal and stages without a cap (formal_8)

Change. formal_7 was stopped at episode 5690, in stage 2. formal_8 starts from zero with formal_7's configuration (mean loss, priority boost on attacks on revealed pieces, earlier opponents kept in the pool) and three changes:

- *The material potential is five times larger* (weight 5 instead of 1). The support of the value distribution is widened from ± 2 to ± 6 (0.24 per atom) so that it still covers every return (outcome plus $-\Phi(s_0)$ with $\Phi \in [-5, 5]$). A won exchange of average size now moves the target by about 0.3, more than an atom, against the same outcome noise as before.
- *No episode cap on any stage.* A stage ends only when its evaluation gates are passed (stage 1: 70% against the random opponent and 50% against the heuristic).
- *A direct measure of skill 3 in the evaluations.* Every evaluation game counts the agent's attacks on revealed pieces and how many it won or lost by the battle rule; the log shows the share lost beside each opponent's result.

Why. Section C.18. The capacity probe of Section C.16 put the rank-swap ordering right in 94% of positions when the signal was five times larger, against 68% at the run's size. Stage 1 is the only stage in which no run has collapsed, so the agent stays there until it has the skill the next stage requires.

Caveat. The shaping is potential-based, so it leaves the optimal policy unchanged [42]. The optimal shaped values are the unshaped ones minus $\Phi(s)$, a term that depends only on the state, so in the limit the gap between two actions is the same at any weight. A larger potential makes the exchange visible in the early targets, before the bootstrapped values have learned $-\Phi$. It speeds up

learning and does not change what is learned. This is the intended effect: the probes showed the agent learning skill 3 too slowly, not learning the wrong thing.

Result. The cold start ran as before (the heuristic won 51, lost 23 and drew 26 of the 100 games; 5000 pre-training updates, loss 4.40 to 4.00, margin loss 0.084). The run was stopped at episode 2386. Its evaluations, with formal_7 at the same episodes in brackets:

Episode	500	1000	1500	2000	sum
wins against random	10 (16)	11 (13)	18 (12)	18 (23)	57 (64)
wins against heuristic	4 (4)	12 (4)	5 (10)	5 (7)	26 (25)
revealed attacks lost, against random	63%	69%	62%	60%	
revealed attacks lost, against heuristic	77%	72%	72%	78%	

The wins do not differ from formal_7's. The new measure of skill 3 does not improve over the run. In each 40-game evaluation the agent attacked 450 to 510 revealed pieces against the random opponent and 160 to 180 against the heuristic, and it lost 60 to 78% of those attacks.

The probe of Section C.17 on the checkpoint of episode 2000 separates the target from what the network learned. The won–lost gap of the target means was 0.25 (about 0.1 in formal_7): the five-fold potential did make the signal in the targets two and a half times larger. The network's Q gap between won and lost attacks was 0.06. In the rank swap the network valued the attack higher on a revealed Scout than on a revealed Marshal in 53% of the 300 positions, with a mean gap of 0.003. That is no better than chance, and below formal_7's 66% at the same episode. (The probe ran with formal_8's support and potential. Only the 66% is comparable for the formal_7 checkpoint; its values are on the wrong scale.)

So the size of the signal relative to the target's width was not what kept the network from learning skill 3. Section C.16 explained the failure that way, from a fresh network trained on batches made only of these attacks; that network learned the rule. In the run, a signal two and a half times larger left the network's response to the rank at chance. Something that differs between the probe and the run stops it: the mix of the batch, the moving bootstrapped targets, the NoisyNet head, or the rest of the training pipeline. Which one is the next question. A direct test is to train the probe's fresh network on batches drawn from the run's own replay buffer, adding one difference at a time, and watch the rank swap.

Lesson. A probe that isolates one factor can show that the factor is *sufficient* to explain a failure, not that it is the explanation. The capacity probe showed a small signal learnt slowly; the run with a larger signal learnt nothing more. Before scaling a fix up to a full run, reproduce the failure inside the probe by adding back the run's conditions one at a time, until the probe fails as the run does.

C.20 Engineering notes

- formal_6 and formal_7 were both killed partway, with no Python error. formal_7's error log showed `forrtl: error (200): program aborting due to window-CLOSE event` once for each process. The Intel Fortran runtime that numpy's maths library loads installs a console handler that aborts the process when its console window closes. Starting the run detached from the app's shell was not enough, because the launcher still gave it a console. Runs are now started with `pythonw.exe`, which has no console, with the output sent to log files and `FOR_DISABLE_CONSOLE_CTRL_HANDLER=1` set. A long run should not own a console window that something else can close.

- An `import time` statement inside the training loop made `time` a local name for the whole training function, so any use of `time` before that line failed. Modules should be imported once, at the top of the file.

- A checkpoint loader that silently falls back to a fresh network turns a missing file into a misleading result: a probe of a checkpoint that did not exist yet reported the values of an untrained network. Diagnostic scripts now assert that the file exists.

- Most source files have Windows line endings. An edit script that matches multi-line text must normalise them first, or it silently matches nothing; a patch helper that asserts the number of matches made every such miss fail loudly instead.

- The first run of the capacity probe above passed an all-zero next-state distribution to the C51 projection for its terminal targets. The projection moves probability mass, so the targets were empty, the loss was zero, and no network learnt anything. Every configuration then gave the same numbers to three digits, which exposed it. A probe should assert that its targets are distributions (they sum to one) before trusting a negative result.

- A patch helper made tolerant of trailing whitespace also treated the end of a replaced block as optional whitespace, and so ate the indentation of the line that followed. Four source files no longer compiled. Compiling every edited file at once, before running any test, found all of them in one pass.

REFERENCES

[1] Adams, G., Padmanabhan, S., and Shekhar, S. Resolving implicit coordination in multi-agent deep rl with deep q-networks & game theory. *arXiv preprint arXiv:2012.09136* (2023).

[2] Al-Selwi, S. M., Hassan, M. F., Abdulkadir, S. J., Muneer, A., Sumiea, E. H., Alqushaibi, A., and Ragab, M. G. Rnn-lstm: From applications to modeling techniques and beyond—systematic review. *Journal of King Saud University - Computer and Information Sciences 36*, 5 (2024), 102068.

[3] Amaliani, R. N., Soewardikoen, D. W., Azhar, H., and Rahman, Y. Designing board games as an enhancer of a sense of community and cognition for the elderly of tresna werdha budi pertiwi social care. *Advances in Social Humanities Research 3*, 4 (2025).

[4] Author, F., and Author, S. Bridging local and global knowledge via transformer in board games. In *Proceedings of the International Joint Conference on Artificial Intelligence* (2025).

[5] Barzilai, G., Shamir, O., and Zamani, M. Are convex optimization curves convex? *arXiv preprint arXiv:2503.10138* (2025).

[6] Becker, T., and Sunberg, Z. Imperfect information games and counterfactual regret minimization in space domain awareness. In *Advanced Maui Optical and Space Surveillance Technologies Conference (AMOS)* (2022), AMOS Technologies.

[7] Bellemare, M. G., Dabney, W., and Munos, R. A distributional perspective on reinforcement learning. In *International Conference on Machine Learning* (2017), PMLR, pp. 449–458.

[8] Bellemare, M. G., Dabney, W., and Rowland, M. *Distributional Reinforcement Learning*. MIT Press, 2023.

[9] Bengio, Y., Louradour, J., Collobert, R., and Weston, J. Curriculum learning. In *International Conference on Machine Learning* (2009), pp. 41–48.

[10] Blüml, J., Czech, J., and Kersting, K. Alphaze**: Alphazero-like baselines for imperfect information games are surprisingly strong. *Frontiers in Artificial Intelligence Volume 6 - 2023* (2023).

[11] Brim, A. Deep reinforcement learning pairs trading with a double deep q-network. In *Proceedings of the 10th Annual Computing and Communication Workshop and Conference (CCWC)* (Las Vegas, Nevada, USA, Jan.–2020 2020), pp. 222–227.

[12] Brown, N., and Sandholm, T. Solving imperfect-information games via subgame solving and depth-limited search. *Artificial Intelligence 297* (2021), 103503.

[13] Canese, L., Cardarilli, G. C., Di Nunzio, L., Fazzolari, R., Giardino, D., Re, M., and Spanò, S. Multi-agent reinforcement learning: A review of challenges and applications. *Applied Sciences 11*, 11 (2021), 4948. Article number; MDPI journal uses article numbers instead of page ranges.

[14] Chen, S., Fried, D., and Topcu, U. Human-agent cooperation in games under incomplete information through natural language communication, 2024.

[15] Chitayat, P. P., Colman, A. M., Hyde, R. J., Drachen, A., and Cowling, P. Wards: Modelling the worth of vision in MOBA's. In *Intelligent Systems and Applications - Proceedings of the 2020 Intelligent Systems Conference (IntelliSys)* (2020), vol. 1251 of *Advances in Intelligent Systems and Computing*, Springer, pp. 55–76.

[16] Dbouk, H. M., Ghorayeb, K., Kassem, H., Hayek, H., Torrens, R., and Wells, O. Facility placement layout optimization. *Journal of Petroleum Science and Engineering 207* (2021), 109079.

[17] DeepSeek-AI. DeepSeek-R1: Incentivizing reasoning capability in LLMs via reinforcement learning. arXiv:2501.12948, 2025.

[18] Devlin, S., and Kudenko, D. Potential-based reward shaping in large-scale mdps. *Reinforcement Learning Journal 3*, 2 (2022), 145–167.

[19] Echchahed, A., and Castro, P. S. A survey of state representation learning for deep reinforcement learning. *Transactions on Machine Learning Research* (2025).

[20] El Shar, I., and Jiang, D. Weakly coupled deep q-networks. In *Advances in Neural Information Processing Systems 36* (2023), Curran Associates, Inc. NeurIPS 2023 Conference Track.

[21] Espeholt, L., Soyer, H., Munos, R., Simonyan, K., Mnih, V., Ward, T., Doron, Y., Firoiu, V., Harley, T., Dunning, I., Legg, S., and Kavukcuoglu, K. IMPALA: Scalable distributed deep-RL with importance weighted actor-learner architectures. In *International Conference on Machine Learning* (2018), PMLR.

[22] Feng, L., Tung, F., Hajimirsadeghi, H., Ahmed, M. O., Bengio, Y., and Mori, G. Attention as an rnn. *arXiv preprint arXiv:2405.13956* (2024).

[23] Fortunato, M., Azar, M. G., Piot, B., Menick, J., Osband, I., Graves, A., Mnih, V., Munos, R., Hassabis, D., Pietquin, O., et al. Noisy networks for exploration. In *International Conference on Learning Representations* (2018).

[24] Fu, W., Gao, J., Shen, X., others, and Wu, Y. AReaL: A large-scale asynchronous reinforcement learning system for language reasoning. arXiv:2505.24298, 2025.

[25] Harrington, J. Let's meetup? board game communities in hong kong. *Games and Culture 20*, 3 (2025), 279–297.

[26] Hessel, M., Modayil, J., van Hasselt, H., Schaul, T., Ostrovski, G., Dabney, W., Horgan, D., Piot, B., Azar, M., and Silver, D. Rainbow: Combining improvements in deep reinforcement learning. *Journal of Machine Learning Research 22*, 85 (2021), 1–52.

[27] Hester, T., Vecerik, M., Pietquin, O., Lanctot, M., Schaul, T., Piot, B., Horgan, D., Quan, J., Sendonaris, A., Osband, I., Dulac-Arnold, G., Agapiou, J., Leibo, J. Z., and Gruslys, A. Deep Q-learning from demonstrations. In *Proceedings of the AAAI Conference on Artificial Intelligence* (2018).

[28] Horgan, D., Quan, J., Budden, D., Barth-Maron, G., Hessel, M., van Hasselt, H., and Silver, D. Distributed prioritized experience replay. In *International Conference on Learning Representations* (2018).

[29] Hu, C., Zhao, Y., Wang, Z., Du, H., and Liu, J. Games for artificial intelligence research: A review and perspectives. *IEEE Transactions on Artificial Intelligence 5*, 12 (2024), 5949–5968.

[30] Huang, Y. Deep q-networks. In *Deep Reinforcement Learning: Fundamentals, Research, and Applications*, S. Z. Hao Dong, Zihan Ding, Ed. Springer Nature, 2020, ch. 4, pp. 135–160. `http://www.deepreinforcementlearningbook.org`.

[31] Kaur, A., Kumar, K., Prakash, A., and Tripathi, R. Imperfect csi-based resource management in cognitive iot networks: A deep recurrent reinforcement learning framework. *IEEE Transactions on Cognitive Communications and Networking 9*, 5 (2023), 1271–1281.

[32] Kensert, A., Libin, P., Desmet, G., and Cabooter, D. Deep reinforcement learning for the direct optimization of gradient separations in liquid chromatography. *Journal of Chromatography A 1720* (2024), 464768.

[33] Kirkpatrick, J., Pascanu, R., Rabinowitz, N., Veness, J., Desjardins, G., Rusu, A. A., et al. Overcoming catastrophic forgetting in neural networks. *Proceedings of the National Academy of Sciences 114*, 13 (2017), 3521–3526.

[34] Le, N., Rathour, V. S., Yamazaki, K., Luu, K., and Savvides, M. Deep reinforcement learning in computer vision: A comprehensive survey, 2021.

[35] Li, B., Fang, Z., and Huang, L. Rl-cfr: Improving action abstraction for imperfect information extensive-form games with reinforcement learning. *arXiv preprint arXiv:2403.04344* (2024).

[36] Li, J., Zanuttini, B., and Ventos, V. Opponent-model search in games with incomplete information. In *Proceedings of the AAAI Conference on Artificial Intelligence* (2024), vol. 38, pp. 9840–9847.

[37] Li, S., Zhang, Y., Wang, X., Xue, W., and An, B. Cfr-mix: Solving imperfect information extensive-form games with combinatorial action space. In *Proceedings of the Thirtieth International Joint Conference on Artificial Intelligence (IJCAI-21)* (2021), IJCAI, pp. 3663–3669.

[38] Li, X., and Zhang, Y. A survey of deep reinforcement learning algorithms for dueling network architectures. *Artificial Intelligence Review 55* (2022), 3153–3170.

[39] Li, Z., Ji, Q., Ling, X., and Liu, Q. A comprehensive review of multi-agent reinforcement learning in video games. *arXiv preprint arXiv:2509.03682v1* (2025).

[40] Luo, Z., Chen, Z., and Welsh, J. Multi-agent reinforcement learning with deep networks for diverse q-vectors. *arXiv preprint arXiv:2406.07848v1* (2024). Version v1, June 2024.

[41] Nair, R. R., Babu, T., Kishore, S., Pavithra, K., and Sumana, S. G. Improving alpha-beta pruning efficiency in chess engines. In *2024 International Conference on IT Innovation and Knowledge Discovery (ITIKD)* (Manama, Bahrain, 2025), IEEE, pp. 1–6.

[42] Ng, A. Y., Harada, D., and Russell, S. Policy invariance under reward transformations: Theory and application to reward shaping. In *International Conference on Machine Learning* (1999), pp. 278–287.

[43] Novikov, A., and Yanovskyi, V. Analysis of the decision-making algorithm efficiency in complex game environments on the example of pac-man. *Information Technologies and Computer Engineering 3*, 21 (2024), 108–118.

[44] Ouyang, X., and Zhou, T. Imperfect-information game ai agent based on reinforcement learning using tree search and a deep neural network. *Electronics 12*, 11 (2023), 2453.

[45] Petersen, C. The reality of the board game community: And the connection, identity and experience that creates it. Master's thesis, Eastern University, Aug. 2024.

[46] Pérolat, J., et al. Mastering the game of stratego with model-free multiagent reinforcement learning. *Science 378*, 6623 (2022), 990–996.

[47] Qu, T., Zhou, Q., Zhu, J., and Duan, F. Strategy optimization of imperfect information games based on nfsp with ddqn. In *Advances in Guidance, Navigation and Control*, L. Yan, H. Duan, and Y. Deng, Eds., vol. 845 of *Lecture Notes in Electrical Engineering*. Springer, Singapore, 2023, pp. 4376–4383.

[48] Saffidine, A., Finnsson, H., and Buro, M. Alpha-beta pruning for games with simultaneous moves. In *Proceedings of the AAAI Conference on Artificial Intelligence* (2021), vol. 26, pp. 556–562.

[49] Schmid, M., Moravčík, M., Burch, N., Kadlec, R., Davidson, J., Waugh, K., Bard, N., Timbers, F., Lanctot, M., Holland, G. Z., et al. Student of games: A unified learning algorithm for both perfect and imperfect information games. *Science Advances 9*, 46 (2023), eadg3256.

[50] Sokota, S., Vinitsky, E., Hu, H., Kolter, J. Z., and Farina, G. Superhuman ai for stratego using self-play reinforcement learning and test-time search, 2025.

[51] Sudhakar, A. V., Nekoei, H., Reymond, M., Liu, M., Rajendran, J., and Chandar, S. A generalist hanabi agent. *arXiv preprint arXiv:2503.14555v1* (2025).

[52] Wang, X., Li, Y., and Zhang, W. Deep reinforcement learning in imperfect-information games: A survey. *IEEE Transactions on Games 15*, 3 (2023), 421–438.

[53] Wang, Y. Game-theoretic understandings of multi-agent systems with multiple objectives, 2025.

[54] Wang, Y., Li, L., and Li, W. Constructing non-markovian decision process via history aggregator. *Applied Sciences 16*, 2 (2026), 955.

[55] Wang, Y., and Wellman, M. P. Regularization for strategy exploration in empirical game-theoretic analysis, 2023.

[56] Wang, Z., Schaul, T., Hessel, M., Hasselt, H., Lanctot, M., and De Freitas, N. Dueling network architectures for deep reinforcement learning. In *International conference on machine learning* (2016), PMLR, pp. 1995–2003.

[57] Wongkamjan, W., Gu, F., Wang, Y., Hermjakob, U., May, J., Stewart, B., Kummerfeld, J., Peskoff, D., and Boyd-Graber, J. More victories, less cooperation: Assessing cicero's diplomacy play. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)* (2024), Association for Computational Linguistics, p. 12423–12441.

[58] Xie, S., and Li, Z. Implicit bias of adamw: ℓ_∞ norm constrained optimization. *arXiv preprint arXiv:2404.04454v1* (2024).

[59] Xu, H., Li, K., Fu, H., Fu, Q., and Xing, J. Autocfr: Learning to design counterfactual regret minimization algorithms. In *Proceedings of the Thirty-Sixth AAAI Conference on Artificial Intelligence (AAAI-22)* (2022), Association for the Advancement of Artificial Intelligence.

[60] Xu, H., Li, K., Fu, H., FU, Q., Xing, J., and Cheng, J. Dynamic discounted counterfactual regret minimization. In *The Twelfth International Conference on Learning Representations* (2024).

[61] Yang, T., Tang, H., Bai, C., Liu, J., Hao, J., Meng, Z., Liu, P., and Wang, Z. Exploration in deep reinforcement learning: From single-agent to multiagent domain. *IEEE Transactions on Neural Networks and Learning Systems 34*, 10 (2023), 7065–7086.

[62] Zakharenkov, A., and Makarov, I. Deep reinforcement learning with dqn vs. ppo in vizdoom. In *2021 IEEE 21st International Symposium on Computational Intelligence and Informatics (CINTI)* (Budapest, Hungary, 2021), IEEE, pp. 131–136.

[63] Zhang, B., and Sandholm, T. Subgame solving without common knowledge. *Advances in Neural Information Processing Systems 34* (2021), 23993–24004.

[64] Zhang, B. H., and Sandholm, T. General search techniques without common knowledge for imperfect-information games, and application to superhuman fog of war chess. *arXiv preprint arXiv:2506.01242* (2025).

[65] Zhang, K., Yang, Z., and Başar, T. Multi-agent reinforcement learning: A selective overview of theories and algorithms. *Handbook of reinforcement learning and control* (2021), 321–384.

[66] Zheng, S., Trott, A., Srinivasa, S., Naik, N., Gruesbeck, M., Parkes, D. C., and Socher, R. The ai economist: Improving equality and productivity with ai-driven tax policies. *arXiv preprint arXiv:2004.13332* (2020).

[67] Zhu, K., Liu, Q., Yan, X., Wu, F., Zhao, X., Zhou, P., and Yang, X. A comprehensive survey of multi-agent reinforcement learning. *IEEE Transactions on Neural Networks and Learning Systems 34*, 7 (2022), 3074–3093.

VITA

James Gabriel P. Mabagos is BS Computer Science student of the Department of Computer Science at the Ateneo de Naga University.

Mark Lawrence M. Quibot is BS Computer Science student of the Department of Computer Science at the Ateneo de Naga University.